Algorithms for Data Science
Exam 2025
Group **A**

June 2025

Your Name: _____ **Your Student ID:** _____

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	5	4	4	6	5	9	5	4	4	5	5	4	0	60
Score:														

The last task gives 4 bonus points. Thus, you can obtain 60+4 points in total (regular+bonus points).

Check if your name and student ID are printed on top of every single page!

Write your name and your student ID on the first page!

Check whether you received all 14 pages (containing 13 tasks in total).

The regular time to solve the exam is 90 minutes.

1. [5 points] Consider the following incomplete Python function. The input is an array $a[0..n-1]$ and a target value x. The array is sorted reversely, that is, $a[0]$ is the largest value in a, and $a[n-1]$ the smallest one.The function outputs the maximum position i of the array such that all values in a from position 0 up to position i have value at least x. In other words, the function returns the maximum i such that, for all $0 \leq j \leq i$, we have $a[j] \geq x$.If $a[0]$ is smaller than x, then the function should return -1.

   Hint: the task is related to the first programming task.

```
1  def compute_max_pos(a, x):
2      n = len(a)
3      left = -1
4      right = n - 1
5      # ??? [Line 5] (Starts with while...)
6          # ??? [Line 6]
7          # ??? [Line 7] (Starts with if...)
8              # ??? [Line 8]
9          else:
10             # ??? [Line 10]
11
12     return left
```

   Complete the code by choosing a correct code snippet for each empty line (that starts with `# ???`). Do it by filling the blanks with the correct labels from the code snippets below:

   - Line 5: _____**G**_____
   - Line 6: _____**O**_____
   - Line 7: _____**A**_____
   - Line 8: _____**I**_____
   - Line 10: _____**C**_____

   Code snippets to choose from:

   - (A) `if x > a[p]:`
   - (B) `p = right - left`
   - (C) `left = p`
   - (D) `left = p + 1`
   - (E) `right = p`
   - (F) `right = p + 1`
   - (G) `while left < right:`
   - (H) `while left > right:`
   - (I) `right = p - 1`
   - (J) `if x < a[p]:`
   - (K) `if x == a[p]:`
   - (L) `p = (left + right + 1) * 2`
   - (M) `while left == right:`
   - (N) `left = p - 1`
   - (O) `p = (left + right + 1) // 2`

   Note: The indendation of your code is the same as of `# ???`*. In other words, your code for an empty line will start at the position of* `#`*.*

2. [4 points] Write a recurrence for the running time of the following algorithm `AlgoA(`$a[1..n]$`)` in the form of

$$T(n) = aT(n/b) + \Theta(f(n)) \ .$$

You don't need to solve the recurrence! And you don't need to understand what the algorithm computes!

The input of `AlgoA(`$a[1..n]$`)` *is an array* a *of* n *integers.*

The input of the auxiliary function `Copy_Sub_Array(`$a[1..n]$`,`i`,`m`)` *is an array* $a[1..n]$*, a position* i *and a length* m*. The function returns a copy of the subarray* $a[i+1..i+m]$*.*

```
AlgoA(a[1..n])
    if n ≥ 10 then
        ret = 0
        p = n · n · n · n
        for j = 1 to p · (n · n) do
            ret = ret + a[10]
        d = 3 · 3
        m = ⌈n/d⌉
        i = n
        while i > n − 9 do
            c = Copy_Sub_Array(a,i,m)
            ret = ret + AlgoA(c)
            i = i − 1
        for i = 1 to 5 do
            c = Copy_Sub_Array(a,i,m)
            ret = ret + AlgoA(c)
        return ret
    else
        return 7
```

```
Copy_Sub_Array(a[1..n],i,m)
    b[1..m] = new empty array
    for j = 1 to m do
        b[j] = a[i + j]
    return b
```

Your answer: $T(n) =$ $14T(\lceil n/9 \rceil) + \Theta(n^6)$

3. [4 points] Solve the following recurrence using the master theorem. Determine (for yourself) the values for a, b, and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = 64 \cdot T(n/2) + \Theta(n^6)$$

Your answers:

$c =$ **6**

Case: **B**

Resulting running time $T(n) = \Theta(n^6 \log n)$

4. Simplify the following running times as far as possible:

(a) [2 points] $\Theta(\ 9(\log_2 n)^3 + 7\log_2(n^5) + 2\ ) = \Theta(\ (\log_2 n)^3\ )$

(b) [2 points] $\Theta(\ 9\sqrt{n} + (n^{3/6+3/6})/7 + 1\ ) = \Theta(\ n\ )$

(c) [2 points] $\Theta(\ 22 \cdot n^6 + 36 \cdot 2^n + 3 \cdot 4^n\ ) = \Theta(\ 4^n\ )$

5. [5 points] Consider the following array $a[1..21]$ containing twenty-one elements:

$$a = \langle 105, 90, 100, 60, 85, 95, 40, 65, 25, 80, 45, 95, 75, 35, 30, 50, 55, 20, 15, 65, 70 \rangle$$

1. Draw the *complete* array as a complete binary tree (as defined for heaps),
2. Mark the edge whose endpoints (parent and child) violate the heap invariant,
3. Choose an appropriate index position j and **decrease** the value of $a[j]$ by the **maximum amount possible** such that a satisfies the heap invariant.

What index j do you choose?
(Write a single number between 1 *and* 21*; there is only one correct answer.)*

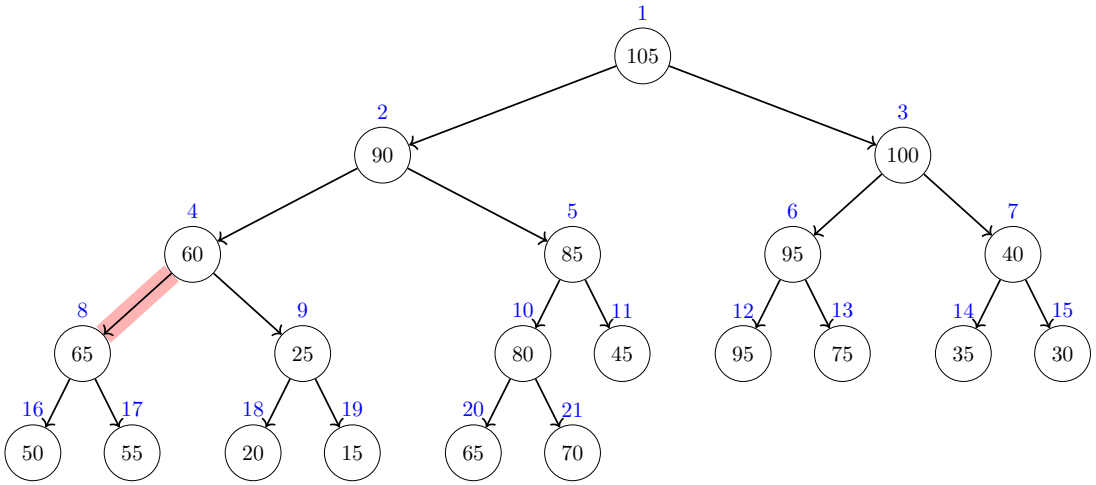
Your answer: $j =$ **8** ***(the violating edge's other endpoint is at* 4*.)***

What is the new value for $a[j]$?
(Write a single number, as small as possible and in particular smaller than the old value of $a[j]$*.)*

Your answer: $a[j] =$ **55** ***(the smallest value from* $[55, 60]$*)***

Your tree hould contain ***all* the elements of the array**, from $a[1] = 105$ up to $a[21] = 70$:

Solution:



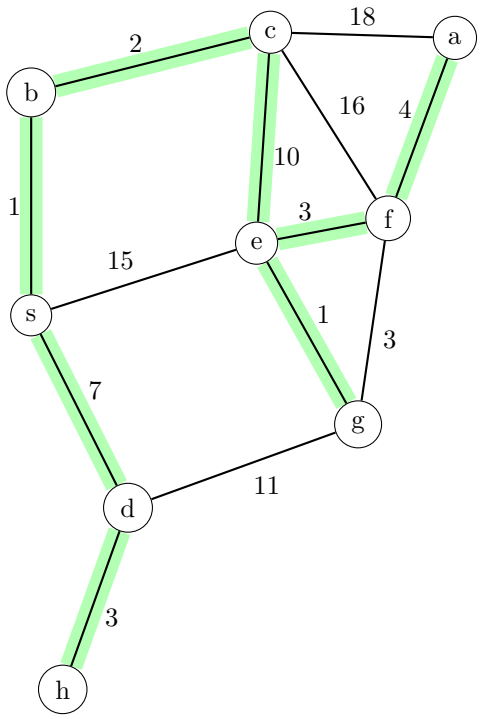
Explanation (not required): The heap invariant is violated at the red edge as the child (index **8***) has a larger value than its parent (index* **4***). To fix the heap invariant, it suffices to decrease* $a[8]$ *to any value between* 55 *and* 60*. The smallest possible value (as asked in the task) is thus* 55*.*

6. [9 points] Dijkstra's algorithm (see below) is run on the following graph where the start node is s.

   What are the values of the *dist* and *parent* tables when Dijkstra's algorithm has finished? Fill in the missing entries in the table below!

   Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```
dist[0..n − 1] filled with INF
parent[0..n − 1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time then
        for every edge (v, w) do
            if dist[w] > dist[v] + c_{v,w} then
                dist[w] = dist[v] + c_{v,w}
                parent[w] = v
                q.add(w, -dist[w])
```



Solution:

	a	b	c	d	e	f	g	h	s
distance	20	1	3	7	13	16	14	10	0
parent	f	s	b	s	c	e	e	d	NONE

Note:

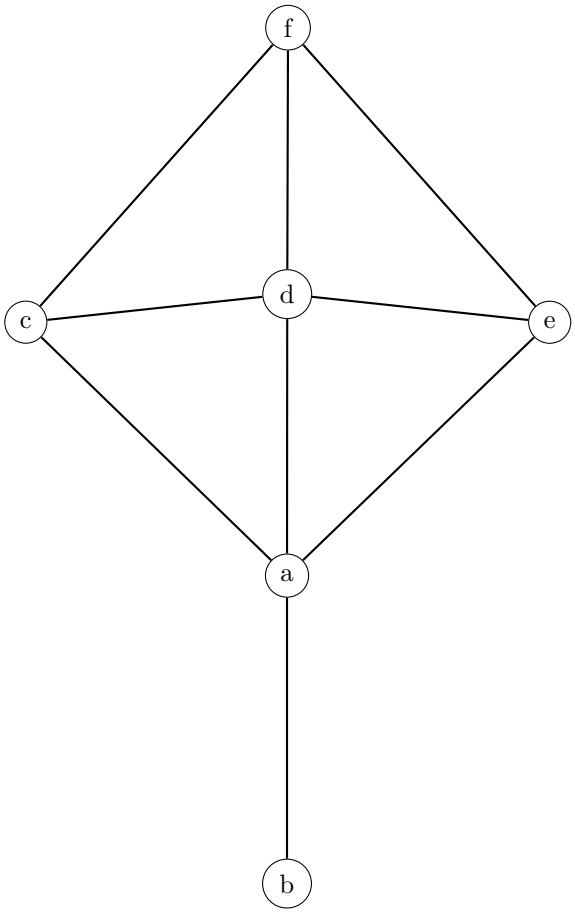
- *The entries of the parent table consist of the names of the nodes or the special value* `NONE`*. For example, the parent of some node could be* s *whereas the parent of* s *is* `NONE`*.*
- *If a vertex has more neighbors, we consider them* ***in lexicographic order*** *(that is, node* a *would come before node* b *and so on)!*

7. [5 points] Draw an (undirected) graph without edge crossings corresponding to the following adjacency matrix. For your drawing, just connect the given vertices below. Your edges don't need to be straight and are allowed to be curves.

   For your score, it's better to draw *all* of the edges even if they cross each other than not to draw one or more edges.

	a	b	c	d	e	f
a	0	1	1	1	1	0
b	1	0	0	0	0	0
c	1	0	0	1	0	1
d	1	0	1	0	1	1
e	1	0	0	1	0	1
f	0	0	1	1	1	0

Solution:



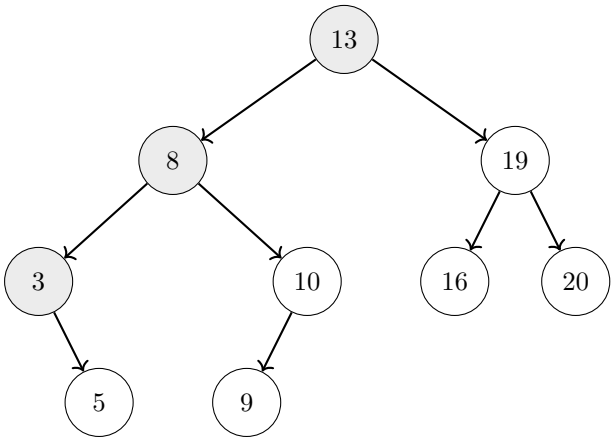
Note that a correct solution has to be crossing free! However, the depicted solution has been automatically drawn by a graph drawing program that does not guarantee that edges do not cross. Thus, if the depicted solution contains edge crossings, keep in mind that such a solution is not fully correct and you would not get all the points by drawing it. However, note that a crossing-free solution can be easily obtained from the depicted solution by redrawing one or more edges as curves around the graph.

8. [4 points] Add the following keys into the following binary search tree **in the order** as they are given below. It suffices that you draw how the binary search looks at the end with each key written in its respective node. (For simplicity, we consider only keys and no values.)

   Note: Do not change connections between previously added nodes! Just add the keys one by one in the given order to their correct positions.

   10, 5, 9, 19, 16, 20

Solution:



9. [4 points] What is the most appropriate data structure for the following scenario, chosen from the list below?

   Unless stated otherwise in the scenario, the goal is to minimize memory usage and the worst-case running time of each operation.

   In your answer, briefly explain how your chosen data structure applies to the scenario. Then give its memory and time complexity, and explain why it performs better than *every* other option, considering the scenario's specific constraints and goals.

   You are developing a system for a small embedded device that needs to store sensor readings, each identified by a unique integer key within a small, predefined range. It is critical that inserting sensor data into your system, as well as accessing these readings by key, is performed with guarantees of efficient performance even in the worst case. Due to hardware constraints, the data structure must be very simple. Furthermore, it has only a single consecutive block of memory at its disposal for storing the keys.

   1. Array indexed by keys
   2. AVL tree
   3. Find-union
   4. Hash table
   5. Heap/Priority queue
   6. Sorted Linked list
   7. Queue
   8. Stack
   9. Sorted array
   10. Unsorted array

   Most appropriate data structure: **Array indexed by keys**

   Explanation:

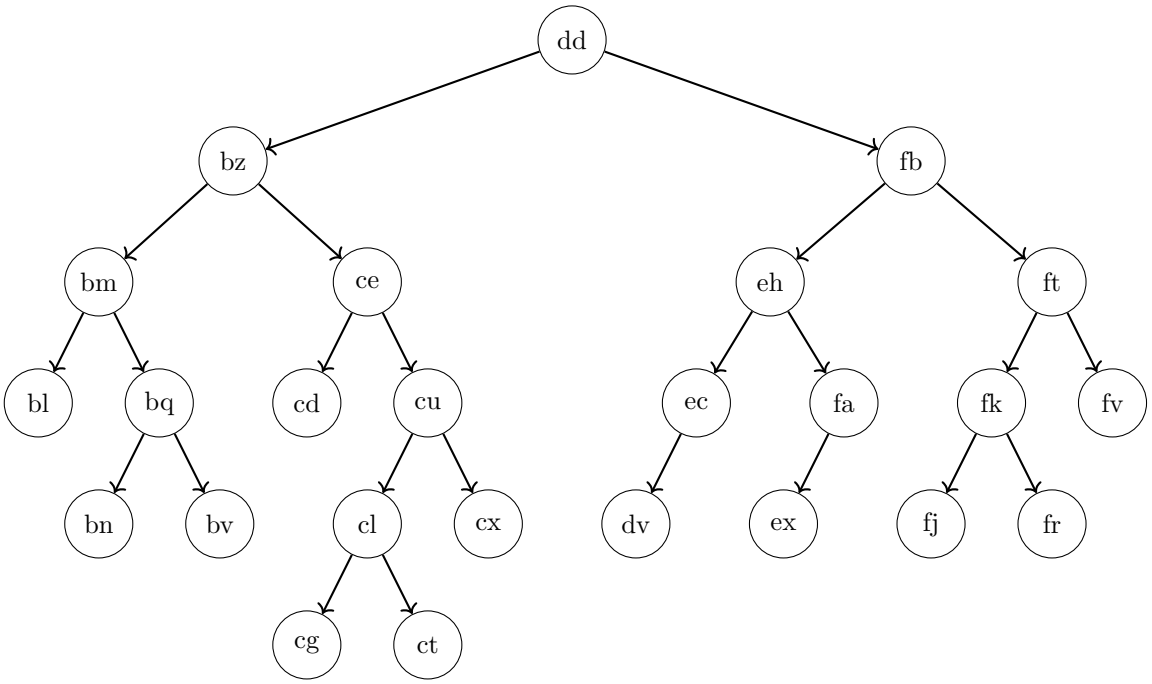
   Solution: An array indexed by keys is optimal in this scenario because it allows $O(1)$ time for all standard dictionary operations for accessing the data whereas its memory usage of $O(M)$ (where M is the maximum key) is not too large as the keys belong to a small range. All the other data structures are worse under this scenario as they either don't provide feasible operations (find-union, stack, queue, heap), or take more time in the worst case (AVL tree, hash table, sorted and unsorted array, sorted linked list), while their memory complexities are not better. Furthermore, this data structure satisfies the requirement of a simple, contiguous block of memory.

10. [5 points] What is the height of the node *bz* in this binary search tree, what is the height of the root (as defined in the lecture and exercises)?

Recall that the height of a node v is the number of nodes *on a longest path starting at v within the subtree of v (not counting the dummy "NONE" nodes that have height* 0*).*

Height of *bz*: **5** (*Write only a single number.*)

Height of the root: **6** (*Write only a single number.*)



Does the tree satisfy the AVL invariant? Your answer: **no** (*Write "yes" or "no".*)

Write the names of two nodes,

X=**ce** and Y=**cd**, such that the following holds:

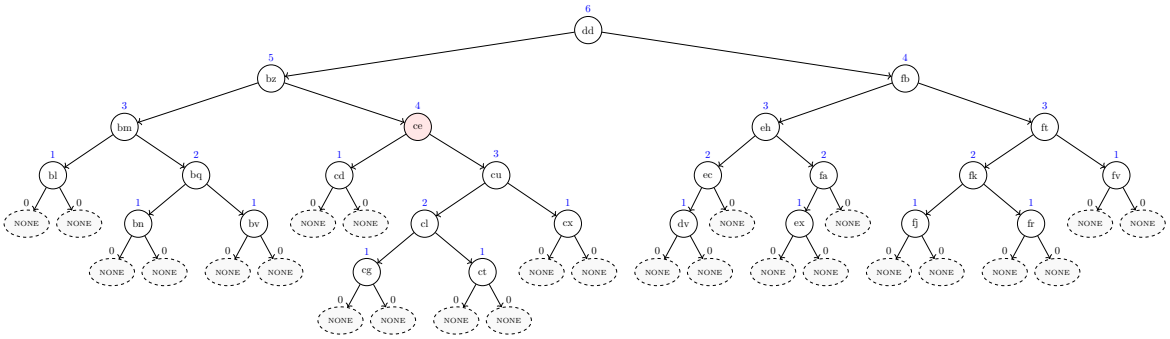
- *If the tree violates the AVL invariant*, then the violation occurs at node X (caused by the heights of X's children), and adding a new child to node Y will restore the AVL invariant.
- *Otherwise, if the tree satisfies the AVL invariant*, then adding a new child to node X will cause a violation of the AVL invariant, and the violation will occur at node Y (caused by the heights of Y's children).
- *If there is more than one valid choice for X*, choose the one with the alphabetically **smallest** name. **After selecting** X, if there is more than one valid choice for Y, choose the one with the alphabetically **smallest** name. (*Example: "bl" is alphabetically smaller than "fv".)*

In other words, X is either a node that violates the AVL invariant, or a node where adding a new child would cause such a violation. Similarly, Y is either a node where adding a new child restores the AVL invariant, or it is a node that violates the invariant after a new child is added to X. If there are multiple valid choices for X, choose the one with the alphabetically smallest name. If there are multiple valid choices for Y (given your choice of X), choose the one with the alphabetically smallest name.

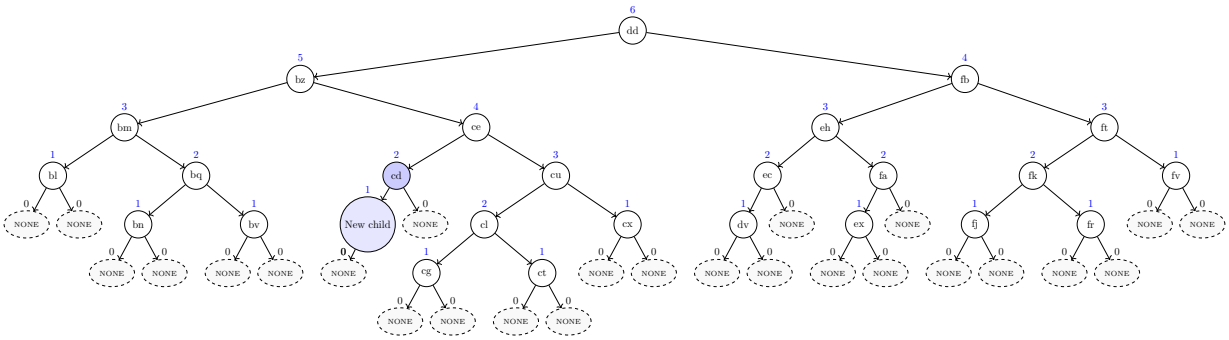
Note that there is exactly one correct solution that satisfies all the conditions above. It is also possible that $X = Y$.

See next page for an explanation of the solution!

Explanation:
The tree violates the AVL invariant at $\boldsymbol{X = \text{ce}}$. The first tree shows the heights and dummy nodes, as well as the violation in red (the parent X whose children differ in height by more than 1).



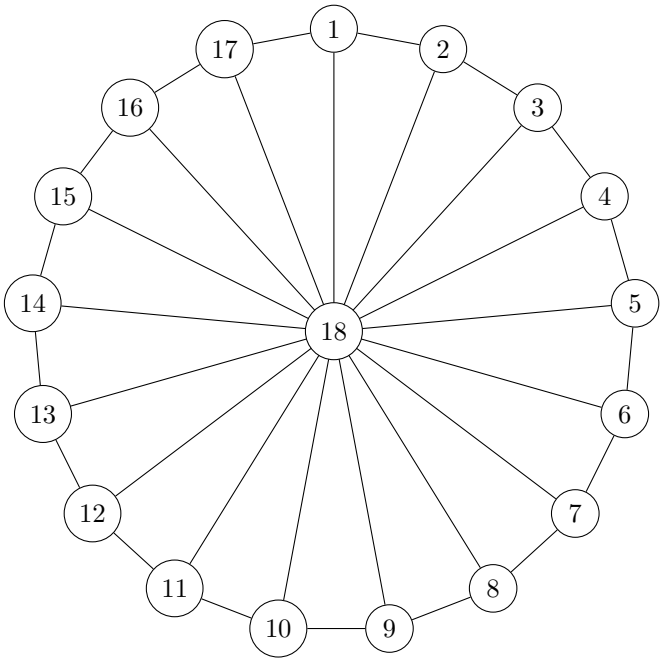
The second tree shows the parent $\boldsymbol{Y = \text{cd}}$ and its new added child (both in blue) where the new child causes the tree to satisfy the AVL invariant.



11. [5 points] Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to n=18 and each edge has the same weight 1. Observe that the vertices 1 to 17 lie on a big cycle and each of them is connected to the center vertex 18 via an edge.

    Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 7) already after 13 iterations.

```
1  δ[1..18][1..18] = new 2D array filled with +∞
2  for i = 1 to 18 do
3      δ[i][i] = 0
4  foreach edge (v, w, c) do
       // v and w are the endpoints and c is the weight of the edge
5      δ[v][w] = c
6      δ[w][v] = c
7  for k = 1 to 13 do
8      foreach pair (v, w) do
9          δ[v][w] = min(δ[v][w],   δ[v][k] + δ[k][w])
10 return δ
```



Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 13 times**!

Recall that the weight of every edge is equal to 1.

*If the distance is **infinite**, write "INF". Note: There are exactly two fields where the correct answer is "INF". If you write "INF" in more than two fields, you get 0 points for those fields.*

$\delta[15][3] =$ **INF** $\qquad\qquad \delta[18][15] =$ **1**

$\delta[17][14] =$ **14** $\qquad\qquad \delta[13][16] =$ **INF**

$\delta[2][12] =$ **10**

12. [4 points] Consider the following algorithm.

Input: integer b
$x = 4$
$e = 0$
while $x < b$ **do**
  $t = 4$
  $x = x + t$
  $e = e + t$
  $x = x - 1$
  $e = e - 11$

Someone wrote the following invariant for the loop:

$$A \cdot x + C \cdot e = D$$

with the three constants

- $A = 7$
- $C = 2$
- $D = 28$

The invariant should hold each time the algorithm evaluates the loop condition. Unfortunately, the invariant has a mistake: one of the three constants, A, C, or D has a wrong value.

Fix the invariant by changing the value of one of the constants!

Which constant do you choose? _____**C**_____ *(Write the name of one of the three constants.)*

The new value for the chosen constant: _____**3**_____

Note: You are allowed to change only one constant.

13. 4 points (bonus)

Consider the following decision problem A:

You are given a list of users of a social network. Furthermore, you also know for every two users whether they are friends or not. Is there an ordering of some of the users (not necessarily all) such that the first user is Alice *and the last user is* Bob *and any two consecutive users in the ordering are friends with each other?*

Consider the following decision problem B:

With the exam just around the corner, you decided to watch lecture recordings. Each recording has a different length and a different rating based on how helpful it is. Unfortunately, time is short, and you won't be able to watch everything. Your goal is to maximize the total rating of the recordings you watch, but you need to carefully choose which ones to fit into your limited time. Thus, is there a selection of recordings that you can still watch in your remaining study time and whose total rating is above some given rating value?

Mark all the correct sentences and give a short explanation.

- √ **A can be reduced to B in polynomial time.**
- ◯ B can be reduced to A in polynomial time.
- √ **A is in P.**
- ◯ B is in P.
- ◯ It is unknown whether A can be reduced to B in polynomial time.
- √ **It is unknown whether B can be reduced to A in polynomial time.**
- ◯ It is unknown whether A is in P.
- √ **It is unknown whether B is in P.**

Solution: Problem B is equivalent to the NP-complete problem Knapsack (the items are the recordings, total capacity is the time left, and the target total value is the given rating value), whereas problem A can be solved in polynomial time: we can reduce the problem (in polynomial time) to the connectivity problem in graphs as follows: users are vertices, we have an edge between two vertices if the respective users are friends. We ask: is there a path connecting the vertex of *Alice* with the vertex of *Bob*? If there is such a path, then, going along the path, we get an ordering of users where consecutive users are friends (edges) and where the first user is *Alice* and the last one is *Bob*. On the other hand, if there is such an ordering, then there is also such a path. This completes the polynomial-time reduction.
Next, observe that the connectivity problem can be solved in polynomial time, for example, by using breadth-first search (BFS). Consequently, the problem that we reduced to the graph problem is also in P.

Consequently, as problem A can be solved in polynomial time, it can reduced in polynomial time to any other problem, thus also to B.

Since it is unknown whether NP-complete problems can be solved in polynomial time, we do not know whether B is in P, and, consequently, whether there exist a polynomial time reduction from B to any polynomial-time problem (e.g., to A).

Algorithms for Data Science
Exam 2025
Group **B**

June 2025

Your Name: _____ **Your Student ID:** _____

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	5	4	4	6	5	9	5	4	4	5	5	4	0	60
Score:														

The last task gives 4 bonus points. Thus, you can obtain 60+4 points in total (regular+bonus points).

Check if your name and student ID are printed on top of every single page!

Write your name and your student ID on the first page!

Check whether you received all 14 pages (containing 13 tasks in total).

The regular time to solve the exam is 90 minutes.

1. [5 points] Consider the following incomplete Python function. The input is an array $a[0..n-1]$ and a target value x. The array is sorted reversely, that is, $a[0]$ is the largest value in a, and $a[n-1]$ the smallest one.The function outputs the maximum position i of the array such that all values in a from position 0 up to position i have value at least x. In other words, the function returns the maximum i such that, for all $0 \leq j \leq i$, we have $a[j] \geq x$.If $a[0]$ is smaller than x, then the function should return -1.

   Hint: the task is related to the first programming task.

```
1  def compute_max_pos(a, x):
2      n = len(a)
3      left = -1
4      right = n - 1
5      # ??? [Line 5] (Starts with while...)
6          # ??? [Line 6]
7          # ??? [Line 7] (Starts with if...)
8              # ??? [Line 8]
9          else:
10             # ??? [Line 10]
11
12     return left
```

   Complete the code by choosing a correct code snippet for each empty line (that starts with `# ???`). Do it by filling the blanks with the correct labels from the code snippets below:

   - Line 5: **D**
   - Line 6: **G**
   - Line 7: **O**
   - Line 8: **A**
   - Line 10: **I**

   Code snippets to choose from:

   - (A) `right = p - 1`
   - (B) `if x < a[p]:`
   - (C) `while left == right:`
   - (D) `while left < right:`
   - (E) `right = p + 1`
   - (F) `left = p - 1`
   - (G) `p = (left + right + 1) // 2`
   - (H) `p = (left + right + 1) * 2`
   - (I) `left = p`
   - (J) `if x == a[p]:`
   - (K) `right = p`
   - (L) `p = right - left`
   - (M) `while left > right:`
   - (N) `left = p + 1`
   - (O) `if x > a[p]:`

   Note: The indendation of your code is the same as of `# ???`*. In other words, your code for an empty line will start at the position of* `#`*.*

2. [4 points] Write a recurrence for the running time of the following algorithm `AlgoA(`$a[1..n]$`)` in the form of

$$T(n) = aT(n/b) + \Theta(f(n)) \ .$$

You don't need to solve the recurrence! And you don't need to understand what the algorithm computes!

The input of `AlgoA(`$a[1..n]$`)` *is an array* a *of* n *integers.*

The input of the auxiliary function `Copy_Sub_Array(`$a[1..n]$`,`i`,`m`)` *is an array* $a[1..n]$*, a position* i *and a length* m*. The function returns a copy of the subarray* $a[i+1..i+m]$*.*

```
AlgoA(a[1..n])
    if n ≥ 9 then
        ret = 0
        p = n · n
        for j = 1 to (p · p) do
            ret = ret + a[10]
        d = 4 · 2
        m = ⌈n/d⌉
        i = n
        while i > n − 4 do
            c = Copy_Sub_Array(a,i,m)
            ret = ret + AlgoA(c)
            i = i − 1
        for i = 1 to 2 do
            c = Copy_Sub_Array(a,i,m)
            ret = ret + AlgoA(c)
        return ret
    else
        return 10
```

```
Copy_Sub_Array(a[1..n],i,m)
    b[1..m] = new empty array
    for j = 1 to m do
        b[j] = a[i + j]
    return b
```

Your answer: $T(n) =$ $6T(\lceil n/8 \rceil) + \Theta(n^4)$

3. [4 points] Solve the following recurrence using the master theorem. Determine (for yourself) the values for a, b, and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = 10^3 \cdot 10^8 \cdot T(n/10) + \Theta(n^{13})$$

Your answers:

$c =$ **11**

Case: **C**

Resulting running time $T(n) =$ $\Theta(n^{13})$

4. Simplify the following running times as far as possible:

(a) [2 points] $\Theta(\ 3\sqrt{n} + 4 + (n^{5/10+5/10})/2\ ) = \Theta($ n $)$

(b) [2 points] $\Theta(\ 29 \cdot n^{20} + 2 \cdot 11^n + 40 \cdot 10^n\ ) = \Theta($ 11^n $)$

(c) [2 points] $\Theta(\ 3(\log_2 n)^5 + 5 + 1\log_2(n^6)\ ) = \Theta($ $(\log_2 n)^5$ $)$

5. [5 points] Consider the following array $a[1..21]$ containing twenty-one elements:

$$a = \langle 105, 65, 100, 60, 50, 80, 95, 55, 60, 55, 35, 75, 70, 85, 90, 25, 15, 20, 30, 40, 45 \rangle$$

1. Draw the *complete* array as a complete binary tree (as defined for heaps),
2. Mark the edge whose endpoints (parent and child) violate the heap invariant,
3. Choose an appropriate index position j and **decrease** the value of $a[j]$ by the **maximum amount possible** such that a satisfies the heap invariant.

What index j do you choose?
(Write a single number between 1 *and* 21*; there is only one correct answer.)*

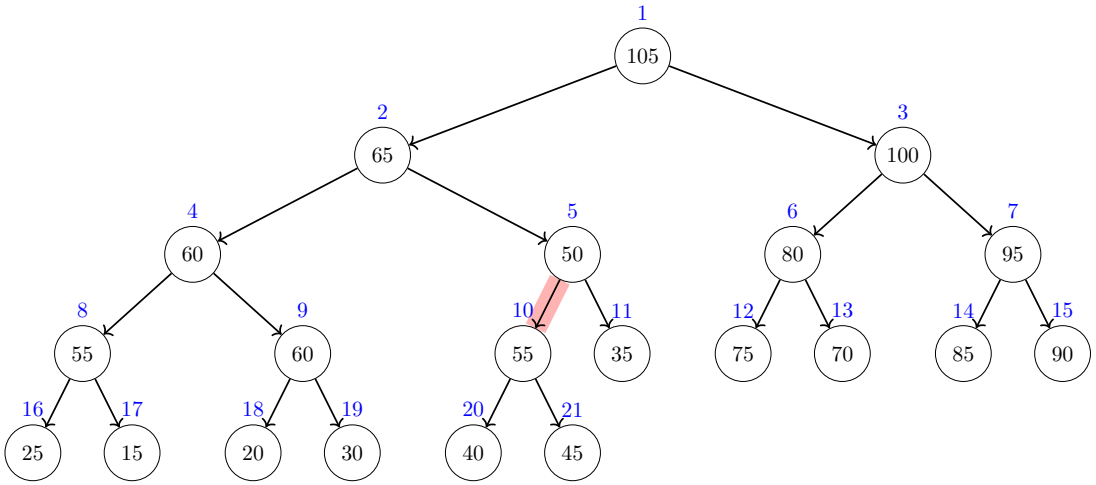
Your answer: $j =$ **10** ***(the violating edge's other endpoint is at* 5*.)***

What is the new value for $a[j]$?
(Write a single number, as small as possible and in particular smaller than the old value of $a[j]$*.)*

Your answer: $a[j] =$ **45** ***(the smallest value from* $[45, 50]$*)***

Your tree hould contain ***all* the elements of the array**, from $a[1] = 105$ up to $a[21] = 45$:

Solution:



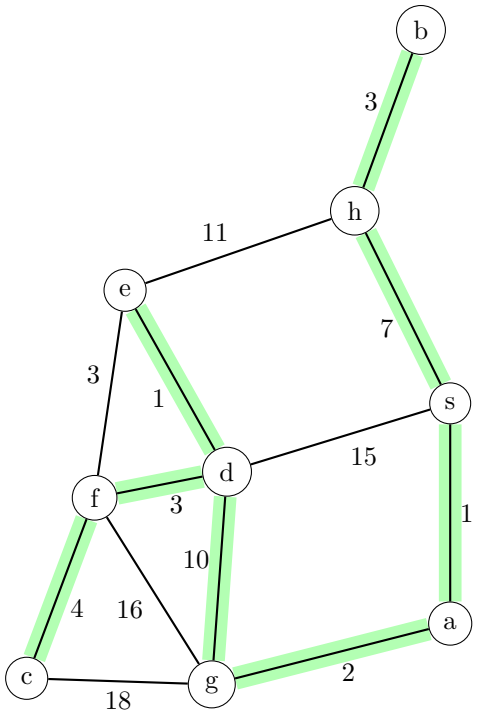
Explanation (not required): The heap invariant is violated at the red edge as the child (index **10***) has a larger value than its parent (index* **5***). To fix the heap invariant, it suffices to decrease* $a[10]$ *to any value between* 45 *and* 50*. The smallest possible value (as asked in the task) is thus* 45*.*

6. [9 points] Dijkstra's algorithm (see below) is run on the following graph where the start node is s.

   What are the values of the *dist* and *parent* tables when Dijkstra's algorithm has finished? Fill in the missing entries in the table below!

   Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```
dist[0..n − 1] filled with INF
parent[0..n − 1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time then
        for every edge (v, w) do
            if dist[w] > dist[v] + c_{v,w} then
                dist[w] = dist[v] + c_{v,w}
                parent[w] = v
                q.add(w, -dist[w])
```



Solution:

	a	b	c	d	e	f	g	h	s
distance	1	10	20	13	14	16	3	7	0
parent	s	h	f	g	d	d	a	s	NONE

Note:

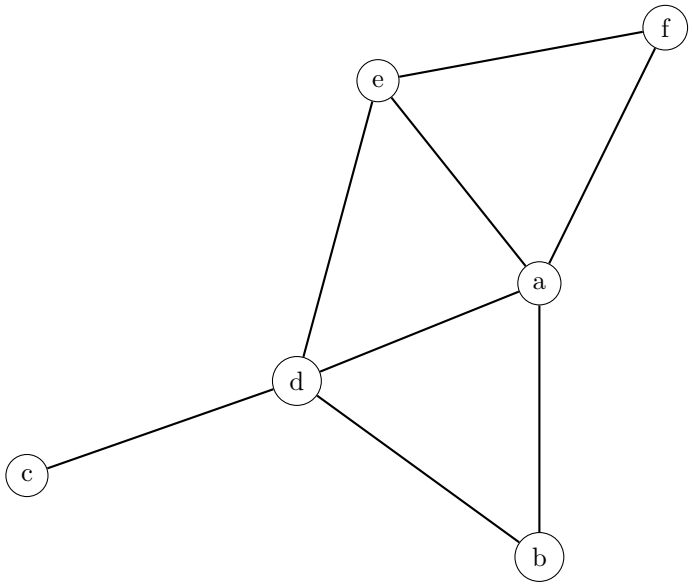
- *The entries of the parent table consist of the names of the nodes or the special value* `NONE`*. For example, the parent of some node could be* s *whereas the parent of* s *is* `NONE`*.*
- *If a vertex has more neighbors, we consider them* ***in lexicographic order*** *(that is, node* a *would come before node* b *and so on)!*

7. [5 points] Draw an (undirected) graph without edge crossings corresponding to the following adjacency matrix. For your drawing, just connect the given vertices below. Your edges don't need to be straight and are allowed to be curves.

   For your score, it's better to draw *all* of the edges even if they cross each other than not to draw one or more edges.

	a	b	c	d	e	f
a	0	1	0	1	1	1
b	1	0	0	1	0	0
c	0	0	0	1	0	0
d	1	1	1	0	1	0
e	1	0	0	1	0	1
f	1	0	0	0	1	0

Solution:



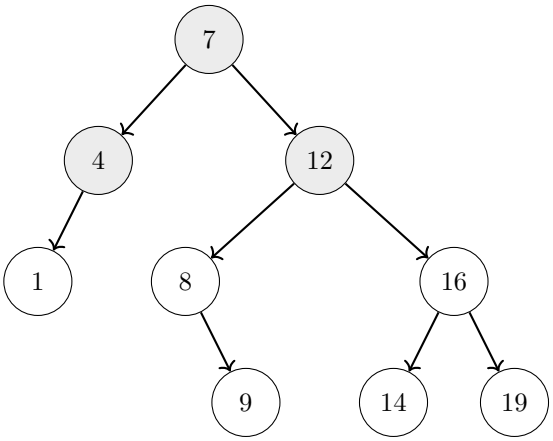
Note that a correct solution has to be crossing free! However, the depicted solution has been automatically drawn by a graph drawing program that does not guarantee that edges do not cross. Thus, if the depicted solution contains edge crossings, keep in mind that such a solution is not fully correct and you would not get all the points by drawing it. However, note that a crossing-free solution can be easily obtained from the depicted solution by redrawing one or more edges as curves around the graph.

8. [4 points] Add the following keys into the following binary search tree **in the order** as they are given below. It suffices that you draw how the binary search looks at the end with each key written in its respective node. (For simplicity, we consider only keys and no values.)

   Note: Do not change connections between previously added nodes! Just add the keys one by one in the given order to their correct positions.

   8, 16, 1, 9, 14, 19

Solution:



9. [4 points] What is the most appropriate data structure for the following scenario, chosen from the list below?

   Unless stated otherwise in the scenario, the goal is to minimize memory usage and the worst-case running time of each operation.

   In your answer, briefly explain how your chosen data structure applies to the scenario. Then give its memory and time complexity, and explain why it performs better than *every* other option, considering the scenario's specific constraints and goals.

   You are implementing the task scheduling subsystem of a web browser. Various subtasks (layout, paint, script execution, etc.) are assigned a dynamic importance score (an arbitrarily large integer), and you must always pick and remove the task with the highest score next. Efficient insertion of new tasks and peeking at the current highest score are critical. Due to the browser architecture, all tasks must be stored exclusively in a single contiguous block of memory.

   1. Array indexed by keys
   2. AVL tree
   3. Find-union
   4. Hash table
   5. Heap/Priority queue
   6. Sorted Linked list
   7. Queue
   8. Stack
   9. Sorted array
   10. Unsorted array

   Most appropriate data structure: **Heap/Priority queue**

   Explanation:

   Solution: We use a binary heap where each element is a task and its importance score serves as the priority; calling `ExtractMax()` returns and removes the task with the highest score.
   The memory usage is best possible as the size of the heap is proportional to the number of elements it contains. Furthermore, a heap stores its elements in an array, that is, in an exclusive contiguous block of memory as required.

   The running time of `FindMax()` is best possible as it takes only constant time. The running times of `Add` and `RemoveMax()` are a little worse with $O(\log n)$ time (where n is the number of elements currently in the data structure), but this is still reasonable fast and the best we can get among all the data structures given in the list.

   All the other data structures are worse under this scenario as they don't provide feasible operations (queue, stack, find-union), take more time in the worst case (AVL tree, hash table, sorted/unsorted array, linked list), are not based on an array (AVL tree, sorted linked list), or take too much memory as the maximum key is very large (array indexed by keys).

   Note that in a standard AVL tree, retrieving the maximum key requires $O(\log n)$ time. Although augmenting each node with its subtree's maximum key would allow $O(1)$ retrieval by inspecting the root, an AVL tree inherently relies on pointer-based nodes scattered arbitrarily in memory. This violates our requirement that all keys be stored exclusively in a single contiguous array.

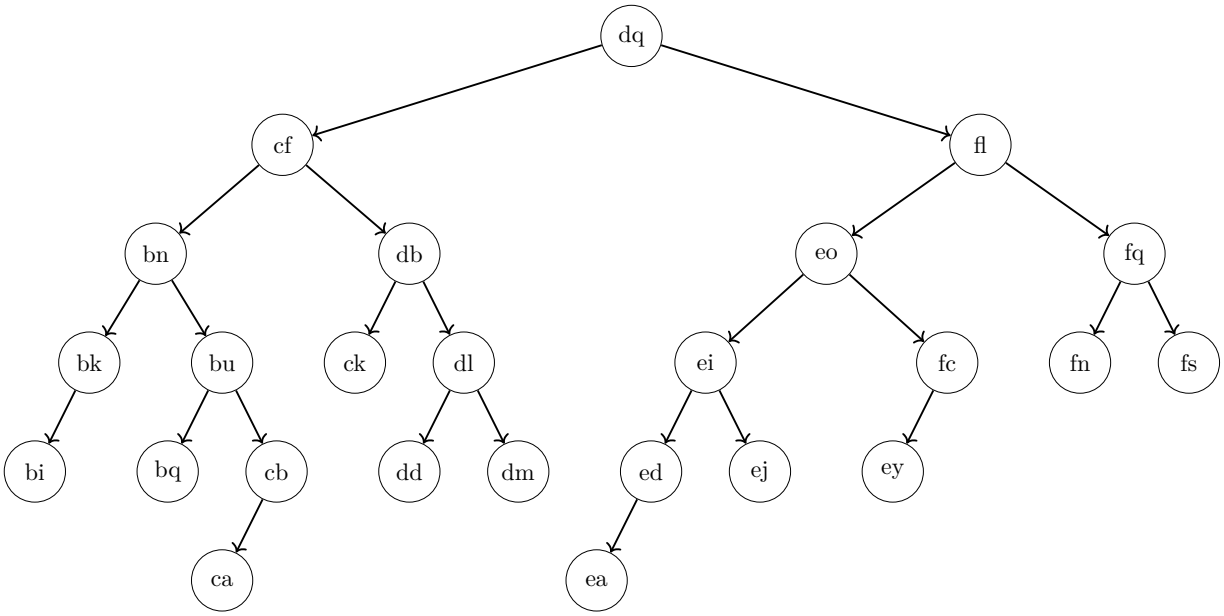
   Note that array indexed by keys is bad in this scenario even if we neglect its memory problem: if M is the maximum key size, it takes $\Theta(M)$ time in the worst-case to find the maximum key.

10. [5 points] What is the height of the node cf in this binary search tree, what is the height of the root (as defined in the lecture and exercises)?

Recall that the height of a node v is the number of nodes *on a longest path starting at v within the subtree of v (not counting the dummy "NONE" nodes that have height* 0*).*

Height of cf: **5** (*Write only a single number.*)

Height of the root: **6** (*Write only a single number.*)



Does the tree satisfy the AVL invariant? Your answer: **no** (*Write "yes" or "no".*)

Write the names of two nodes,

X=**fl** and Y=**fn**, such that the following holds:

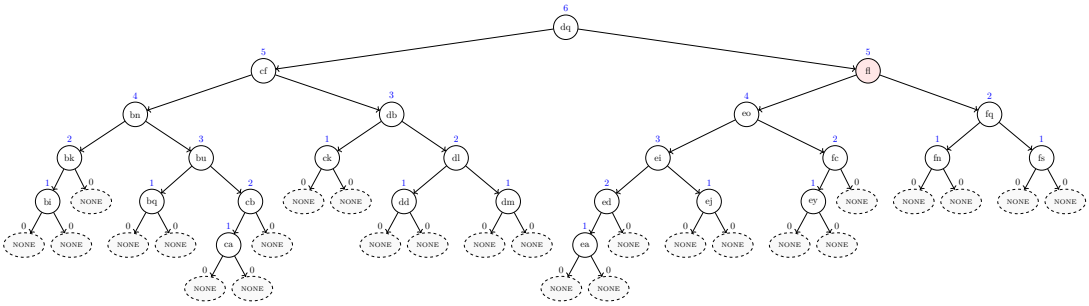
- *If the tree violates the AVL invariant*, then the violation occurs at node X (caused by the heights of X's children), and adding a new child to node Y will restore the AVL invariant.
- *Otherwise, if the tree satisfies the AVL invariant*, then adding a new child to node X will cause a violation of the AVL invariant, and the violation will occur at node Y (caused by the heights of Y's children).
- *If there is more than one valid choice for X*, choose the one with the alphabetically **largest** name. **After selecting** X, if there is more than one valid choice for Y, choose the one with the alphabetically **smallest** name. (*Example: "bi" is alphabetically smaller than "fs".)*

In other words, X is either a node that violates the AVL invariant, or a node where adding a new child would cause such a violation. Similarly, Y is either a node where adding a new child restores the AVL invariant, or it is a node that violates the invariant after a new child is added to X. If there are multiple valid choices for X, choose the one with the alphabetically largest name. If there are multiple valid choices for Y (given your choice of X), choose the one with the alphabetically smallest name.

Note that there is exactly one correct solution that satisfies all the conditions above. It is also possible that $X = Y$.

See next page for an explanation of the solution!

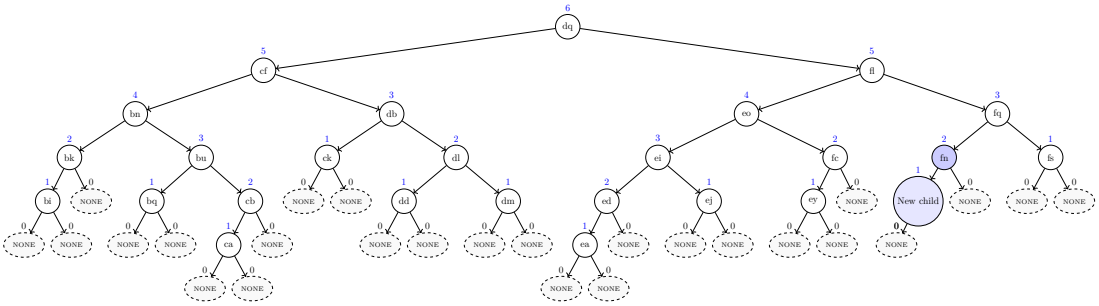
Explanation:
The tree violates the AVL invariant at $\boldsymbol{X}$ = **fl**. The first tree shows the heights and dummy nodes, as well as the violation in red (the parent X whose children differ in height by more than 1).



There are two possibilities to choose a parent where adding a child will fix the AVL invariant:

fn and **fs**.

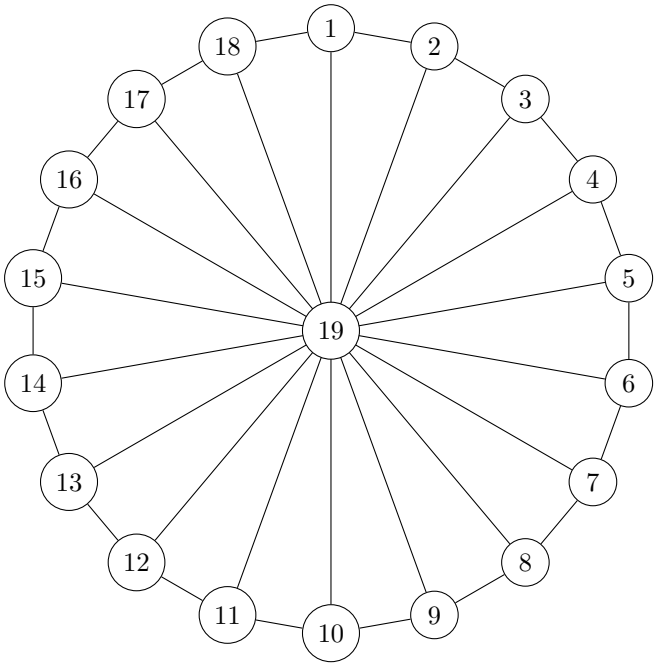
According to the task, you have to choose the parent with the alphabetically smallest label; hence, Y = fn.
The second tree shows the parent $\boldsymbol{Y}$ = **fn** and its new added child (both in blue) where the new child causes the tree to satisfy the AVL invariant.



11. [5 points] Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to n=19 and each edge has the same weight 1. Observe that the vertices 1 to 18 lie on a big cycle and each of them is connected to the center vertex 19 via an edge.

    Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 7) already after 12 iterations.

```
1  δ[1..19][1..19] = new 2D array filled with +∞
2  for i = 1 to 19 do
3      δ[i][i] = 0
4  foreach edge (v, w, c) do
       // v and w are the endpoints and c is the weight of the edge
5      δ[v][w] = c
6      δ[w][v] = c
7  for k = 1 to 12 do
8      foreach pair (v, w) do
9          δ[v][w] = min(δ[v][w],   δ[v][k] + δ[k][w])
10 return δ
```



Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 12 times**!

Recall that the weight of every edge is equal to 1.

*If the distance is **infinite**, write "INF". Note: There are exactly two fields where the correct answer is "INF". If you write "INF" in more than two fields, you get 0 points for those fields.*

$\delta[7][14] =$ **INF** $\quad\quad \delta[17][12] =$ **INF**

$\delta[11][2] =$ **9** $\quad\quad \delta[14][19] =$ **1**

$\delta[13][18] =$ **13**

12. [4 points] Consider the following algorithm.

```
Input: integer e
u = 4
t = 0
while u < e do
    u = u + 45
    h = −3
    t = t − h
    u = u + h
    t = t − 9
```

Someone wrote the following invariant for the loop:

$$K \cdot u + D \cdot t = F$$

with the three constants

- $K = 2$
- $D = 14$
- $F = 4$

The invariant should hold each time the algorithm evaluates the loop condition. Unfortunately, the invariant has a mistake: one of the three constants, K, D, or F has a wrong value.

Fix the invariant by changing the value of one of the constants!

Which constant do you choose? _____**F**_____ *(Write the name of one of the three constants.)*

The new value for the chosen constant: _____**8**_____

Note: You are allowed to change only one constant.

13. 4 points (bonus)

Consider the following decision problem A:

You are given the floor plan of a museum consisting of rooms connected by corridors. You want to plan a guided tour that starts and ends at the museum entrance, visiting each room exactly once without retracing your steps. Is such a tour possible before closing time?

Consider the following decision problem B:

You manage a server with limited disk space. You have a set of files, each with a given size (in GB) and an importance rating. Can you choose a subset of these files whose total size does not exceed S GB and whose total importance is at least V?

Mark all the correct sentences and give a short explanation.

- √ **A can be reduced to B in polynomial time.**
- √ **B can be reduced to A in polynomial time.**
- ◯ A is in P.
- ◯ B is in P.
- ◯ It is unknown whether A can be reduced to B in polynomial time.
- ◯ It is unknown whether B can be reduced to A in polynomial time.
- √ **It is unknown whether A is in P.**
- √ **It is unknown whether B is in P.**

Solution: Problem A is equivalent to the NP-complete problem Hamiltonian cycle (by interpreting each room as a vertex, each corridor as an edge, and the desired tour as a cycle visiting every vertex exactly once in the resulting graph), and problem B is equivalent to the NP-complete problem Knapsack (by interpreting each file as an item with weight equal to its size, importance as value, storage limit S as capacity, and target importance V as the value threshold).

Both problems can be reduced to each other in polynomial time because

1. both problems are in NP (as they are NP-complete),
2. both problems are NP-hard (as they are NP-complete), and
3. all problems in NP (thus, including A and B) can be reduced in polynomial time to an NP-hard problem (for example, to A and B).

Since it is unknown whether NP-complete problems can be solved in polynomial time, we do not know whether each of the two problems is in P.

Algorithms for Data Science
Exam 2025
Group **C**

June 2025

Your Name: _____ **Your Student ID:** _____

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	5	4	4	6	5	9	5	4	4	5	5	4	0	60
Score:														

The last task gives 4 bonus points. Thus, you can obtain 60+4 points in total (regular+bonus points).

Check if your name and student ID are printed on top of every single page!

Write your name and your student ID on the first page!

Check whether you received all 14 pages (containing 13 tasks in total).

The regular time to solve the exam is 90 minutes.

1. [5 points] Consider the following incomplete Python function. A sequence of unit blocks is dropped one by one onto integer coordinates along a line; when a block lands on an occupied coordinate, it stacks on top, raising that column's height by 1.
Coordinates may be arbitrarily large and all start at height 0. The function *blocks*(*pos*,*h*) takes the list *pos* of drop positions and the target height *h*, and returns the x-coordinate where a column's height first reaches *h*.

   Hint: the task is related to the third programming task.

```
1  def blocks(pos, h):
2      n = len(pos)
3      # ??? [Line 3]
4      for i in range(n):
5          x = pos[i]
6          # ??? [Line 6] (Starts with if..)
7              # ??? [Line 7]
8          else:
9              # ??? [Line 9]
10         # ??? [Line 10]
11             return x
```

   Complete the code by choosing a correct code snippet for each empty line (that starts with `# ???`). Do it by filling the blanks with the correct labels from the code snippets below:

   - Line 3: _____**O**_____
   - Line 6: _____**A**_____
   - Line 7: _____**I**_____
   - Line 9: _____**C**_____
   - Line 10: _____**M**_____

   Code snippets to choose from:

   - (A) `if x not in heights:`
   - (B) `if x < len(heights):`
   - (C) `heights[x] += 1`
   - (D) `if heights[x] < h:`
   - (E) `heights[x] = h`
   - (F) `heights[x] += i`
   - (G) `if heights[h] == i:`
   - (H) `heights = [0] * n`
   - (I) `heights[x] = 1`
   - (J) `if heights[x] == 0:`
   - (K) `heights[i] = x`
   - (L) `heights = [0] * max(pos)`
   - (M) `if heights[x] == h:`
   - (N) `heights[i] += x`
   - (O) `heights = {}`

   Note: The indendation of your code is the same as of `# ???`*. In other words, your code for an empty line will start at the position of* `#`*.*

2. [4 points] Complete the following algorithm `AlgoA(`$a[1..n]$`)` such that its running time $T(n)$ satisfies the recurrence

$$T(n) = 17T(\lceil n/30 \rceil) + \Theta(n^8) \ .$$

You don't need to understand what the algorithm computes!

The input of `AlgoA(`$a[1..n]$`)` *is an array* a *of* n *integers.*

The input of the auxiliary function `Copy_Sub_Array(`$a[1..n]$`,`i`,`m`)` *is an array* $a[1..n]$*, a position* i *and a length* m*. The function returns a copy of the subarray* $a[i+1..i+m]$*.*

```
AlgoA(a[1..n])
    if n < 31 then
        return 25
    else
        d = 5
        m = ⌈n/(6 · d)⌉
        sum = 0
        for i = 1 to 6 do
            b = Copy_Sub_Array(a,i,m)
            sum = sum + AlgoA(b)

        for i = 1 to 11 do
            b = Copy_Sub_Array(a,i,m)
            sum = sum + AlgoA(b)
        w = 1

        for j = 1 to 8 do
            w = w · n // Time: O(1)
        for k = 1 to w do
            sum = sum + a[31]
        return sum
```

```
Copy_Sub_Array(a[1..n],i,m)
    b[1..m] = new empty array
    for j = 1 to m do
        b[j] = a[i + j]
    return b
```

Fill in the three blanks in the algorithm above, each time writing **a single appropriate integer number**!

3. [4 points] Solve the following recurrence using the master theorem. Determine (for yourself) the values for a, b, and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = 100^3 \cdot 100 \cdot T(n/100) + \Theta(n^3)$$

Your answers:

$c =$ **4**

Case: **A**

Resulting running time $T(n) = \Theta(n^4)$

4. Simplify the following running times as far as possible:

(a) [2 points] $\Theta(\ 4n^3 + 12n^2 \log n + 2n^3\ ) = \Theta($ n^3 $)$

(b) [2 points] $\Theta(\ 2 \cdot n^{(3-3)} + 5 + 2^{(n-n)} \cdot 4\ ) = \Theta($ 1 $)$

(c) [2 points] $\Theta(\ (10^{\log_{10} n})^{10} + (2^{n/10})^2\ ) = \Theta($ $2^{n/5}$ $)$

5. [5 points] Consider the following array $a[1..21]$ containing twenty-one elements:

$$a = \langle 105, 100, 40, 95, 90, 25, 30, 65, 80, 50, 85, 20, 15, 30, 35, 55, 60, 75, 70, 50, 45 \rangle$$

1. Draw the *complete* array as a complete binary tree (as defined for heaps),
2. Mark the edge whose endpoints (parent and child) violate the heap invariant,
3. Choose an appropriate index position j and **increase** the value of $a[j]$ by the **maximum amount possible** such that a satisfies the heap invariant.

What index j do you choose?
(Write a single number between 1 *and* 21*; there is only one correct answer.)*

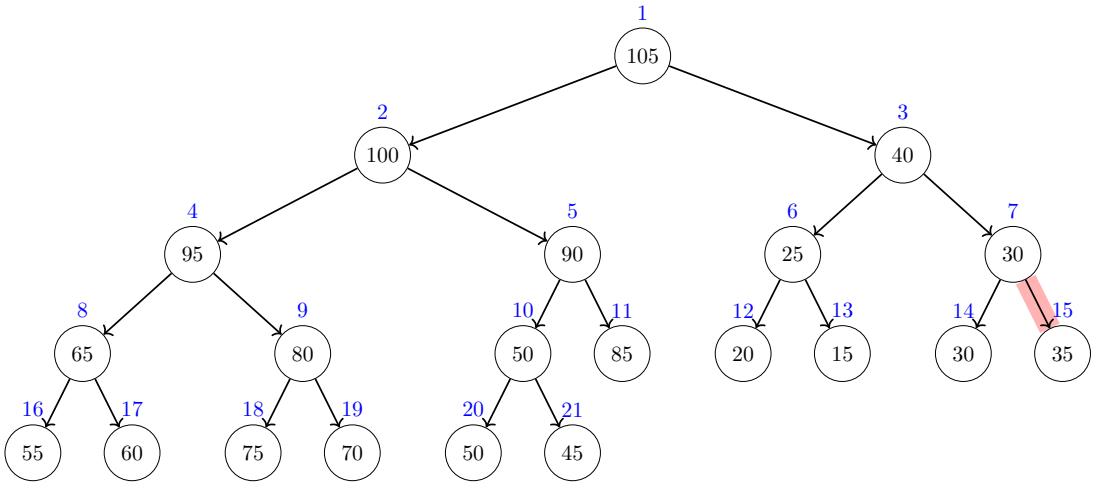
Your answer: $j =$ **7** ***(the violating edge's other endpoint is at* 15*.)***

What is the new value for $a[j]$?
(Write a single number, as large as possible and larger than the old value of $a[j]$*.)*

Your answer: $a[j] =$ **40** ***(the largest value from* [35, 40]*)***

Your tree hould contain ***all* the elements of the array**, from $a[1] = 105$ up to $a[21] = 45$:

Solution:



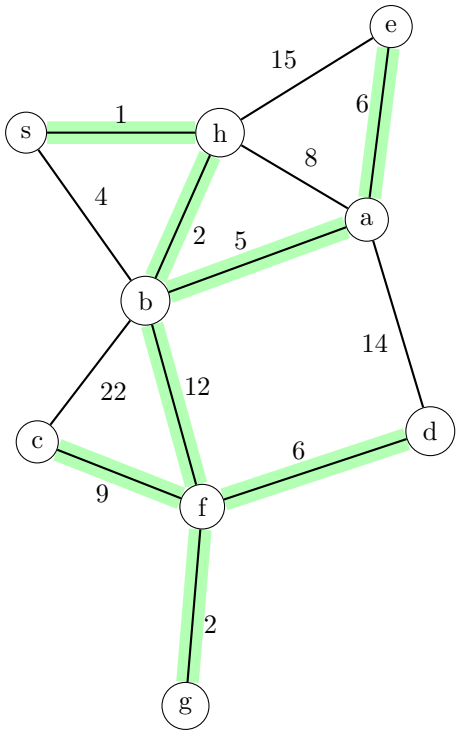
Explanation (not required): The heap invariant is violated at the red edge as the parent (index **7***) has a smaller value than its child (index* **15***). To fix the heap invariant, it suffices to increase* $a[7]$ *to any value between* 35 *and* 40*. The largest possible value (as asked in the task) is thus* 40*.*

6. [9 points] Dijkstra's algorithm (see below) is run on the following graph where the start node is s.

   What are the values of the *dist* and *parent* tables when Dijkstra's algorithm has finished? Fill in the missing entries in the table below!

   Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```
dist[0..n − 1] filled with INF
parent[0..n − 1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time then
        for every edge (v, w) do
            if dist[w] > dist[v] + c_{v,w} then
                dist[w] = dist[v] + c_{v,w}
                parent[w] = v
                q.add(w, -dist[w])
```



Solution:

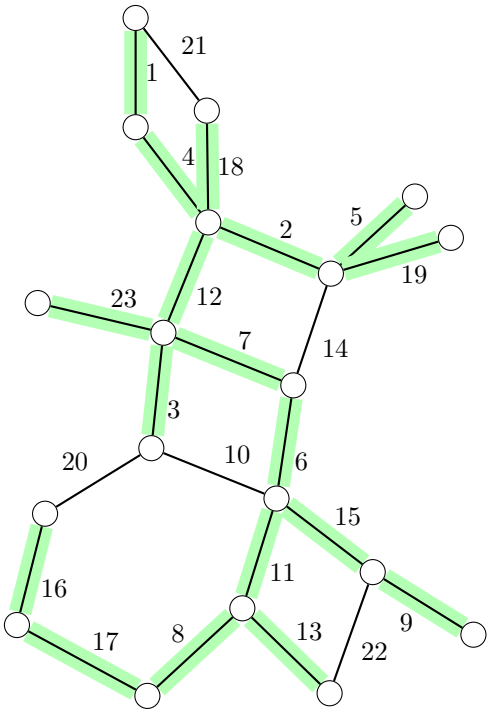
	a	b	c	d	e	f	g	h	s
distance	8	3	24	21	14	15	17	1	0
parent	b	h	f	f	a	b	f	s	NONE

Note:

- *The entries of the parent table consist of the names of the nodes or the special value* `NONE`*. For example, the parent of some node could be* s *whereas the parent of* s *is* `NONE`*.*
- *If a vertex has more neighbors, we consider them* ***in lexicographic order*** *(that is, node a would come before node b and so on)!*

7. [5 points] Mark all the edges that belong to the minimum spanning tree of the following graph. *(Marking an edge means to draw along that edge.)*

Solution:



8. [4 points] Add the following keys (in the order as given) into the hash table below using the hash function

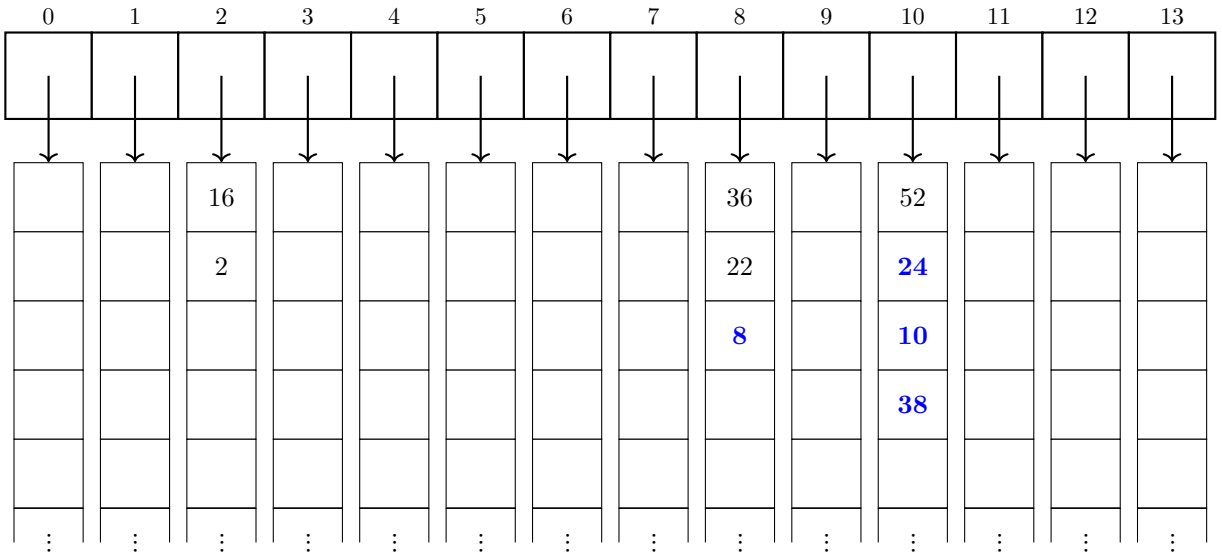
$$h(k) = k \mod 14$$

Keys: 24, 10, 38, 8

Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored (in the order as added; the first element of an array is at the top).

Note that for simplicity, we consider only keys and not key-value pairs. Also note that the hash table has been already filled with some other keys.

Solution:
Hash table:



9. [4 points] What is the most appropriate data structure for the following scenario, chosen from the list below?

   Unless stated otherwise in the scenario, the goal is to minimize memory usage and the worst-case running time of each operation.

   In your answer, briefly explain how your chosen data structure applies to the scenario. Then give its memory and time complexity, and explain why it performs better than *every* other option, considering the scenario's specific constraints and goals.

   You are implementing the task scheduling subsystem of a web browser. You record each visited URL; when the user clicks "Back", you must retrieve and remove the most recently visited URL. Efficiently adding a new URL and undoing to the last URL is critical. URL identifiers can be arbitrarily large integers.

   1. Array indexed by keys
   2. AVL tree
   3. Find-union
   4. Hash table
   5. Heap/Priority queue
   6. Sorted Linked list
   7. Queue
   8. Stack
   9. Sorted array
   10. Unsorted array

   Most appropriate data structure: _____**Stack**_____

   Explanation:

   Solution: In this browser-history context, we always need to undo the very last navigation, and never any older one. Thus a last-in-first-out discipline (as realized by a stack) is required to get the correct URL.
   The running times of a stack are best possible as `Add` and `ExtractNewest` take only constant time. The memory usage is also best possible as the size of the stack is proportional to the number of elements it contains.

   All the other data structures are worse under this scenario as they don't provide feasible operations (queue, heap, find-union), take more time in the worst case (AVL tree, hash table, sorted/unsorted array, sorted linked list), or take too much memory as the maximum key is very large (array indexed by keys).

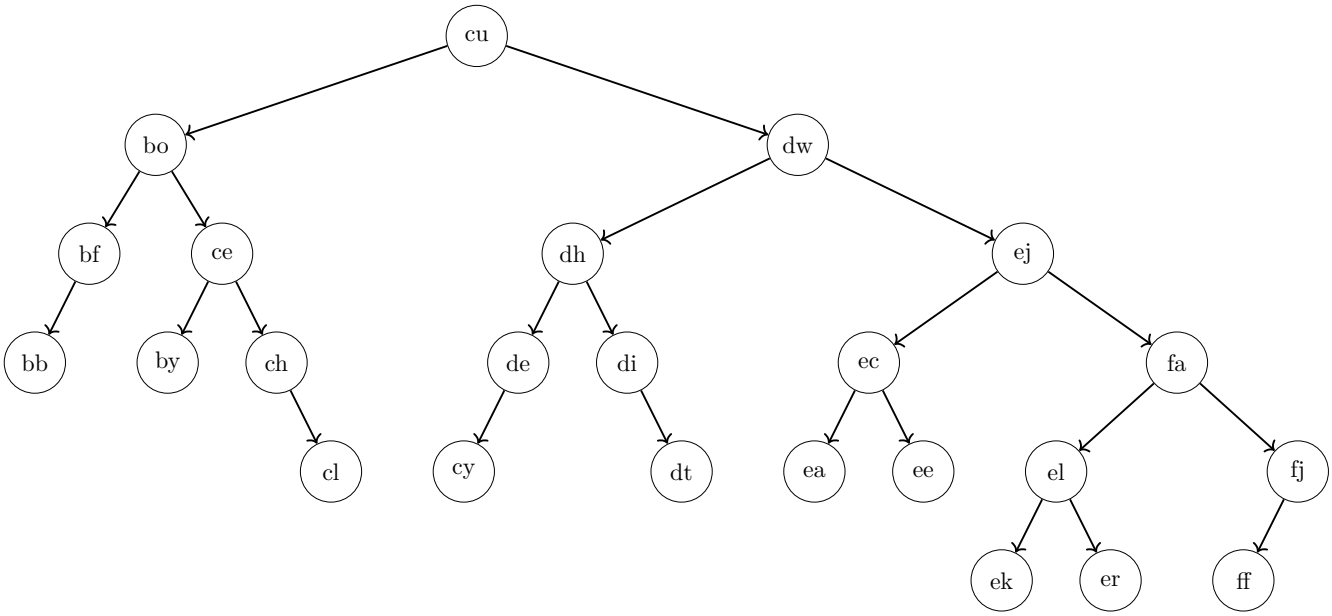
   At first glance, an unsorted array may also seem like a good candidate for the best data structure: new elements can be appended to the end in $O(1)$ amortized time, and removing the most recently added element (i.e., the one at the end of the array) can also be done in amortized constant time. For many practical applications, such performance is sufficient. However, in the scenario of this task, minimizing the worst-case time is crucial. Even though we always remove from the end, the worst-case time for both adding and removing elements in an unsorted array is $\Theta(n)$, since a resize may be required which involves allocating a new array and copying all elements. Therefore, the stack outperforms the unsorted array in this setting, as both its add and remove operations run in constant time even in the worst case.

10. [5 points] What is the height of the node *de* in this binary search tree, what is the height of the root (as defined in the lecture and exercises)?

Recall that the height of a node v is the number of nodes *on a longest path starting at v within the subtree of v (not counting the dummy "NONE" nodes that have height* 0*).*

Height of *de*: **2** (*Write only a single number.*)

Height of the root: **6** (*Write only a single number.*)



Does the tree satisfy the AVL invariant? Your answer: **yes** (*Write "yes" or "no".*)

Write the names of two nodes,

X=**ff** and Y=**cu**, such that the following holds:

- *If the tree violates the AVL invariant*, then the violation occurs at node X (caused by the heights of X's children), and adding a new child to node Y will restore the AVL invariant.
- *Otherwise, if the tree satisfies the AVL invariant*, then adding a new child to node X will cause a violation of the AVL invariant, and the violation will occur at node Y (caused by the heights of Y's children).
- *If there is more than one valid choice for X*, choose the one with the alphabetically **largest** name. **After selecting** X, if there is more than one valid choice for Y, choose the one with the alphabetically **smallest** name. (*Example: "bb" is alphabetically smaller than "fj".)*

In other words, X is either a node that violates the AVL invariant, or a node where adding a new child would cause such a violation. Similarly, Y is either a node where adding a new child restores the AVL invariant, or it is a node that violates the invariant after a new child is added to X. If there are multiple valid choices for X, choose the one with the alphabetically largest name. If there are multiple valid choices for Y (given your choice of X), choose the one with the alphabetically smallest name.

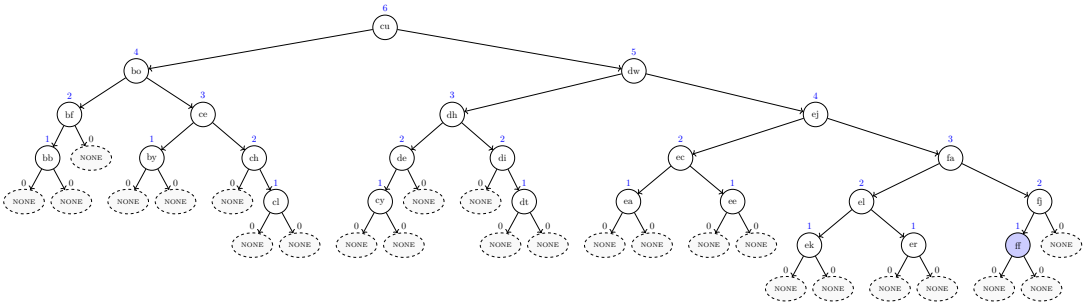
Note that there is exactly one correct solution that satisfies all the conditions above. It is also possible that $X = Y$.

See next page for an explanation of the solution!

Explanation:
The tree satisfies the AVL invariant. There are seven different candidates for X, where adding a node leads to a violation of the AVL invariant:

bb, **cl**, **cy**, **dt**, **ek**, **er** and **ff**.

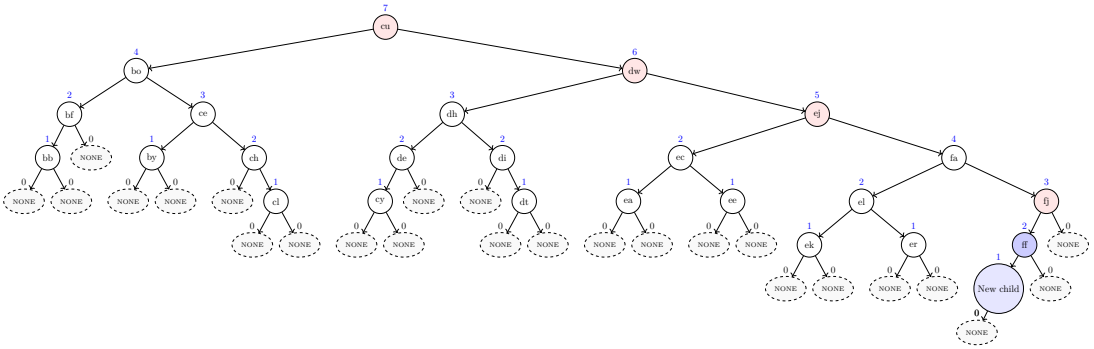
Recall that out of these different possibilties, the task asks you to choose the one where X is the alphabetically largest node; hence, $X =$ ff.
The first tree shows the heights and dummy nodes, as well as the node $\boldsymbol{X} = \mathbf{ff}$ (in blue).



The second tree shows the new child (also in blue) that has been added to X as well as all the four violating parent nodes in red (whose children differ in height by more than 1):

fj, **ej**, **dw** and **cu**.

Among these violating nodes, the task asks you to choose the alphabetically smallest one; hence $\boldsymbol{Y} = \mathbf{cu}$.



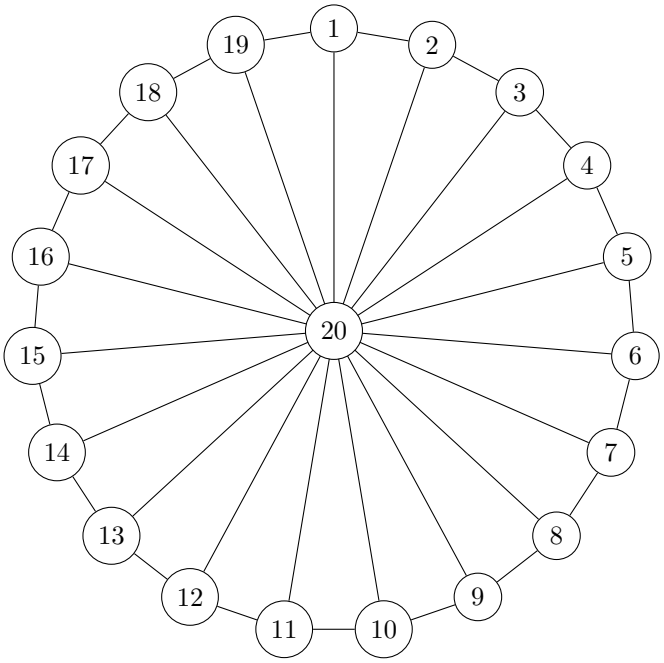
For completness, for each (wrong) candidate for X a list of violating parent nodes. The alphabetically optimal violating parent node is underlined.

- Candidate **bb** →**bf**
- Candidate **cl** →**ch**, **ce** and **bo**
- Candidate **cy** →**de**
- Candidate **dt** →**di**
- Candidate **ek** →**ej**, **dw** and **cu**
- Candidate **er** →**ej**, **dw** and **cu**

11. [5 points] Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to n=20 and each edge has the same weight 1. Observe that the vertices 1 to 19 lie on a big cycle and each of them is connected to the center vertex 20 via an edge.

    Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 7) already after 11 iterations.

```
1  δ[1..20][1..20] = new 2D array filled with +∞
2  for i = 1 to 20 do
3      δ[i][i] = 0
4  foreach edge (v, w, c) do
       // v and w are the endpoints and c is the weight of the edge
5      δ[v][w] = c
6      δ[w][v] = c
7  for k = 1 to 11 do
8      foreach pair (v, w) do
9          δ[v][w] = min(δ[v][w],   δ[v][k] + δ[k][w])
10 return δ
```



Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 11 times**!

Recall that the weight of every edge is equal to 1.

*If the distance is **infinite**, write "INF". Note: There are exactly two fields where the correct answer is "INF". If you write "INF" in more than two fields, you get 0 points for those fields.*

$\delta[18][11] =$ **INF** $\quad\quad \delta[20][18] =$ **1**

$\delta[6][15] =$ **INF** $\quad\quad \delta[12][19] =$ **12**

$\delta[1][6] =$ **5**

12. [4 points] Consider the following algorithm.

```
Input: integer h
u = 0
l = 2
while l ≤ h do
    u = u + 4
    r = 2
    l = l − r
    u = u − r
    l = l + 8
```

Someone wrote the following invariant for the loop:

$$B \cdot l = A \cdot u + T$$

with the three constants

- $B = 15$
- $A = 36$
- $T = 24$

The invariant should hold each time the algorithm evaluates the loop condition. Unfortunately, the invariant has a mistake: one of the three constants, B, A, or T has a wrong value.

Fix the invariant by changing the value of one of the constants!

Which constant do you choose? _____**B**_____ *(Write the name of one of the three constants.)*

The new value for the chosen constant: _____**12**_____

Note: You are allowed to change only one constant.

13. 4 points (bonus)

Consider the following decision problem A:

You are configuring n Boolean security sensors, and the system's safety requirements are expressed as a complex logical formula over these sensor variables using "and", "or", and "not". You are also given a specific assignment of true/false to each sensor. Does this assignment satisfy the safety formula?

Consider the following decision problem B:

You manage a server with limited disk space. You have a set of files, all of which have exactly the same size s GB, and each file has an importance rating. Can you choose a subset of these files whose total size does not exceed S GB and whose total importance is at least V ?

Mark all the correct sentences and give a short explanation.

- √ **A can be reduced to B in polynomial time.**
- √ **B can be reduced to A in polynomial time.**
- √ **A is in P.**
- √ **B is in P.**
- ◯ It is unknown whether A can be reduced to B in polynomial time.
- ◯ It is unknown whether B can be reduced to A in polynomial time.
- ◯ It is unknown whether A is in P.
- ◯ It is unknown whether B is in P.

Solution: Both problems A and B are in P as they can be solved in polynomial time. To solve problem A, we evaluate the given assignment in polynomial time by recursively computing each subformula's value. To solve problem B, since every file has identical size s, you can store at most $k = \lfloor S/s \rfloor$ files; simply sort the files by importance in descending order, sum the top k ratings (in polynomial time), and compare to V.

Consequently, as problems A and B can be solved in polynomial time, they can be reduced in polynomial time to any other problem, thus also to each other.

Algorithms for Data Science
Exam 2025
Group **D**

June 2025

Your Name: _____ **Your Student ID:** _____

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	5	4	4	6	5	9	5	4	4	5	5	4	0	60
Score:														

The last task gives 4 bonus points. Thus, you can obtain 60+4 points in total (regular+bonus points).

Check if your name and student ID are printed on top of every single page!

Write your name and your student ID on the first page!

Check whether you received all 14 pages (containing 13 tasks in total).

The regular time to solve the exam is 90 minutes.

1. [5 points] Consider the following incomplete Python function. A sequence of unit blocks is dropped one by one onto integer coordinates along a line; when a block lands on an occupied coordinate, it stacks on top, raising that column's height by 1.
Coordinates may be arbitrarily large and all start at height 0. The function *blocks*(*pos*,*h*) takes the list *pos* of drop positions and the target height *h*, and returns the x-coordinate where a column's height first reaches *h*.

   Hint: the task is related to the third programming task.

```
1  def blocks(pos, h):
2      n = len(pos)
3      # ??? [Line 3]
4      for i in range(n):
5          x = pos[i]
6          # ??? [Line 6] (Starts with if..)
7              # ??? [Line 7]
8          else:
9              # ??? [Line 9]
10         # ??? [Line 10]
11             return x
```

   Complete the code by choosing a correct code snippet for each empty line (that starts with `# ???`). Do it by filling the blanks with the correct labels from the code snippets below:

   - Line 3: _____**G**_____
   - Line 6: _____**O**_____
   - Line 7: _____**A**_____
   - Line 9: _____**I**_____
   - Line 10: _____**C**_____

   Code snippets to choose from:

   - (A) `heights[x] = 1`
   - (B) `if heights[x] == 0:`
   - (C) `if heights[x] == h:`
   - (D) `if heights[h] == i:`
   - (E) `heights[x] += i`
   - (F) `heights[i] += x`
   - (G) `heights = {}`
   - (H) `heights = [0] * max(pos)`
   - (I) `heights[x] += 1`
   - (J) `heights[i] = x`
   - (K) `heights[x] = h`
   - (L) `if x < len(heights):`
   - (M) `heights = [0] * n`
   - (N) `if heights[x] < h:`
   - (O) `if x not in heights:`

   Note: The indendation of your code is the same as of `# ???`*. In other words, your code for an empty line will start at the position of* `#`*.*

2. [4 points] Complete the following algorithm `AlgoA(`$a[1..n]$`)` such that its running time $T(n)$ satisfies the recurrence

$$T(n) = 21T(\lceil n/28 \rceil) + \Theta(n^{10}) \ .$$

You don't need to understand what the algorithm computes!

The input of `AlgoA(`$a[1..n]$`)` *is an array* a *of* n *integers.*

The input of the auxiliary function `Copy_Sub_Array(`$a[1..n]$`,`i`,`m`)` *is an array* $a[1..n]$*, a position* i *and a length* m*. The function returns a copy of the subarray* $a[i+1..i+m]$*.*

```
AlgoA(a[1..n])
    if n < 29 then
        return 21
    else
        d = 7
        m = ⌈n/(4 · d)⌉
        sum = 0
        for i = 1 to 6 do
            b = Copy_Sub_Array(a,i,m)
            sum = sum + AlgoA(b)
        for i = 1 to 15 do
            b = Copy_Sub_Array(a,i,m)
            sum = sum + AlgoA(b)
        w = 1
        for j = 1 to 10 do
            w = w · n // Time: O(1)
        for k = 1 to w do
            sum = sum + a[29]
        return sum
```

```
Copy_Sub_Array(a[1..n],i,m)
    b[1..m] = new empty array
    for j = 1 to m do
        b[j] = a[i + j]
    return b
```

Fill in the three blanks in the algorithm above, each time writing **a single appropriate integer number**!

3. [4 points] Solve the following recurrence using the master theorem. Determine (for yourself) the values for a, b, and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = 10^4 \cdot 10 \cdot T(n/10) + \Theta(n^5)$$

Your answers:

$c =$ **5**

Case: **B**

Resulting running time $T(n) = \Theta(n^5 \log n)$

4. Simplify the following running times as far as possible:

(a) [2 points] $\Theta(\ (16^{\log_{16} n})^{16} + (2^{n/16})^2\ ) = \Theta($ $2^{n/8}$ $)$

(b) [2 points] $\Theta(\ 4\log_2 n + n^9 \log_2 n + 2 \cdot n^{10}\ ) = \Theta($ n^{10} $)$

(c) [2 points] $\Theta(\ 9 \cdot n^{7/10} + 1 \cdot n^{3/5} \cdot n^{2/5}\ ) = \Theta($ n $)$

5. [5 points] Consider the following array $a[1..21]$ containing twenty-one elements:

$$a = \langle 105, 75, 100, 65, 60, 85, 95, 40, 70, 55, 45, 80, 85, 15, 90, 30, 35, 25, 65, 20, 50 \rangle$$

1. Draw the *complete* array as a complete binary tree (as defined for heaps),
2. Mark the edge whose endpoints (parent and child) violate the heap invariant,
3. Choose an appropriate index position j and **increase** the value of $a[j]$ by the **maximum amount possible** such that a satisfies the heap invariant.

What index j do you choose?
(Write a single number between 1 *and* 21*; there is only one correct answer.)*

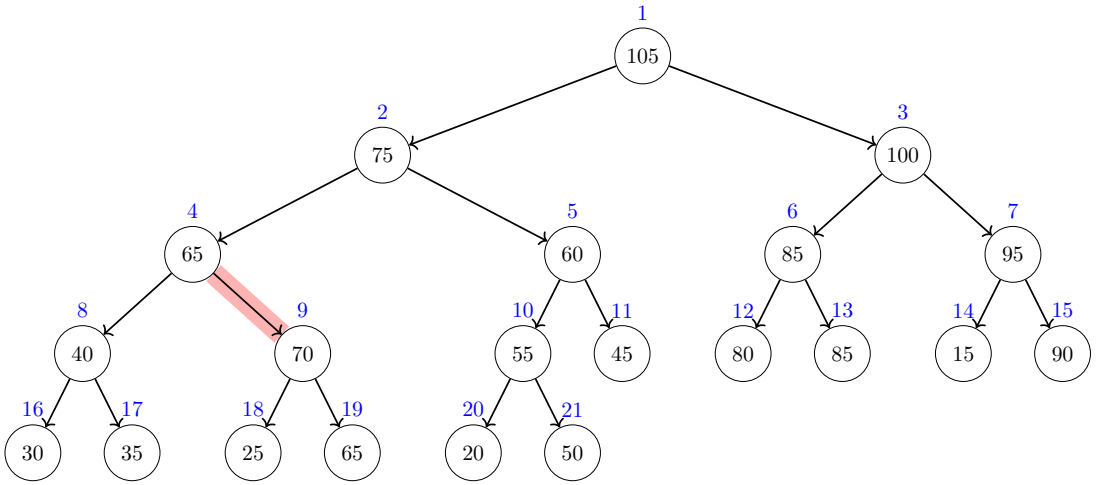
Your answer: $j =$ **4** ***(the violating edge's other endpoint is at* 9*.)***

What is the new value for $a[j]$?
(Write a single number, as large as possible and larger than the old value of $a[j]$*.)*

Your answer: $a[j] =$ **75** ***(the largest value from* [70, 75]*)***

Your tree hould contain ***all* the elements of the array**, from $a[1] = 105$ up to $a[21] = 50$:

Solution:



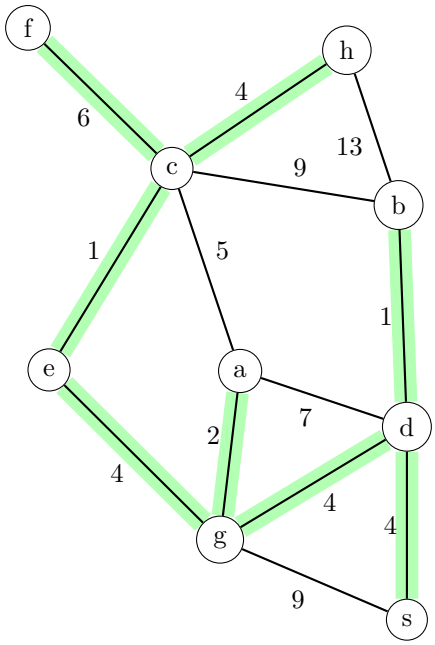
Explanation (not required): The heap invariant is violated at the red edge as the parent (index **4***) has a smaller value than its child (index* **9***). To fix the heap invariant, it suffices to increase* $a[4]$ *to any value between* 70 *and* 75*. The largest possible value (as asked in the task) is thus* 75*.*

6. [9 points] Dijkstra's algorithm (see below) is run on the following graph where the start node is s.

   What are the values of the *dist* and *parent* tables when Dijkstra's algorithm has finished? Fill in the missing entries in the table below!

   Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```
dist[0..n − 1] filled with INF
parent[0..n − 1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time then
        for every edge (v, w) do
            if dist[w] > dist[v] + c_{v,w} then
                dist[w] = dist[v] + c_{v,w}
                parent[w] = v
                q.add(w, -dist[w])
```



Solution:

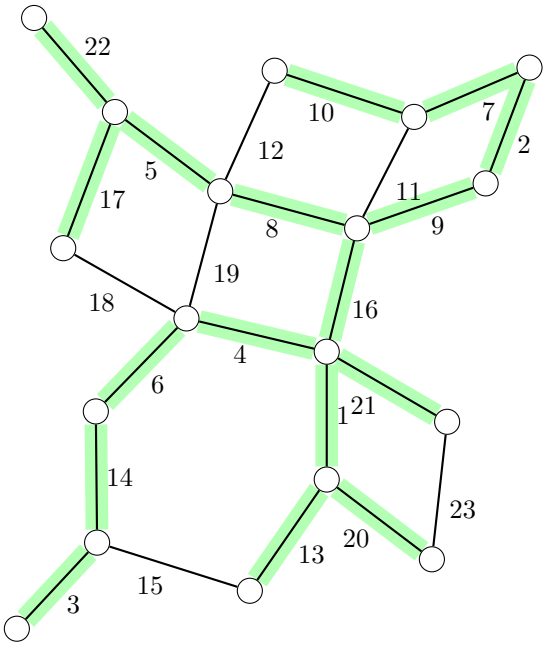
	a	b	c	d	e	f	g	h	s
distance	10	5	13	4	12	19	8	17	0
parent	g	d	e	s	g	c	d	c	NONE

Note:

- *The entries of the parent table consist of the names of the nodes or the special value* `NONE`*. For example, the parent of some node could be* s *whereas the parent of* s *is* `NONE`*.*
- *If a vertex has more neighbors, we consider them* ***in lexicographic order*** *(that is, node a would come before node b and so on)!*

7. [5 points] Mark all the edges that belong to the minimum spanning tree of the following graph. *(Marking an edge means to draw along that edge.)*

Solution:



8. [4 points] Add the following keys (in the order as given) into the hash table below using the hash function

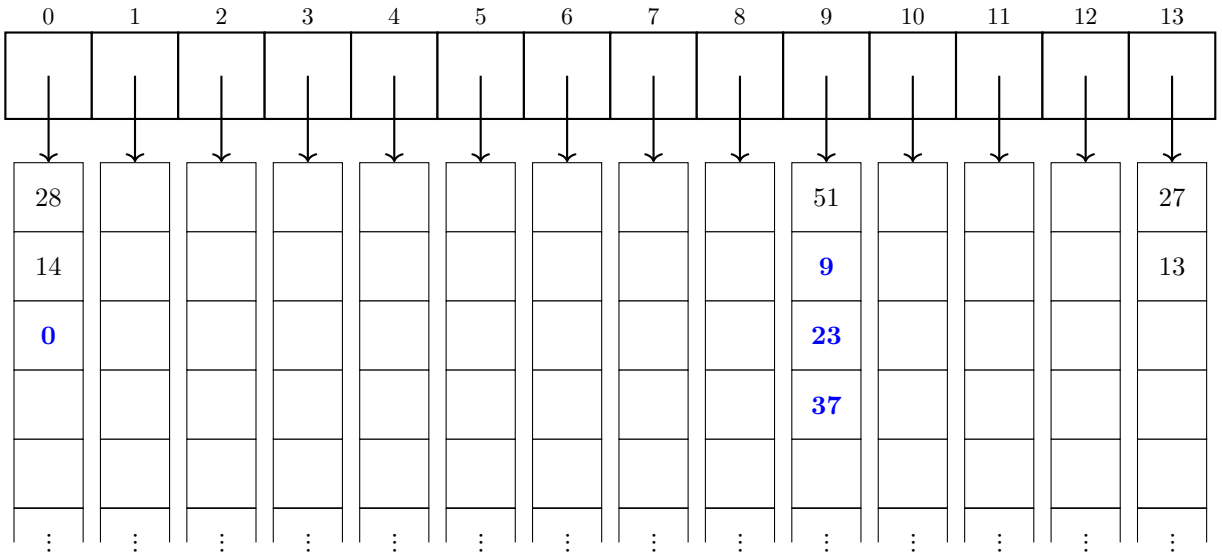
$$h(k) = k \mod 14$$

Keys: 9, 23, 0, 37

Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored (in the order as added; the first element of an array is at the top).

Note that for simplicity, we consider only keys and not key-value pairs. Also note that the hash table has been already filled with some other keys.

Solution:
Hash table:



9. [4 points] What is the most appropriate data structure for the following scenario, chosen from the list below?

   Unless stated otherwise in the scenario, the goal is to minimize memory usage and the worst-case running time of each operation.

   In your answer, briefly explain how your chosen data structure applies to the scenario. Then give its memory and time complexity, and explain why it performs better than *every* other option, considering the scenario's specific constraints and goals.

   You are implementing the task scheduling subsystem of a web browser. Resource requests (images, scripts, etc.) arrive in the order the page's HTML is parsed, and must be fetched in that same order. Request IDs can be extremely large. You need to support adding new requests and removing the oldest one, both in the smallest worst-case time.

   1. Array indexed by keys
   2. AVL tree
   3. Find-union
   4. Hash table
   5. Heap/Priority queue
   6. Sorted Linked list
   7. Queue
   8. Stack
   9. Sorted array
   10. Unsorted array

   Most appropriate data structure: _____**Queue**_____

   Explanation:

   Solution: In this fetch-scheduler, we always process the earliest-seen request next, never a newer one. A first-in-first-out structure as a queue is essential to respect request ordering.
   The running times of a queue are best possible as `Add` and `ExtractOldest` take only constant time. The memory usage is also best possible as the size of the queue is proportional to the number of elements it contains.

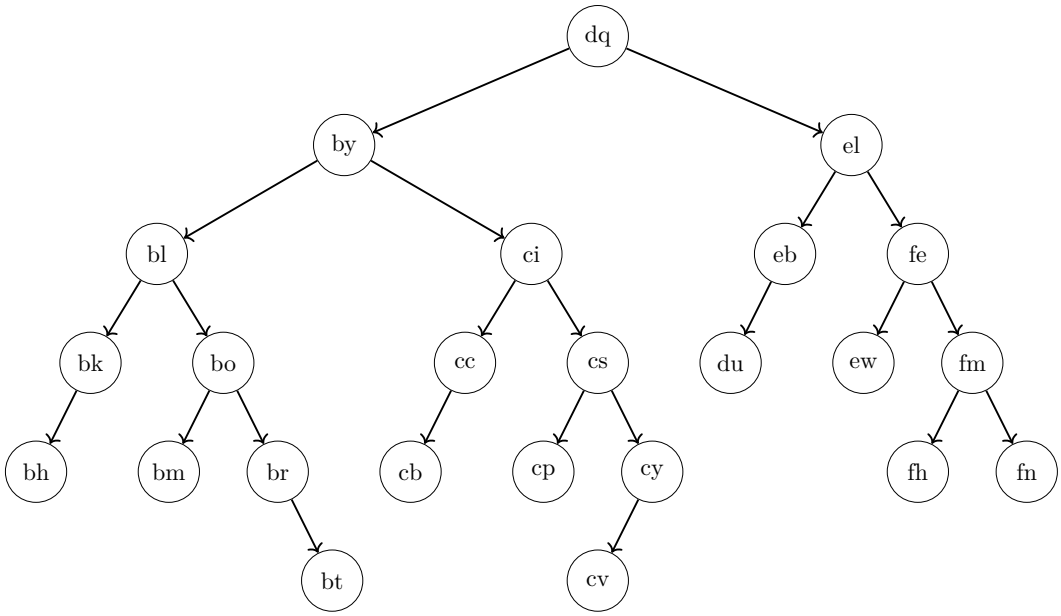
   All the other data structures are worse under this scenario as they don't provide feasible operations (stack, heap, find-union), take more time in the worst case (AVL tree, hash table, sorted/unsorted array, sorted linked list), or take too much memory as the maximum key is very large (array indexed by keys)

10. [5 points] What is the height of the node *bl* in this binary search tree, what is the height of the root (as defined in the lecture and exercises)?

Recall that the height of a node v is the number of nodes *on a longest path starting at v within the subtree of v (not counting the dummy "NONE" nodes that have height* 0*).*

Height of *bl*: _____**4**_____ (*Write only a single number.*)

Height of the root: _____**6**_____ (*Write only a single number.*)



Is the AVL invariant violated? Your answer: _____**no**_____ (*Write "yes" or "no".*)

Write the names of two nodes,

X=_____**fn**_____ and Y=_____**el**_____, such that the following holds:

- *If the tree violates the AVL invariant*, then the violation occurs at node X (caused by the heights of X's children), and adding a new child to node Y will restore the AVL invariant.
- *Otherwise, if the tree satisfies the AVL invariant*, then adding a new child to node X will cause a violation of the AVL invariant, and the violation will occur at node Y (caused by the heights of Y's children).
- *If there is more than one valid choice for X*, choose the one with the alphabetically **largest** name. **After selecting** X, if there is more than one valid choice for Y, choose the one with the alphabetically **smallest** name. (*Example: "bh" is alphabetically smaller than "fn".)*

In other words, X is either a node that violates the AVL invariant, or a node where adding a new child would cause such a violation. Similarly, Y is either a node where adding a new child restores the AVL invariant, or it is a node that violates the invariant after a new child is added to X. If there are multiple valid choices for X, choose the one with the alphabetically largest name. If there are multiple valid choices for Y (given your choice of X), choose the one with the alphabetically smallest name.

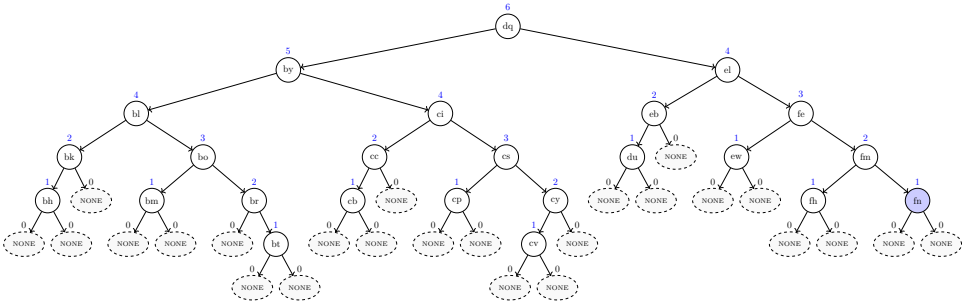
Note that there is exactly one correct solution that satisfies all the conditions above. It is also possible that $X = Y$.

See next page for an explanation of the solution!

Explanation:
The tree satisfies the AVL invariant. There are seven different candidates for X, where adding a node leads to a violation of the AVL invariant:

bh, **bt**, **cb**, **cv**, **du**, **fh** and **<u>fn</u>**.

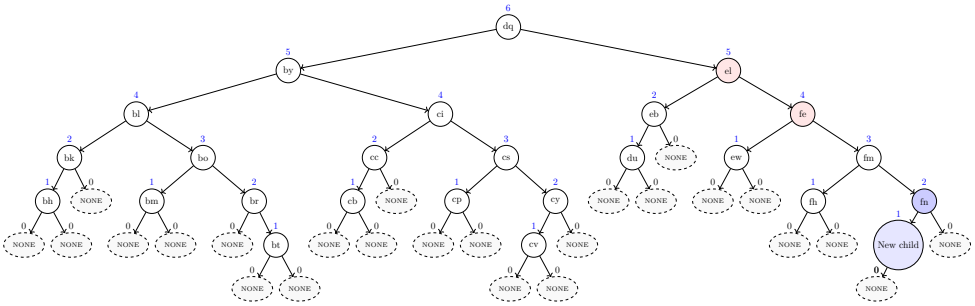
Recall that out of these different possibilties, the task asks you to choose the one where X is the alphabetically largest node; hence, $X = \text{fn}$.
The first tree shows the heights and dummy nodes, as well as the node $\boldsymbol{X = \text{fn}}$ (in blue).



The second tree shows the new child (also in blue) that has been added to X as well as all the two violating parent nodes in red (whose children differ in height by more than 1):

fe and **<u>el</u>**.

Among these violating nodes, the task asks you to choose the alphabetically smallest one; hence $\boldsymbol{Y = \text{el}}$.



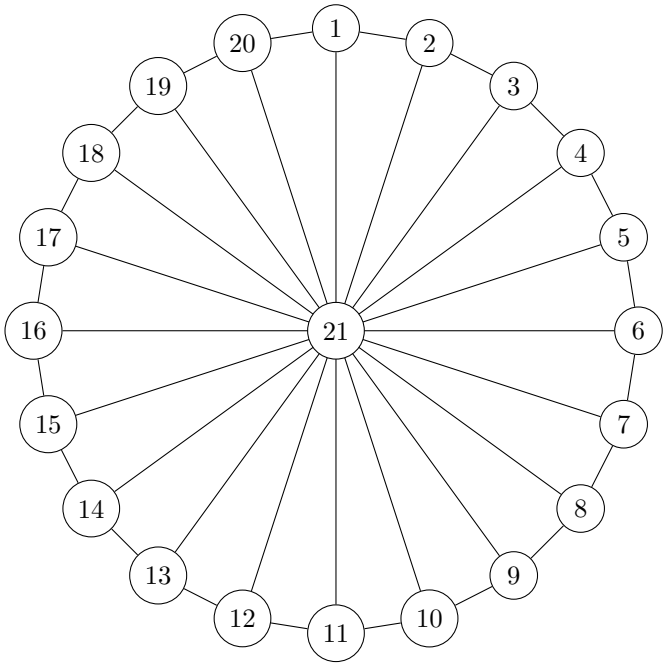
For completness, for each (wrong) candidate for X a list of violating parent nodes. The alphabetically optimal violating parent node is underlined.

- Candidate **bh** →**<u>bk</u>**
- Candidate **bt** →**br**, **bo**, **<u>bl</u>** and **dq**
- Candidate **cb** →**<u>cc</u>**
- Candidate **cv** →**cy**, **cs**, **<u>ci</u>** and **dq**
- Candidate **du** →**<u>eb</u>**
- Candidate **fh** →**fe** and **<u>el</u>**

11. [5 points] Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to n=21 and each edge has the same weight 1. Observe that the vertices 1 to 20 lie on a big cycle and each of them is connected to the center vertex 21 via an edge.

    Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 7) already after 12 iterations.

```
1  δ[1..21][1..21] = new 2D array filled with +∞
2  for i = 1 to 21 do
3      δ[i][i] = 0
4  foreach edge (v, w, c) do
       // v and w are the endpoints and c is the weight of the edge
5      δ[v][w] = c
6      δ[w][v] = c
7  for k = 1 to 12 do
8      foreach pair (v, w) do
9          δ[v][w] = min(δ[v][w],   δ[v][k] + δ[k][w])
10 return δ
```



Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 12 times**!

Recall that the weight of every edge is equal to 1.

*If the distance is **infinite**, write "INF". Note: There are exactly two fields where the correct answer is "INF". If you write "INF" in more than two fields, you get 0 points for those fields.*

$\delta[12][19] =$ **INF** $\quad\quad \delta[20][13] =$ **13**

$\delta[1][11] =$ **10** $\quad\quad \delta[16][12] =$ **INF**

$\delta[14][21] =$ **1**

12. [4 points] Consider the following algorithm.

```
Input: integer q
t = 0
b = 3
while q > q do
    b = b + 8
    h = 4
    t = t − h
    b = b − h
    t = t + 2
```

Someone wrote the following invariant for the loop:

$$Y = E \cdot b + G \cdot t$$

with the three constants

- $Y = 42$
- $E = 12$
- $G = 28$

The invariant should hold each time the algorithm evaluates the loop condition. Unfortunately, the invariant has a mistake: one of the three constants, Y, E, or G has a wrong value.

Fix the invariant by changing the value of one of the constants!

Which constant do you choose? _____**E**_____ *(Write the name of one of the three constants.)*

The new value for the chosen constant: _____**14**_____

Note: You are allowed to change only one constant.

13. 4 points (bonus)

Consider the following decision problem A:

You are configuring n Boolean security sensors, and the system's safety requirements are expressed as a complex logical formula over these sensor variables using "and", "or", and "not". Is there an assignment of true/false to the sensors that makes the formula true?

Consider the following decision problem B:

You are given the floor plan of a museum consisting of rooms connected by corridors. You need to determine whether there exists any route from the entrance to the exit that does not pass through the same room more than once.

Mark all the correct sentences and give a short explanation.

- ◯ A can be reduced to B in polynomial time.
- √ **B can be reduced to A in polynomial time.**
- ◯ A is in P.
- √ **B is in P.**
- √ **It is unknown whether A can be reduced to B in polynomial time.**
- ◯ It is unknown whether B can be reduced to A in polynomial time.
- √ **It is unknown whether A is in P.**
- ◯ It is unknown whether B is in P.

Solution: Problem A is equivalent to the NP-complete problem SAT (by viewing each sensor as a Boolean variable, the safety policy as a logical Boolean SAT formula, and asking whether the formula is satisfiable), whereas problem B can be solved in polynomial time: we interpret rooms as vertices and corridors as edges and ask whether there is a path from the entrance to the exit, which can be decided in polynomial time using breadth-first search (BFS) (we can ignore the "no repeats" requirement as any walk from the entrance to the exit that revisits rooms implies that a simple path without repeats exists).

Consequently, as problem B can be solved in polynomial time, it can reduced in polynomial time to any other problem, thus also to A.

Since it is unknown whether NP-complete problems can be solved in polynomial time, we do not know whether A is in P, and, consequently, whether there exist a polynomial time reduction from A to any polynomial-time problem (e.g., to B).

Algorithms for Data Science
Exam 2025
Group **E**

June 2025

Your Name: _____ **Your Student ID:** _____

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	5	4	4	6	5	9	5	4	4	5	5	4	0	60
Score:														

The last task gives 4 bonus points. Thus, you can obtain 60+4 points in total (regular+bonus points).

Check if your name and student ID are printed on top of every single page!

Write your name and your student ID on the first page!

Check whether you received all 14 pages (containing 13 tasks in total).

The regular time to solve the exam is 90 minutes.

1. [5 points] Consider the following incomplete Python function. A sequence of unit blocks is dropped one by one onto integer coordinates along a line; when a block lands on an occupied coordinate, it stacks on top, raising that column's height by 1.
Coordinates may be arbitrarily large and all start at height 0. The function *blocks*(*pos*,*h*) takes the list *pos* of drop positions and the target height *h*, and returns the x-coordinate where a column's height first reaches *h*.

   Hint: the task is related to the third programming task.

```
1  def blocks(pos, h):
2      n = len(pos)
3      # ??? [Line 3]
4      for i in range(n):
5          x = pos[i]
6          # ??? [Line 6] (Starts with if..)
7              # ??? [Line 7]
8          else:
9              # ??? [Line 9]
10         # ??? [Line 10]
11             return x
```

   Complete the code by choosing a correct code snippet for each empty line (that starts with `# ???`). Do it by filling the blanks with the correct labels from the code snippets below:

   - Line 3: **D**
   - Line 6: **G**
   - Line 7: **O**
   - Line 9: **A**
   - Line 10: **I**

   Code snippets to choose from:

   - (A) `heights[x] += 1`
   - (B) `heights[i] = x`
   - (C) `heights = [0] * n`
   - (D) `heights = {}`
   - (E) `heights[i] += x`
   - (F) `if heights[x] < h:`
   - (G) `if x not in heights:`
   - (H) `if x < len(heights):`
   - (I) `if heights[x] == h:`
   - (J) `heights[x] = h`
   - (K) `heights[x] += i`
   - (L) `if heights[x] == 0:`
   - (M) `heights = [0] * max(pos)`
   - (N) `if heights[h] == i:`
   - (O) `heights[x] = 1`

   Note: The indendation of your code is the same as of `# ???`*. In other words, your code for an empty line will start at the position of* `#`*.*

2. [4 points] Complete the following algorithm `AlgoA(`$a[1..n]$`)` such that its running time $T(n)$ satisfies the recurrence

$$T(n) = 11T(\lceil n/30 \rceil) + \Theta(n^6) \ .$$

You don't need to understand what the algorithm computes!

The input of `AlgoA(`$a[1..n]$`)` *is an array* a *of* n *integers.*

The input of the auxiliary function `Copy_Sub_Array(`$a[1..n]$`,`i`,`$m`)` *is an array* $a[1..n]$*, a position* i *and a length* m*. The function returns a copy of the subarray* $a[i+1..i+m]$*.*

```
AlgoA(a[1..n])
    if n < 31 then
        return 15
    else
        d = 15
        m = ⌈n/(2 · d)⌉
        sum = 0
        for i = 1 to 6 do
            b = Copy_Sub_Array(a,i,m)
            sum = sum + AlgoA(b)
        for i = 1 to 5 do
            b = Copy_Sub_Array(a,i,m)
            sum = sum + AlgoA(b)
        w = 1
        for j = 1 to 6 do
            w = w · n // Time: O(1)
        for k = 1 to w do
            sum = sum + a[31]
        return sum
```

```
Copy_Sub_Array(a[1..n],i,m)
    b[1..m] = new empty array
    for j = 1 to m do
        b[j] = a[i + j]
    return b
```

Fill in the three blanks in the algorithm above, each time writing **a single appropriate integer number**!

3. [4 points] Solve the following recurrence using the master theorem. Determine (for yourself) the values for a, b, and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = 9 \cdot T(n/3) + \Theta(n^3)$$

Your answers:

$c =$ **2**

Case: **C**

Resulting running time $T(n) =$ $\Theta(n^3)$

4. Simplify the following running times as far as possible:

(a) [2 points] $\Theta(\ (40^{\log_{40} n})^{40} + (8^{n/40})^8\ ) = \Theta($ $8^{n/5}$ $)$

(b) [2 points] $\Theta(\ 10n^8 + 36n^7 \log n + 8n^8\ ) = \Theta($ n^8 $)$

(c) [2 points] $\Theta(\ 8^{(n-n)} \cdot 10 + 8 \cdot n^{(8-8)} + 8\ ) = \Theta($ 1 $)$

5. [5 points] Consider the following array $a[1..21]$ containing twenty-one elements:

$$a = \langle 105, 105, 60, 90, 100, 50, 50, 85, 75, 25, 95, 55, 30, 45, 40, 35, 80, 70, 65, 15, 20 \rangle$$

1. Draw the *complete* array as a complete binary tree (as defined for heaps),
2. Mark the edge whose endpoints (parent and child) violate the heap invariant,
3. Choose an appropriate index position j and **increase** the value of $a[j]$ by the **maximum amount possible** such that a satisfies the heap invariant.

What index j do you choose?
(Write a single number between 1 *and* 21*; there is only one correct answer.)*

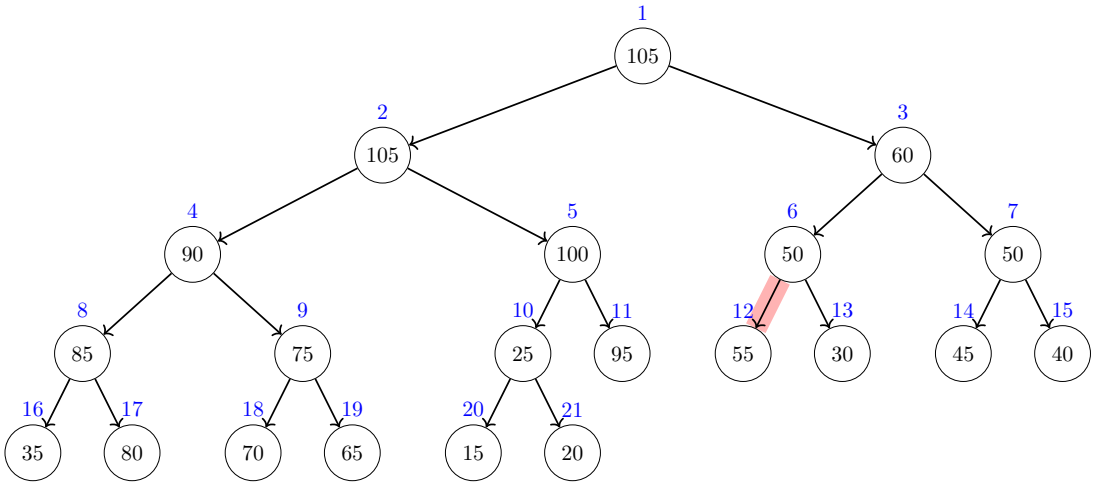
Your answer: $j =$ **6** ***(the violating edge's other endpoint is at* 12*.)***

What is the new value for $a[j]$?
(Write a single number, as large as possible and larger than the old value of $a[j]$*.)*

Your answer: $a[j] =$ **60** ***(the largest value from* [55, 60]*)***

Your tree hould contain ***all* the elements of the array**, from $a[1] = 105$ up to $a[21] = 20$:

Solution:



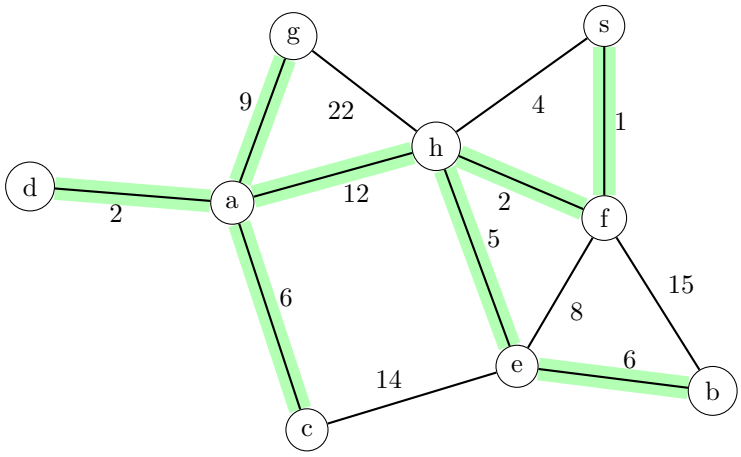
Explanation (not required): The heap invariant is violated at the red edge as the parent (index **6***) has a smaller value than its child (index* **12***). To fix the heap invariant, it suffices to increase* $a[6]$ *to any value between* 55 *and* 60*. The largest possible value (as asked in the task) is thus* 60*.*

6. [9 points] Dijkstra's algorithm (see below) is run on the following graph where the start node is s.

   What are the values of the *dist* and *parent* tables when Dijkstra's algorithm has finished? Fill in the missing entries in the table below!

   Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```
dist[0..n − 1] filled with INF
parent[0..n − 1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time then
        for every edge (v, w) do
            if dist[w] > dist[v] + c_{v,w} then
                dist[w] = dist[v] + c_{v,w}
                parent[w] = v
                q.add(w, -dist[w])
```



Solution:

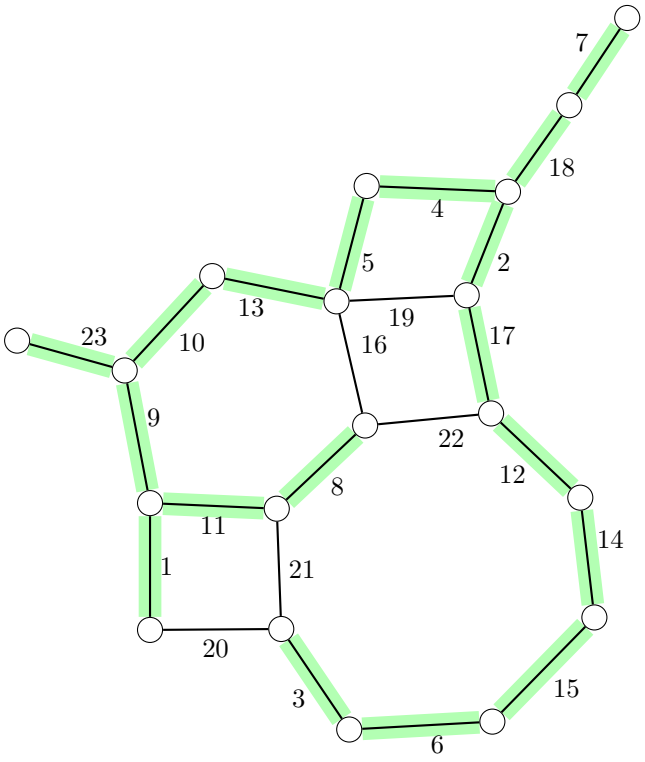
	a	b	c	d	e	f	g	h	s
distance	15	14	21	17	8	1	24	3	0
parent	h	e	a	a	h	s	a	f	NONE

Note:

- *The entries of the parent table consist of the names of the nodes or the special value* `NONE`*. For example, the parent of some node could be* s *whereas the parent of* s *is* `NONE`*.*
- *If a vertex has more neighbors, we consider them* ***in lexicographic order*** *(that is, node* a *would come before node* b *and so on)!*

7. [5 points] Mark all the edges that belong to the minimum spanning tree of the following graph. *(Marking an edge means to draw along that edge.)*

Solution:



8. [4 points] Add the following keys (in the order as given) into the hash table below using the hash function

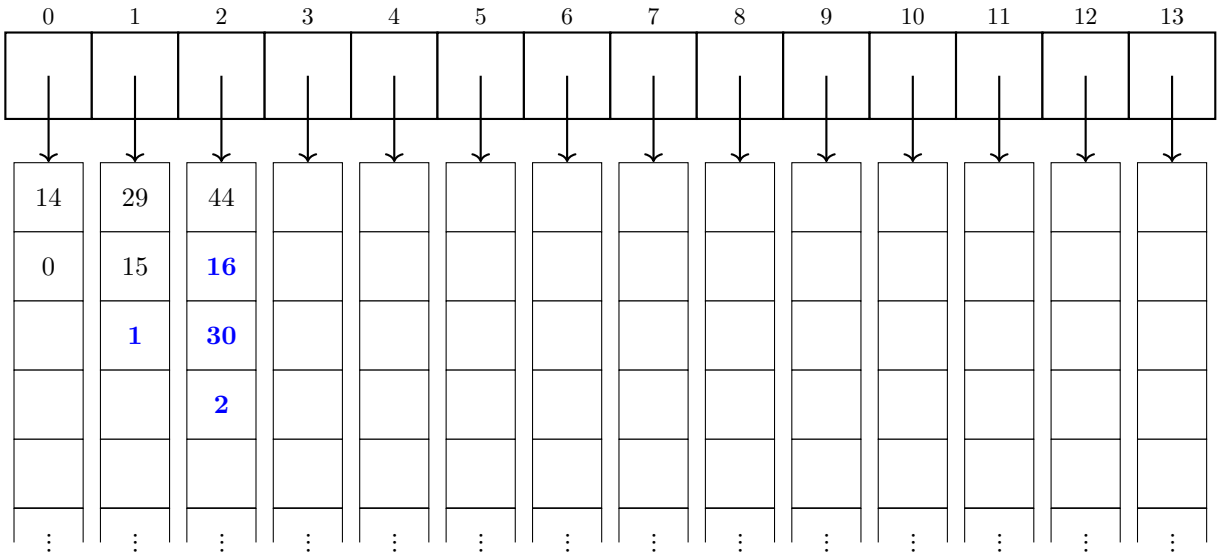
$$h(k) = k \mod 14$$

Keys: 1, 16, 30, 2

Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored (in the order as added; the first element of an array is at the top).

Note that for simplicity, we consider only keys and not key-value pairs. Also note that the hash table has been already filled with some other keys.

Solution:
Hash table:



9. [4 points] What is the most appropriate data structure for the following scenario, chosen from the list below?

   Unless stated otherwise in the scenario, the goal is to minimize memory usage and the worst-case running time of each operation.

   In your answer, briefly explain how your chosen data structure applies to the scenario. Then give its memory and time complexity, and explain why it performs better than *every* other option, considering the scenario's specific constraints and goals.

   You are developing the backend of a social media platform. Each time a user logs in, you store their session token and profile data under a unique user ID (which can be a very large integer). You must support frequent logins and logouts as well as lookups of session data by user ID, aiming for very fast performance in practice—even though very rare slowdowns are allowed to occur.

   1. Array indexed by keys
   2. AVL tree
   3. Find-union
   4. Hash table
   5. Heap/Priority queue
   6. Sorted Linked list
   7. Queue
   8. Stack
   9. Sorted array
   10. Unsorted array

   Most appropriate data structure: _____**Hash table**_____

   Explanation:

   Solution: We store each session under its user ID in a hash table, so that inserting, retrieving, and deleting sessions are essentially instantaneous in everyday use, with only highly unlikely slow operations.
   The running times of all operations are practically optimal: `Add`, `Get` and `Remove` take constant time in the expected case, that is, with high probability they run in $O(1)$, making them very efficient in most use cases. Occasional slow operations are rare and acceptable in our setting. The memory usage is proportional to the number of stored elements.

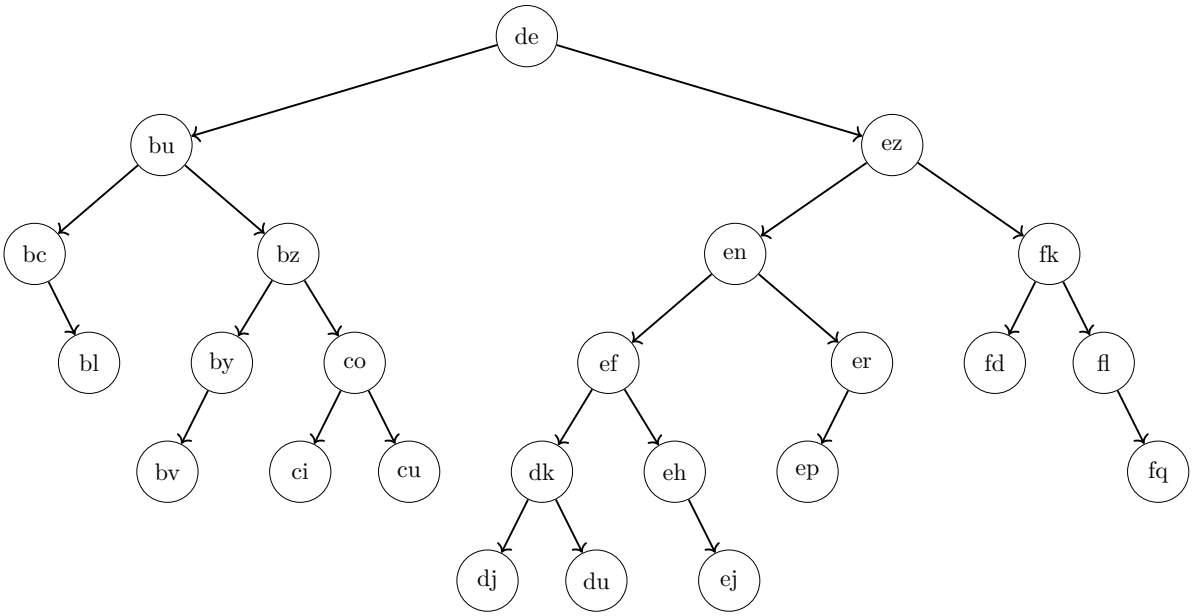
   All the other data structures are worse under this scenario as they don't provide feasible operations (stack, queue, heap, find-union), take more time in the expected case (AVL tree, sorted/unsorted array, sorted linked list) or take too much memory if the maximum key is very large (array indexed by keys).

10. [5 points] What is the height of the node *ez* in this binary search tree, what is the height of the root (as defined in the lecture and exercises)?

Recall that the height of a node v is the number of nodes *on a longest path starting at v within the subtree of v (not counting the dummy "NONE" nodes that have height* 0*).*

Height of *ez*: **5** (*Write only a single number.*)

Height of the root: **6** (*Write only a single number.*)



Does the tree satisfy the AVL invariant? Your answer: **yes** (*Write "yes" or "no".*)

Write the names of two nodes,

X=**fq** and Y=**fl**, such that the following holds:

- *If the tree violates the AVL invariant*, then the violation occurs at node X (caused by the heights of X's children), and adding a new child to node Y will restore the AVL invariant.
- *Otherwise, if the tree satisfies the AVL invariant*, then adding a new child to node X will cause a violation of the AVL invariant, and the violation will occur at node Y (caused by the heights of Y's children).
- *If there is more than one valid choice for X*, choose the one with the alphabetically **largest** name. **After selecting** X, if there is more than one valid choice for Y, choose the one with the alphabetically **largest** name. (*Example: "bc" is alphabetically smaller than "fq".)*

In other words, X is either a node that violates the AVL invariant, or a node where adding a new child would cause such a violation. Similarly, Y is either a node where adding a new child restores the AVL invariant, or it is a node that violates the invariant after a new child is added to X. If there are multiple valid choices for X, choose the one with the alphabetically largest name. If there are multiple valid choices for Y (given your choice of X), choose the one with the alphabetically largest name.

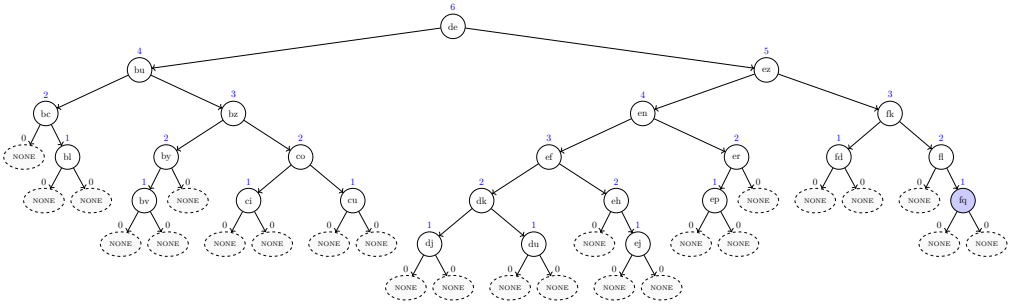
Note that there is exactly one correct solution that satisfies all the conditions above. It is also possible that $X = Y$.

See next page for an explanation of the solution!

Explanation:
The tree satisfies the AVL invariant. There are nine different candidates for X, where adding a node leads to a violation of the AVL invariant:

bl, **bv**, **ci**, **cu**, **dj**, **du**, **ej**, **ep** and **fq**.

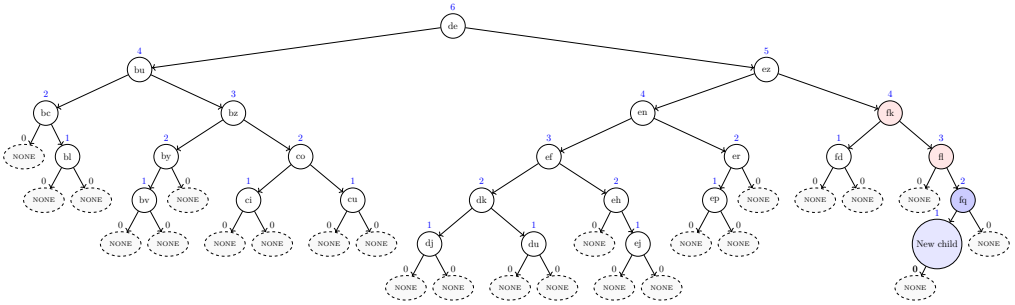
Recall that out of these different possibilties, the task asks you to choose the one where X is the alphabetically largest node; hence, $X = \text{fq}$.
The first tree shows the heights and dummy nodes, as well as the node $\boldsymbol{X} = \textbf{fq}$ (in blue).



The second tree shows the new child (also in blue) that has been added to X as well as all the two violating parent nodes in red (whose children differ in height by more than 1):

fl and **fk**.

Among these violating nodes, the task asks you to choose the alphabetically largest one; hence $\boldsymbol{Y} = \textbf{fl}$.



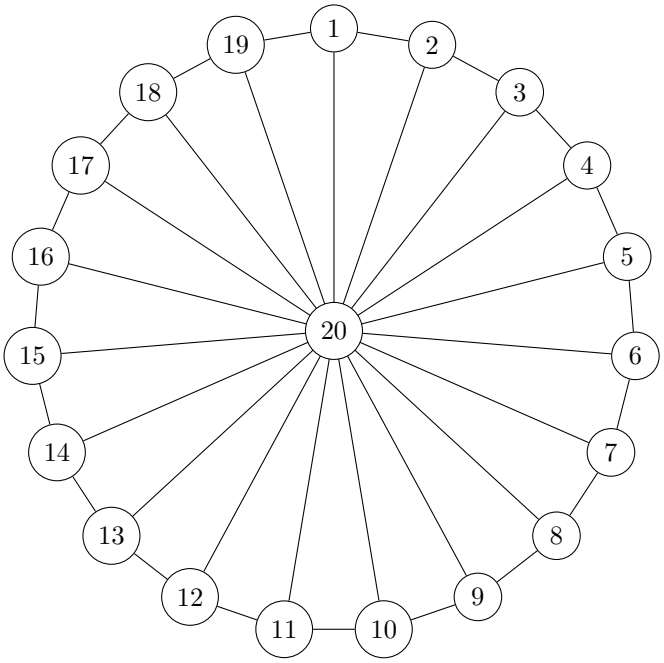
For completness, for each (wrong) candidate for X a list of violating parent nodes. The alphabetically optimal violating parent node is underlined.

- Candidate **bl** →**bc**
- Candidate **bv** →**by** and **bu**
- Candidate **ci** →**bu**
- Candidate **cu** →**bu**
- Candidate **dj** →**en**, **ez** and **de**
- Candidate **du** →**en**, **ez** and **de**
- Candidate **ej** →**eh**, **en**, **ez** and **de**
- Candidate **ep** →**er**

11. [5 points] Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to n=20 and each edge has the same weight 1. Observe that the vertices 1 to 19 lie on a big cycle and each of them is connected to the center vertex 20 via an edge.

    Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 7) already after 13 iterations.

```
1  δ[1..20][1..20] = new 2D array filled with +∞
2  for i = 1 to 20 do
3      δ[i][i] = 0
4  foreach edge (v, w, c) do
       // v and w are the endpoints and c is the weight of the edge
5      δ[v][w] = c
6      δ[w][v] = c
7  for k = 1 to 13 do
8      foreach pair (v, w) do
9          δ[v][w] = min(δ[v][w],   δ[v][k] + δ[k][w])
10 return δ
```



Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 13 times**!

Recall that the weight of every edge is equal to 1.

*If the distance is **infinite**, write "INF". Note: There are exactly two fields where the correct answer is "INF". If you write "INF" in more than two fields, you get 0 points for those fields.*

$\delta[13][18] =$ **INF** $\quad\quad \delta[4][10] =$ **6**

$\delta[18][6] =$ **INF** $\quad\quad \delta[14][19] =$ **14**

$\delta[17][20] =$ **1**

12. [4 points] Consider the following algorithm.

```
Input: integer b
f = 3
r = 0
while b ≥ f do
    r = r + 8
    z = −6
    f = f − z
    r = r + z
    f = f − 2
```

Someone wrote the following invariant for the loop:

$$K \cdot f = W \cdot r + S$$

with the three constants

- $K = 12$
- $W = 26$
- $S = 39$

The invariant should hold each time the algorithm evaluates the loop condition. Unfortunately, the invariant has a mistake: one of the three constants, K, W, or S has a wrong value.

Fix the invariant by changing the value of one of the constants!

Which constant do you choose? _____**K**_____ *(Write the name of one of the three constants.)*

The new value for the chosen constant: _____**13**_____

Note: You are allowed to change only one constant.

13. 4 points (bonus)

Consider the following decision problem A:

You run a newspaper delivery service along a single straight street with n homes numbered in order. You know the driving time between each pair of adjacent homes. Starting and ending at your depot at one end of the street, can you deliver to every home and return within a given time limit T?

Consider the following decision problem B:

Your network administrator has n Boolean firewall rules, each can be enabled or disabled. The overall security policy is given as a complex logical formula over these rule variables using "and", "or", and "not". You are also provided a proposed assignment of enable/disable values for each rule. Does this assignment satisfy the policy?

Mark all the correct sentences and give a short explanation.

- √ **A can be reduced to B in polynomial time.**
- √ **B can be reduced to A in polynomial time.**
- √ **A is in P.**
- √ **B is in P.**
- ◯ It is unknown whether A can be reduced to B in polynomial time.
- ◯ It is unknown whether B can be reduced to A in polynomial time.
- ◯ It is unknown whether A is in P.
- ◯ It is unknown whether B is in P.

Solution: Both problems A and B are in P as they can be solved in polynomial time. To solve problem A, we can ignore the general TSP complexity as on a line the unique optimal tour is to go from the depot to the farthest home and back; summing the adjacent-home travel times in order (in polynomial time) gives the total route time, which we compare to T. To solve problem B, we evaluate the given assignment in linear time by recursively computing each subformula's value.

Consequently, as problems A and B can be solved in polynomial time, they can be reduced in polynomial time to any other problem, thus also to each other.

Algorithms for Data Science
Exam 2025
Group **F**

June 2025

Your Name: _____ **Your Student ID:** _____

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	5	4	4	6	5	9	5	4	4	5	5	4	0	60
Score:														

The last task gives 4 bonus points. Thus, you can obtain 60+4 points in total (regular+bonus points).

Check if your name and student ID are printed on top of every single page!

Write your name and your student ID on the first page!

Check whether you received all 14 pages (containing 13 tasks in total).

The regular time to solve the exam is 90 minutes.

1. [5 points] Consider the following incomplete Python function. A sequence of unit blocks is dropped one by one onto integer coordinates along a line; when a block lands on an occupied coordinate, it stacks on top, raising that column's height by 1.
Coordinates may be arbitrarily large and all start at height 0. The function *blocks*(*pos*,*h*) takes the list *pos* of drop positions and the target height *h*, and returns the x-coordinate where a column's height first reaches *h*.

   Hint: the task is related to the third programming task.

```
1  def blocks(pos, h):
2      n = len(pos)
3      # ??? [Line 3]
4      for i in range(n):
5          x = pos[i]
6          # ??? [Line 6] (Starts with if..)
7              # ??? [Line 7]
8          else:
9              # ??? [Line 9]
10         # ??? [Line 10]
11             return x
```

   Complete the code by choosing a correct code snippet for each empty line (that starts with `# ???`). Do it by filling the blanks with the correct labels from the code snippets below:

   - Line 3: _____**N**_____
   - Line 6: _____**D**_____
   - Line 7: _____**G**_____
   - Line 9: _____**O**_____
   - Line 10: _____**A**_____

   Code snippets to choose from:

   - (A) `if heights[x] == h:`
   - (B) `heights[x] = h`
   - (C) `heights = [0] * max(pos)`
   - (D) `if x not in heights:`
   - (E) `if heights[x] < h:`
   - (F) `if heights[h] == i:`
   - (G) `heights[x] = 1`
   - (H) `if heights[x] == 0:`
   - (I) `heights = [0] * n`
   - (J) `heights[x] += i`
   - (K) `heights[i] += x`
   - (L) `heights[i] = x`
   - (M) `if x < len(heights):`
   - (N) `heights = {}`
   - (O) `heights[x] += 1`

   Note: The indendation of your code is the same as of `# ???`*. In other words, your code for an empty line will start at the position of* `#`*.*

2. [4 points] Complete the following algorithm `AlgoA(`$a[1..n]$`)` such that its running time $T(n)$ satisfies the recurrence

$$T(n) = 20T(\lceil n/24 \rceil) + \Theta(n^4) \ .$$

You don't need to understand what the algorithm computes!

The input of `AlgoA(`$a[1..n]$`)` *is an array* a *of* n *integers.*

The input of the auxiliary function `Copy_Sub_Array(`$a[1..n]$`,`i`,`m`)` *is an array* $a[1..n]$*, a position* i *and a length* m*. The function returns a copy of the subarray* $a[i{+}1..i{+}m]$*.*

```
AlgoA(a[1..n])
    if n < 25 then
        return 12
    else
        d = 12
        m = ⌈n/(2 · d)⌉
        sum = 0
        for i = 1 to 6 do
            b = Copy_Sub_Array(a,i,m)
            sum = sum + AlgoA(b)
        for i = 1 to 14 do
            b = Copy_Sub_Array(a,i,m)
            sum = sum + AlgoA(b)
        w = 1
        for j = 1 to 4 do
            w = w · n // Time: O(1)
        for k = 1 to w do
            sum = sum + a[25]
        return sum
```

```
Copy_Sub_Array(a[1..n],i,m)
    b[1..m] = new empty array
    for j = 1 to m do
        b[j] = a[i + j]
    return b
```

Fill in the three blanks in the algorithm above, each time writing **a single appropriate integer number**!

3. [4 points] Solve the following recurrence using the master theorem. Determine (for yourself) the values for a, b, and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = 27 \cdot T(n/3) + \Theta(n)$$

Your answers:

$c =$ **3**

Case: **A**

Resulting running time $T(n) =$ $\Theta(n^3)$

4. Simplify the following running times as far as possible:

(a) [2 points] $\Theta(\ 8 \cdot n^7 + n^6 \log_2 n + 10 \log_2 n\ ) = \Theta($ n^7 $)$

(b) [2 points] $\Theta(\ 8 \cdot n^{9/10} \cdot n^{1/10} + 7 \cdot n^{19/20}\ ) = \Theta($ n $)$

(c) [2 points] $\Theta(\ (8^{n/48})^8 + (48^{\log_{48} n})^{48}\ ) = \Theta($ $8^{n/6}$ $)$

5. [5 points] Consider the following array $a[1..21]$ containing twenty-one elements:

$$a = \langle 115, 90, 110, 50, 80, 105, 75, 30, 45, 85, 80, 95, 100, 70, 65, 20, 25, 40, 35, 60, 55 \rangle$$

1. Draw the *complete* array as a complete binary tree (as defined for heaps),
2. Mark the edge whose endpoints (parent and child) violate the heap invariant,
3. Choose an appropriate index position j and **increase** the value of $a[j]$ by the **maximum amount possible** such that a satisfies the heap invariant.

What index j do you choose?
(Write a single number between 1 *and* 21*; there is only one correct answer.)*

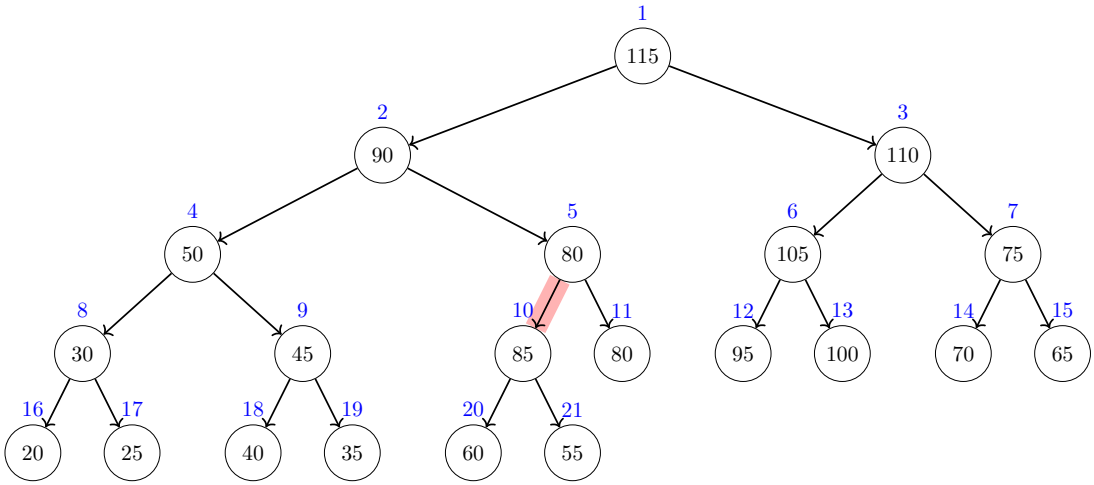
Your answer: $j =$ **5** ***(the violating edge's other endpoint is at* 10*.)***

What is the new value for $a[j]$?
(Write a single number, as large as possible and larger than the old value of $a[j]$*.)*

Your answer: $a[j] =$ **90** ***(the largest value from* [85, 90]*)***

Your tree hould contain ***all* the elements of the array**, from $a[1] = 115$ up to $a[21] = 55$:

Solution:



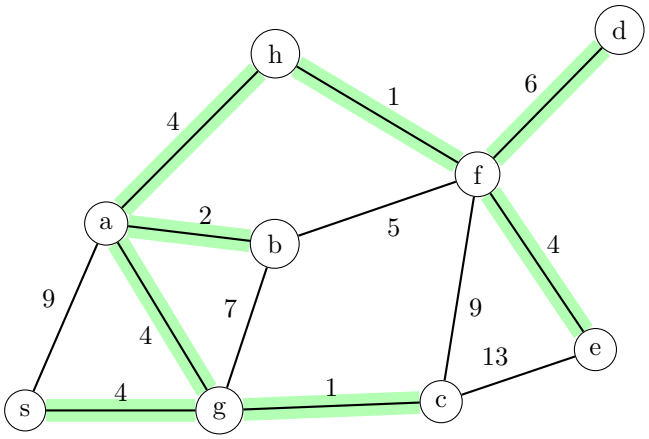
Explanation (not required): The heap invariant is violated at the red edge as the parent (index **5***) has a smaller value than its child (index* **10***). To fix the heap invariant, it suffices to increase* $a[5]$ *to any value between* 85 *and* 90*. The largest possible value (as asked in the task) is thus* 90*.*

6. [9 points] Dijkstra's algorithm (see below) is run on the following graph where the start node is s.

   What are the values of the *dist* and *parent* tables when Dijkstra's algorithm has finished? Fill in the missing entries in the table below!

   Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```
dist[0..n − 1] filled with INF
parent[0..n − 1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time then
        for every edge (v, w) do
            if dist[w] > dist[v] + c_{v,w} then
                dist[w] = dist[v] + c_{v,w}
                parent[w] = v
                q.add(w, -dist[w])
```



Solution:

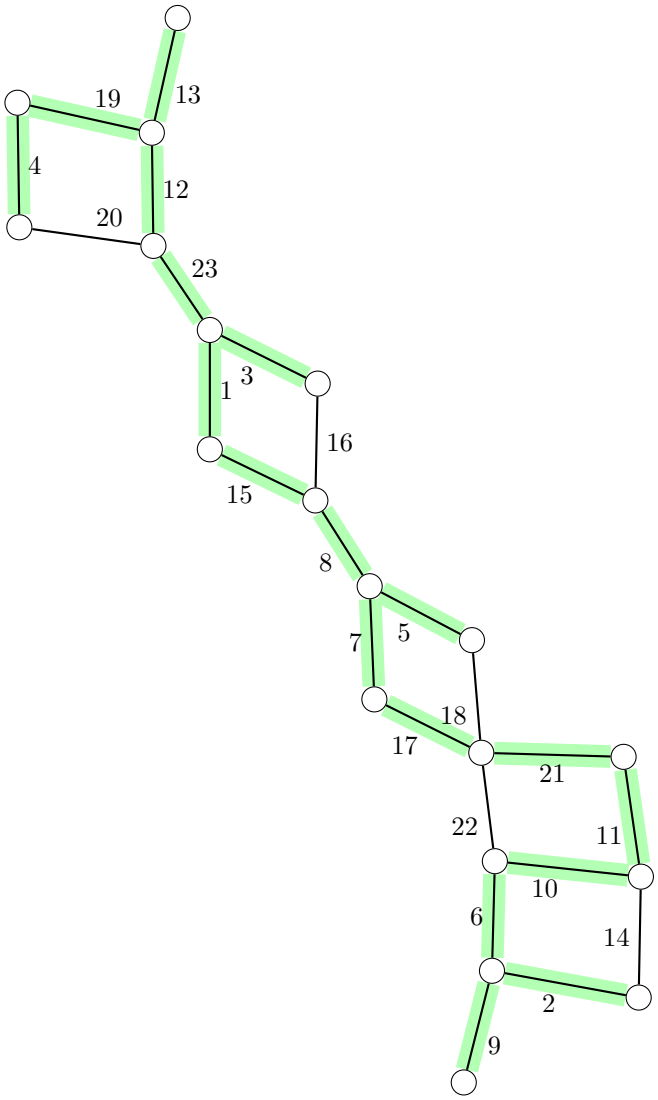
	a	b	c	d	e	f	g	h	s
distance	8	10	5	19	17	13	4	12	0
parent	g	a	g	f	f	h	s	a	NONE

Note:

- *The entries of the parent table consist of the names of the nodes or the special value* `NONE`*. For example, the parent of some node could be s whereas the parent of s is* `NONE`*.*
- *If a vertex has more neighbors, we consider them* ***in lexicographic order*** *(that is, node a would come before node b and so on)!*

7. [5 points] Mark all the edges that belong to the minimum spanning tree of the following graph. *(Marking an edge means to draw along that edge.)*

Solution:



8. [4 points] Add the following keys (in the order as given) into the hash table below using the hash function

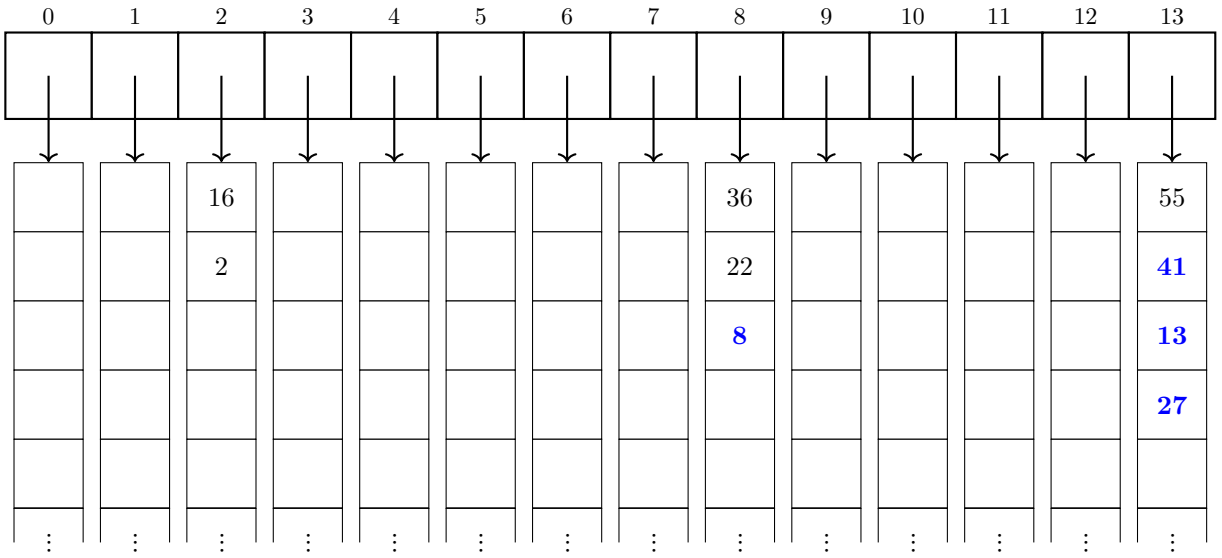
$$h(k) = k \mod 14$$

Keys: 41, 13, 27, 8

Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored (in the order as added; the first element of an array is at the top).

Note that for simplicity, we consider only keys and not key-value pairs. Also note that the hash table has been already filled with some other keys.

Solution:
Hash table:



9. [4 points] What is the most appropriate data structure for the following scenario, chosen from the list below?

   Unless stated otherwise in the scenario, the goal is to minimize memory usage and the worst-case running time of each operation.

   In your answer, briefly explain how your chosen data structure applies to the scenario. Then give its memory and time complexity, and explain why it performs better than *every* other option, considering the scenario's specific constraints and goals.

   You are developing the backend of a social media platform. New posts arrive with strictly increasing timestamps (which can be very large integers), and you need to insert each new post and check whether a post with a given timestamp exists. While each individual lookup must be as fast as possible, you only care about the total time spent inserting posts.

   1. Array indexed by keys
   2. AVL tree
   3. Find-union
   4. Hash table
   5. Heap/Priority queue
   6. Sorted Linked list
   7. Queue
   8. Stack
   9. Sorted array
   10. Unsorted array

   Most appropriate data structure: _____**Sorted array**_____

   Explanation:

   Solution: We use the timestamps as the keys in the sorted array. Thus, new keys are always larger than older keys.
   The memory usage is optimal at $O(n)$, where n is the number of elements. Insertions are efficient (amortized $O(1)$ time) under the assumption that new keys arrive in increasing (or decreasing) order, avoiding the need for shifting. The `Get` operation is efficient as well, requiring only $O(\log n)$ time via binary search.

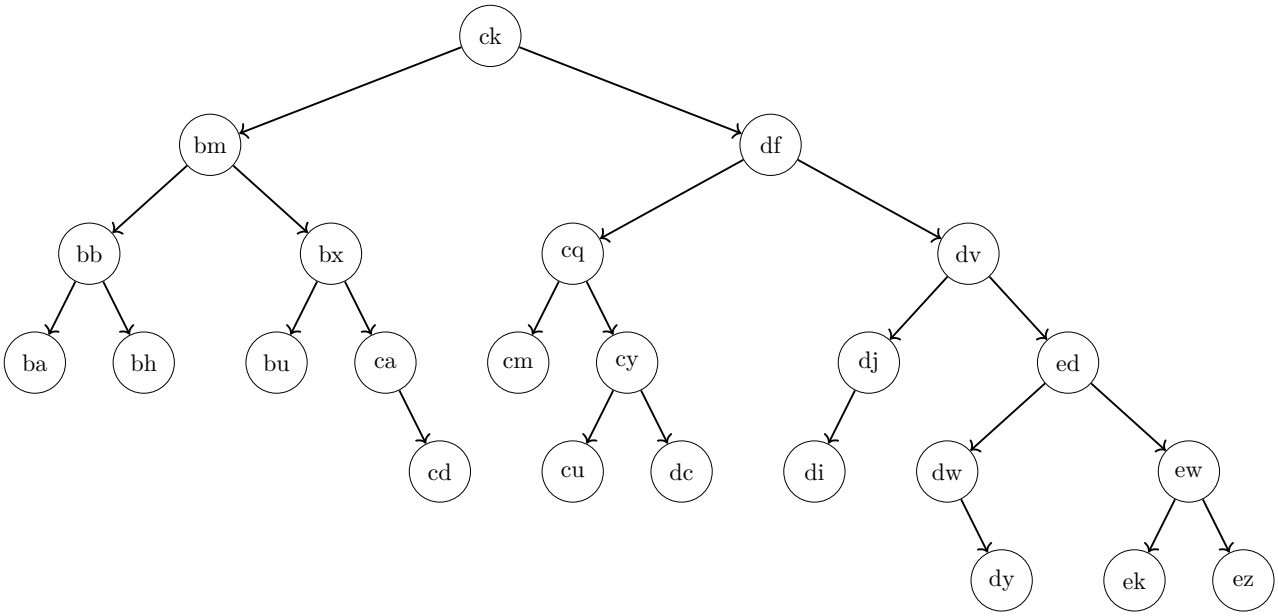
   All the other data structures are worse under this scenario: they either do not support the required operations (stack, queue, find-union), have slower worst-case time per `Get` operation (hash table, unsorted array, sorted linked list), or have a higher total worst-case insertion time (e.g., heap, AVL tree). Some take too much memory when the maximum key is very large (array indexed by keys).

10. [5 points] What is the height of the node *cq* in this binary search tree, what is the height of the root (as defined in the lecture and exercises)?

Recall that the height of a node v is the number of nodes *on a longest path starting at v within the subtree of v (not counting the dummy "NONE" nodes that have height* 0*).*

Height of *cq*: **3** (*Write only a single number.*)

Height of the root: **6** (*Write only a single number.*)



Is the AVL invariant violated? Your answer: **no** (*Write "yes" or "no".*)

Write the names of two nodes,

X=**cd** and Y=**ca**, such that the following holds:

- *If the tree violates the AVL invariant*, then the violation occurs at node X (caused by the heights of X's children), and adding a new child to node Y will restore the AVL invariant.
- *Otherwise, if the tree satisfies the AVL invariant*, then adding a new child to node X will cause a violation of the AVL invariant, and the violation will occur at node Y (caused by the heights of Y's children).
- *If there is more than one valid choice for X*, choose the one with the alphabetically **smallest** name. **After selecting** X, if there is more than one valid choice for Y, choose the one with the alphabetically **largest** name. (*Example: "ba" is alphabetically smaller than "ez".)*

In other words, X is either a node that violates the AVL invariant, or a node where adding a new child would cause such a violation. Similarly, Y is either a node where adding a new child restores the AVL invariant, or it is a node that violates the invariant after a new child is added to X. If there are multiple valid choices for X, choose the one with the alphabetically smallest name. If there are multiple valid choices for Y (given your choice of X), choose the one with the alphabetically largest name.

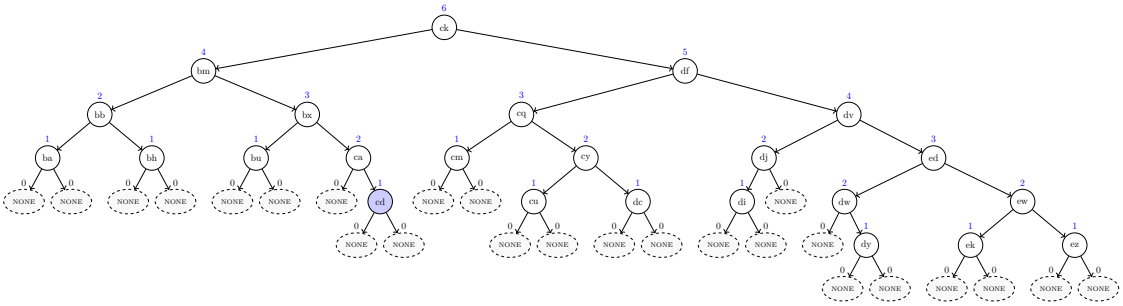
Note that there is exactly one correct solution that satisfies all the conditions above. It is also possible that $X = Y$.

See next page for an explanation of the solution!

Explanation:
The tree satisfies the AVL invariant. There are seven different candidates for X, where adding a node leads to a violation of the AVL invariant:

<u>cd</u>, **cu**, **dc**, **di**, **dy**, **ek** and **ez**.

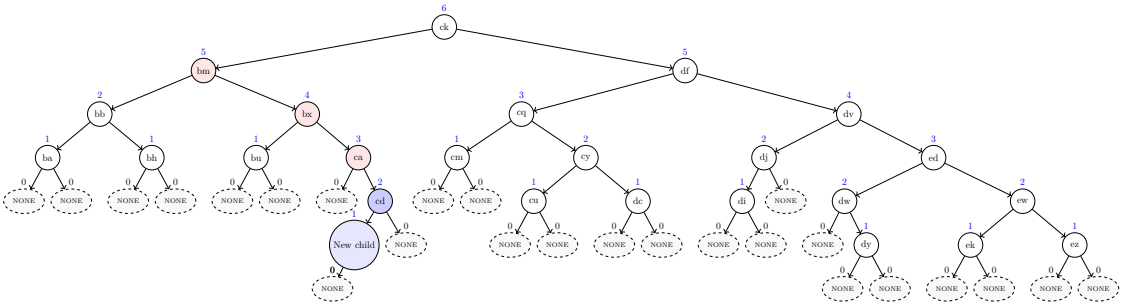
Recall that out of these different possibilties, the task asks you to choose the one where X is the alphabetically smallest node; hence, X = cd.
The first tree shows the heights and dummy nodes, as well as the node $\boldsymbol{X}$ **= cd** (in blue).



The second tree shows the new child (also in blue) that has been added to X as well as all the three violating parent nodes in red (whose children differ in height by more than 1):

<u>ca</u>, **bx** and **bm**.

Among these violating nodes, the task asks you to choose the alphabetically largest one; hence $\boldsymbol{Y}$ **= ca**.



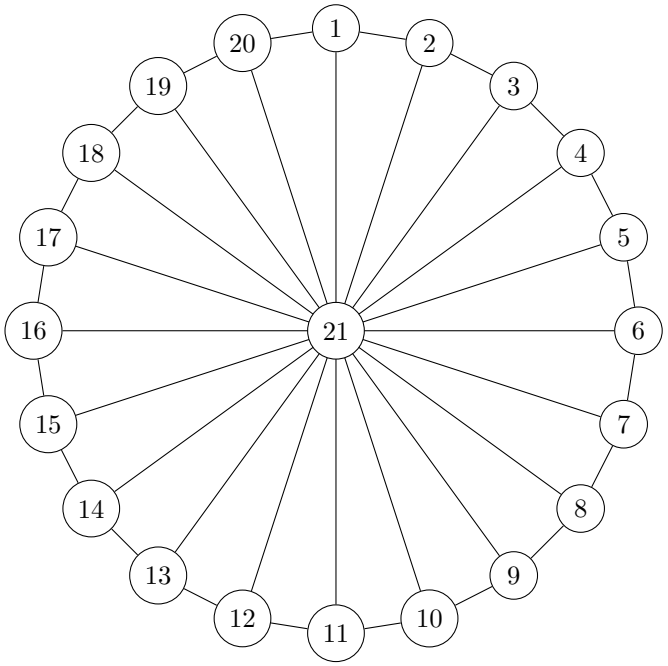
For completness, for each (wrong) candidate for X a list of violating parent nodes. The alphabetically optimal violating parent node is underlined.

- Candidate **cu** →**<u>cq</u>**
- Candidate **dc** →**<u>cq</u>**
- Candidate **di** →**<u>dj</u>**
- Candidate **dy** →**<u>dw</u>**, **dv**, **df** and **ck**
- Candidate **ek** →**<u>dv</u>**, **df** and **ck**
- Candidate **ez** →**<u>dv</u>**, **df** and **ck**

11. [5 points] Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to n=21 and each edge has the same weight 1. Observe that the vertices 1 to 20 lie on a big cycle and each of them is connected to the center vertex 21 via an edge.

    Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 7) already after 14 iterations.

```
1  δ[1..21][1..21] = new 2D array filled with +∞
2  for i = 1 to 21 do
3      δ[i][i] = 0
4  foreach edge (v, w, c) do
       // v and w are the endpoints and c is the weight of the edge
5      δ[v][w] = c
6      δ[w][v] = c
7  for k = 1 to 14 do
8      foreach pair (v, w) do
9          δ[v][w] = min(δ[v][w],   δ[v][k] + δ[k][w])
10 return δ
```



Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 14 times**!

Recall that the weight of every edge is equal to 1.

*If the distance is **infinite**, write "INF". Note: There are exactly two fields where the correct answer is "INF". If you write "INF" in more than two fields, you get 0 points for those fields.*

$\delta[21][19] =$ **1** $\quad\quad \delta[4][13] =$ **9**

$\delta[20][15] =$ **15** $\quad\quad \delta[16][9] =$ **INF**

$\delta[14][19] =$ **INF**

12. [4 points] Consider the following algorithm.

```
Input: integer t
m = 0
y = 3
while y ≤ t do
    m = m + 11
    j = −9
    y = y + j
    m = m + j
    y = y + 17
```

Someone wrote the following invariant for the loop:

$$G \cdot m + A = P \cdot y$$

with the three constants

- $G = 36$
- $A = 27$
- $P = 10$

The invariant should hold each time the algorithm evaluates the loop condition. Unfortunately, the invariant has a mistake: one of the three constants, G, A, or P has a wrong value.

Fix the invariant by changing the value of one of the constants!

Which constant do you choose? _____**P**_____ *(Write the name of one of the three constants.)*

The new value for the chosen constant: _____**9**_____

Note: You are allowed to change only one constant.

13. 4 points (bonus)

Consider the following decision problem A:

You run a newspaper delivery service and need to deliver papers to n homes. You know the driving time between any pair of homes. Starting and ending at your depot, can you complete a route that visits each home within a given time limit T?

Consider the following decision problem B:

You're catering an event and have n ingredients, each with a weight in grams and a nutrition score. Can you select a subset whose total weight is at most W and whose total nutrition is at least N?

Mark all the correct sentences and give a short explanation.

- √ **A can be reduced to B in polynomial time.**
- √ **B can be reduced to A in polynomial time.**
- ◯ A is in P.
- ◯ B is in P.
- ◯ It is unknown whether A can be reduced to B in polynomial time.
- ◯ It is unknown whether B can be reduced to A in polynomial time.
- √ **It is unknown whether A is in P.**
- √ **It is unknown whether B is in P.**

Solution: Problem A is equivalent to the NP-complete problem Traveling Salesman (by interpreting homes as vertices, driving times as edge weights, and the time limit T as the cost threshold), and problem B is equivalent to the NP-complete problem Knapsack (by treating each ingredient as an item with weight and nutrition value, W as capacity, and N as the target value).

Both problems can be reduced to each other in polynomial time because

1. both problems are in NP (as they are NP-complete),
2. both problems are NP-hard (as they are NP-complete), and
3. all problems in NP (thus, including A and B) can be reduced in polynomial time to an NP-hard problem (for example, to A and B).

Since it is unknown whether NP-complete problems can be solved in polynomial time, we do not know whether each of the two problems is in P.

Algorithms for Data Science
Exam 2025
Group **G**

June 2025

Your Name: _____ **Your Student ID:** _____

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	5	4	4	6	5	9	5	4	4	5	5	4	0	60
Score:														

The last task gives 4 bonus points. Thus, you can obtain 60+4 points in total (regular+bonus points).

Check if your name and student ID are printed on top of every single page!

Write your name and your student ID on the first page!

Check whether you received all 14 pages (containing 13 tasks in total).

The regular time to solve the exam is 90 minutes.

1. [5 points] Consider the following incomplete Python function. A sequence of unit blocks is dropped one by one onto integer coordinates along a line; when a block lands on an occupied coordinate, it stacks on top, raising that column's height by 1.
Coordinates may be arbitrarily large and all start at height 0. The function *blocks*(*pos*,*h*) takes the list *pos* of drop positions and the target height *h*, and returns the x-coordinate where a column's height first reaches *h*.

   Hint: the task is related to the third programming task.

```
1  def blocks(pos, h):
2      n = len(pos)
3      # ??? [Line 3]
4      for i in range(n):
5          x = pos[i]
6          # ??? [Line 6] (Starts with if..)
7              # ??? [Line 7]
8          else:
9              # ??? [Line 9]
10         # ??? [Line 10]
11             return x
```

   Complete the code by choosing a correct code snippet for each empty line (that starts with `# ???`). Do it by filling the blanks with the correct labels from the code snippets below:

   - Line 3: _____**F**_____
   - Line 6: _____**N**_____
   - Line 7: _____**D**_____
   - Line 9: _____**G**_____
   - Line 10: _____**O**_____

   Code snippets to choose from:

   - (A) `heights = [0] * n`
   - (B) `heights[x] += i`
   - (C) `if x < len(heights):`
   - (D) `heights[x] = 1`
   - (E) `if heights[h] == i:`
   - (F) `heights = {}`
   - (G) `heights[x] += 1`
   - (H) `heights[i] = x`
   - (I) `heights = [0] * max(pos)`
   - (J) `heights[i] += x`
   - (K) `if heights[x] < h:`
   - (L) `heights[x] = h`
   - (M) `if heights[x] == 0:`
   - (N) `if x not in heights:`
   - (O) `if heights[x] == h:`

   Note: The indendation of your code is the same as of `# ???`*. In other words, your code for an empty line will start at the position of* `#`*.*

2. [4 points] Complete the following algorithm `AlgoA(`$a[1..n]$`)` such that its running time $T(n)$ satisfies the recurrence

$$T(n) = 18T(\lceil n/30 \rceil) + \Theta(n^5) \ .$$

You don't need to understand what the algorithm computes!

The input of `AlgoA(`$a[1..n]$`)` *is an array* a *of* n *integers.*

The input of the auxiliary function `Copy_Sub_Array(`$a[1..n]$`,`i`,`$m`)` *is an array* $a[1..n]$*, a position* i *and a length* m*. The function returns a copy of the subarray* $a[i+1..i+m]$*.*

```
AlgoA(a[1..n])
    if n < 31 then
        return 20
    else
        d = __10__
        m = ⌈n/(3 · d)⌉
        sum = 0
        for i = 1 to 6 do
            b = Copy_Sub_Array(a,i,m)
            sum = sum + AlgoA(b)
        for i = 1 to __12__ do
            b = Copy_Sub_Array(a,i,m)
            sum = sum + AlgoA(b)
        w = 1
        for j = 1 to __5__ do
            w = w · n // Time: O(1)
        for k = 1 to w do
            sum = sum + a[31]
        return sum
```

```
Copy_Sub_Array(a[1..n],i,m)
    b[1..m] = new empty array
    for j = 1 to m do
        b[j] = a[i + j]
    return b
```

Fill in the three blanks in the algorithm above, each time writing **a single appropriate integer number**!

3. [4 points] Solve the following recurrence using the master theorem. Determine (for yourself) the values for a, b, and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = 7 \cdot 2 \cdot T(n/14) + \Theta(n)$$

Your answers:

$c =$ **1**

Case: **B**

Resulting running time $T(n) = \Theta(n \log n)$

4. Simplify the following running times as far as possible:

(a) [2 points] $\Theta(\ 5^{(n-n)} \cdot 8 + 6 + 5 \cdot n^{(4-4)}\ ) = \Theta($ 1 $)$

(b) [2 points] $\Theta(\ (45^{\log_{45} n})^{45} + (5^{n/45})^5\ ) = \Theta($ $5^{n/9}$ $)$

(c) [2 points] $\Theta(\ 8n^6 + 26n^5 \log n + 5n^6\ ) = \Theta($ n^6 $)$

5. [5 points] Consider the following array $a[1..21]$ containing twenty-one elements:

$$a = \langle 105, 100, 85, 35, 95, 80, 70, 20, 25, 90, 50, 60, 75, 65, 45, 10, 15, 30, 25, 55, 40 \rangle$$

1. Draw the *complete* array as a complete binary tree (as defined for heaps),
2. Mark the edge whose endpoints (parent and child) violate the heap invariant,
3. Choose an appropriate index position j and **increase** the value of $a[j]$ by the **maximum amount possible** such that a satisfies the heap invariant.

What index j do you choose?
(Write a single number between 1 *and* 21*; there is only one correct answer.)*

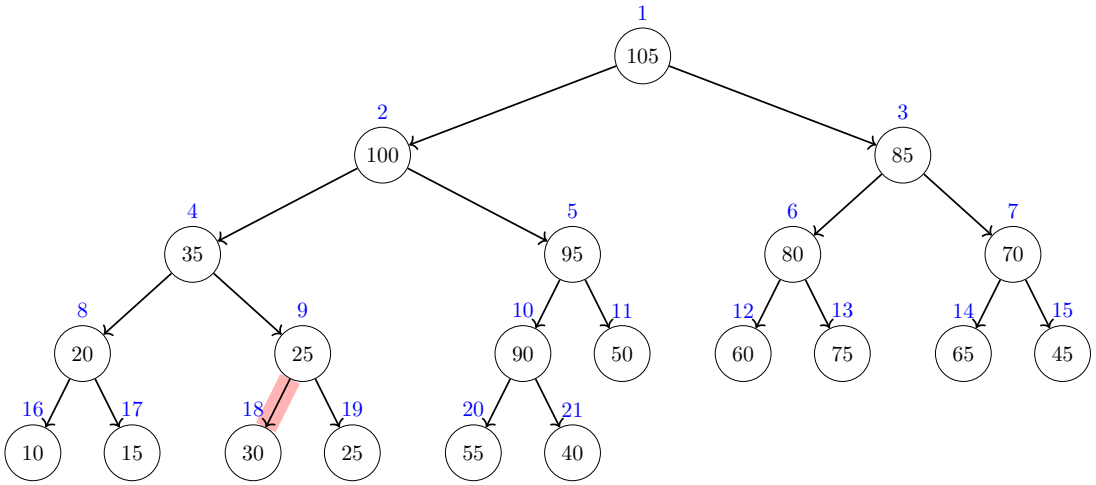
Your answer: $j =$ **9** ***(the violating edge's other endpoint is at* 18*.)***

What is the new value for $a[j]$?
(Write a single number, as large as possible and larger than the old value of $a[j]$*.)*

Your answer: $a[j] =$ **35** ***(the largest value from* $[30, 35]$*)***

Your tree hould contain ***all* the elements of the array**, from $a[1] = 105$ up to $a[21] = 40$:

Solution:



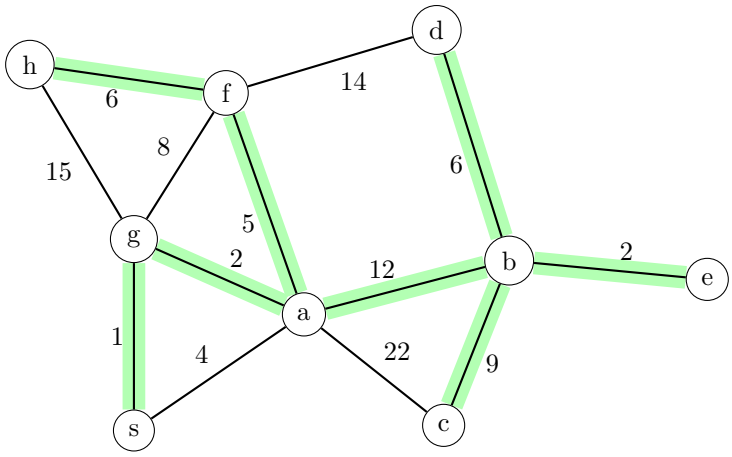
Explanation (not required): The heap invariant is violated at the red edge as the parent (index **9***) has a smaller value than its child (index* **18***). To fix the heap invariant, it suffices to increase* $a[9]$ *to any value between* 30 *and* 35*. The largest possible value (as asked in the task) is thus* 35*.*

6. [9 points] Dijkstra's algorithm (see below) is run on the following graph where the start node is s.

   What are the values of the *dist* and *parent* tables when Dijkstra's algorithm has finished? Fill in the missing entries in the table below!

   Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```
dist[0..n − 1] filled with INF
parent[0..n − 1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time then
        for every edge (v, w) do
            if dist[w] > dist[v] + c_{v,w} then
                dist[w] = dist[v] + c_{v,w}
                parent[w] = v
                q.add(w, -dist[w])
```



Solution:

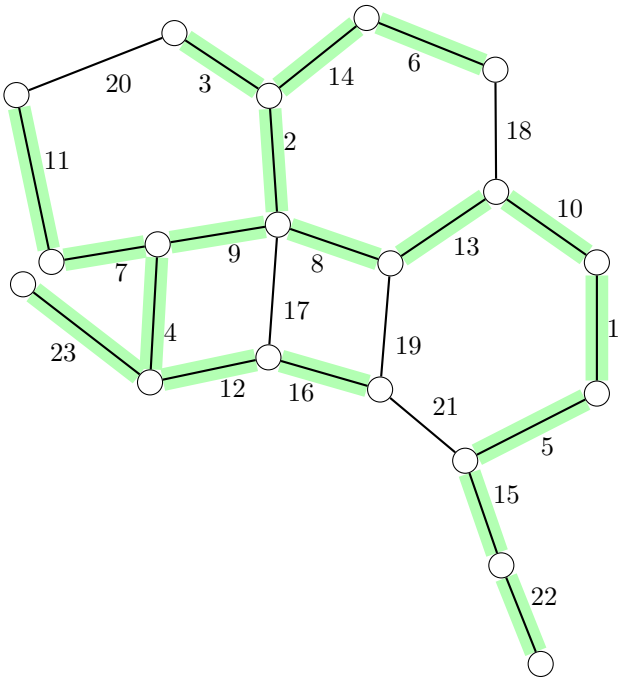
	a	b	c	d	e	f	g	h	s
distance	3	15	24	21	17	8	1	14	0
parent	g	a	b	b	b	a	s	f	NONE

Note:

- *The entries of the parent table consist of the names of the nodes or the special value* `NONE`*. For example, the parent of some node could be* s *whereas the parent of* s *is* `NONE`*.*
- *If a vertex has more neighbors, we consider them* ***in lexicographic order*** *(that is, node* a *would come before node* b *and so on)!*

7. [5 points] Mark all the edges that belong to the minimum spanning tree of the following graph. *(Marking an edge means to draw along that edge.)*

Solution:



8. [4 points] Add the following keys (in the order as given) into the hash table below using the hash function

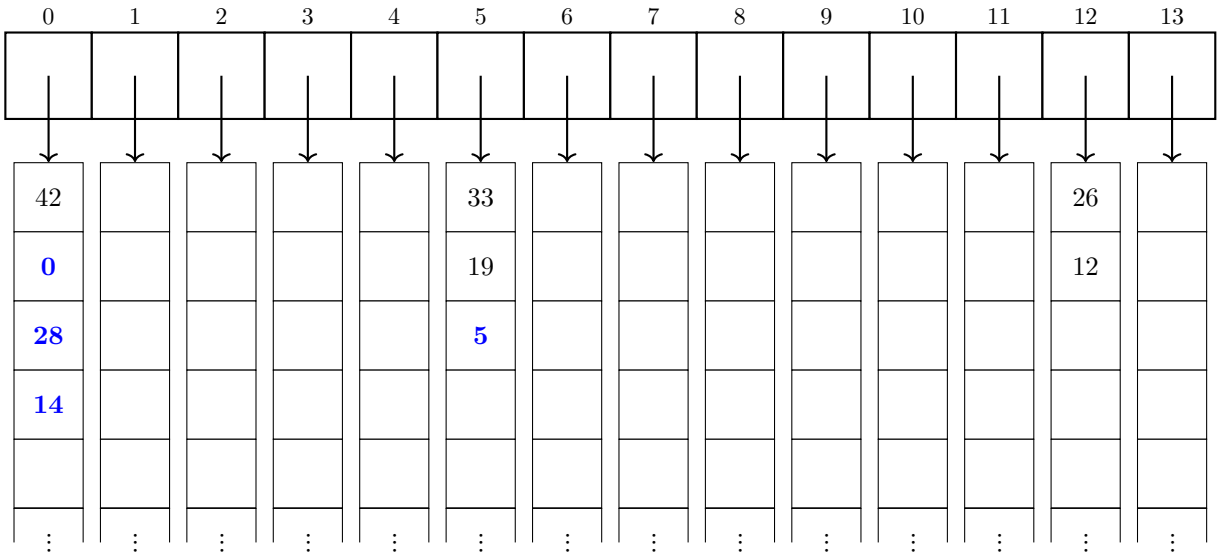
$$h(k) = k \mod 14$$

Keys: 5, 0, 28, 14

Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored (in the order as added; the first element of an array is at the top).

Note that for simplicity, we consider only keys and not key-value pairs. Also note that the hash table has been already filled with some other keys.

Solution:
Hash table:



9. [4 points] What is the most appropriate data structure for the following scenario, chosen from the list below?

   Unless stated otherwise in the scenario, the goal is to minimize memory usage and the worst-case running time of each operation.

   In your answer, briefly explain how your chosen data structure applies to the scenario. Then give its memory and time complexity, and explain why it performs better than *every* other option, considering the scenario's specific constraints and goals.

   You are implementing the inventory tracking subsystem of a warehouse management system. Shipments arrive and expire dynamically, each identified by a large integer shipment ID. You must support adding a new shipment, removing an expired shipment, and querying the quantity for a given shipment ID, all with guaranteed worst-case performance.

   1. Array indexed by keys
   2. AVL tree
   3. Find-union
   4. Hash table
   5. Heap/Priority queue
   6. Sorted Linked list
   7. Queue
   8. Stack
   9. Sorted array
   10. Unsorted array

   Most appropriate data structure: _____**AVL tree**_____

   Explanation:

   Solution: We maintain an AVL tree keyed by shipment ID.
   The worst-case running times of all the standard AVL tree operations are $O(\log n)$, which is asymptotically the best we can get for a dictionary (without further assumptions on the keys). The same holds for the memory usage which is proportional to the number of elements.

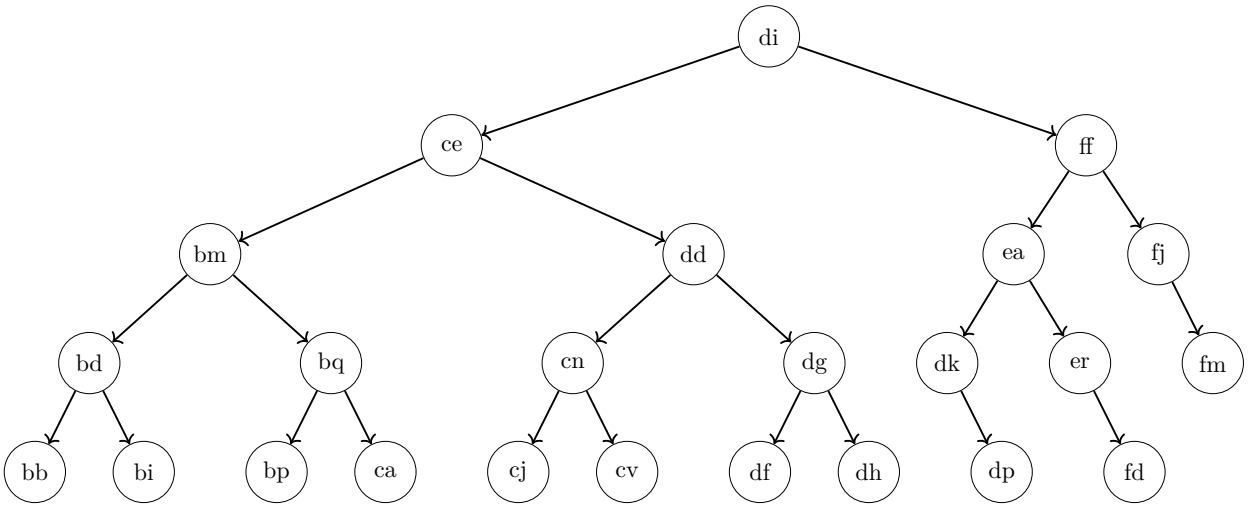
   All the other data structures are worse under this scenario as they don't provide feasible operations (stack, queue, heap, find-union), take more time in the worst case (hash table, sorted/unsorted array, sorted linked list), or take too much memory if the maximum key is very large (array indexed by keys).

10. [5 points] What is the height of the node *ce* in this binary search tree, what is the height of the root (as defined in the lecture and exercises)?

Recall that the height of a node v is the number of nodes *on a longest path starting at v within the subtree of v (not counting the dummy "NONE" nodes that have height 0).*

Height of *ce*: **4** (*Write only a single number.*)

Height of the root: **5** (*Write only a single number.*)



Does the tree satisfy the AVL invariant? Your answer: **yes** (*Write "yes" or "no".*)

Write the names of two nodes,

X=**dp** and Y=**ff**, such that the following holds:

- *If the tree violates the AVL invariant*, then the violation occurs at node X (caused by the heights of X's children), and adding a new child to node Y will restore the AVL invariant.
- *Otherwise, if the tree satisfies the AVL invariant*, then adding a new child to node X will cause a violation of the AVL invariant, and the violation will occur at node Y (caused by the heights of Y's children).
- *If there is more than one valid choice for X*, choose the one with the alphabetically **smallest** name. **After selecting X**, if there is more than one valid choice for Y, choose the one with the alphabetically **largest** name. (*Example: "bb" is alphabetically smaller than "fm".)*

In other words, X is either a node that violates the AVL invariant, or a node where adding a new child would cause such a violation. Similarly, Y is either a node where adding a new child restores the AVL invariant, or it is a node that violates the invariant after a new child is added to X. If there are multiple valid choices for X, choose the one with the alphabetically smallest name. If there are multiple valid choices for Y (given your choice of X), choose the one with the alphabetically largest name.

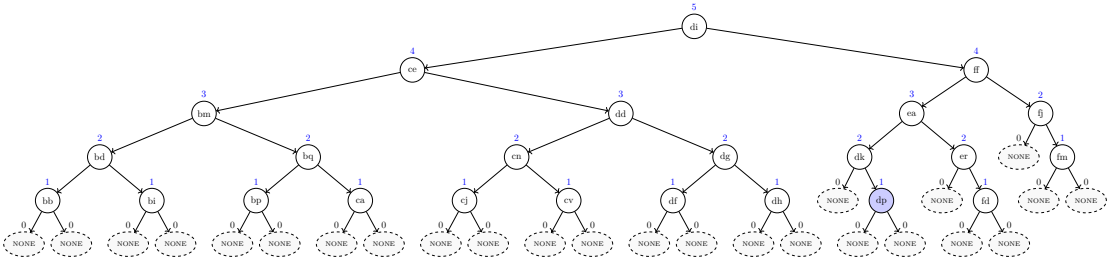
Note that there is exactly one correct solution that satisfies all the conditions above. It is also possible that $X = Y$.

See next page for an explanation of the solution!

Explanation:
The tree satisfies the AVL invariant. There are three different candidates for X, where adding a node leads to a violation of the AVL invariant:

<u>dp</u>, **fd** and **fm**.

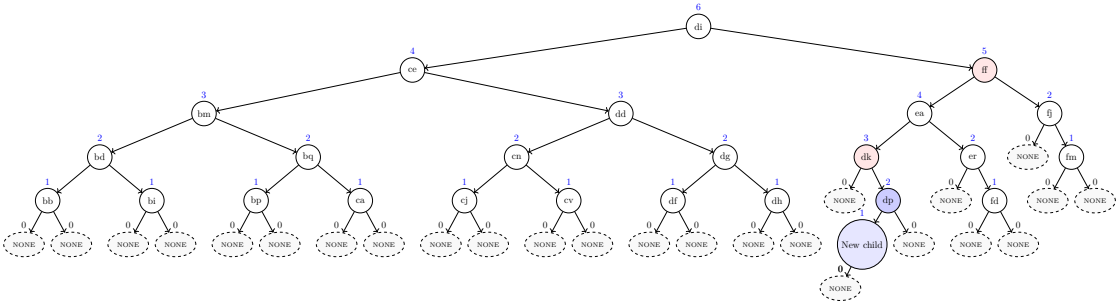
Recall that out of these different possibilties, the task asks you to choose the one where X is the alphabetically smallest node; hence, $X = \text{dp}$.
The first tree shows the heights and dummy nodes, as well as the node $\boldsymbol{X = \textbf{dp}}$ (in blue).



The second tree shows the new child (also in blue) that has been added to X as well as all the two violating parent nodes in red (whose children differ in height by more than 1):

dk and **<u>ff</u>**.

Among these violating nodes, the task asks you to choose the alphabetically largest one; hence $\boldsymbol{Y = \textbf{ff}}$.



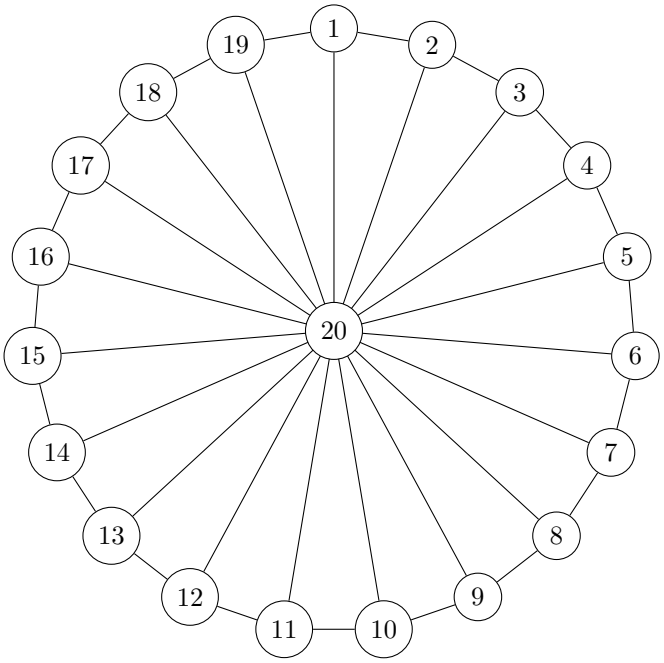
For completness, for each (wrong) candidate for X a list of violating parent nodes. The alphabetically optimal violating parent node is underlined.

- Candidate **fd** →**er** and **<u>ff</u>**
- Candidate **fm** →**<u>fj</u>**

11. [5 points] Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to n=20 and each edge has the same weight 1. Observe that the vertices 1 to 19 lie on a big cycle and each of them is connected to the center vertex 20 via an edge.

    Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 7) already after 15 iterations.

```
1  δ[1..20][1..20] = new 2D array filled with +∞
2  for i = 1 to 20 do
3      δ[i][i] = 0
4  foreach edge (v, w, c) do
       // v and w are the endpoints and c is the weight of the edge
5      δ[v][w] = c
6      δ[w][v] = c
7  for k = 1 to 15 do
8      foreach pair (v, w) do
9          δ[v][w] = min(δ[v][w],   δ[v][k] + δ[k][w])
10 return δ
```



Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 15 times**!

Recall that the weight of every edge is equal to 1.

*If the distance is **infinite**, write "INF". Note: There are exactly two fields where the correct answer is "INF". If you write "INF" in more than two fields, you get 0 points for those fields.*

$\delta[20][17] =$ **1** $\quad\quad \delta[3][17] =$ **INF**

$\delta[19][16] =$ **16** $\quad\quad \delta[11][5] =$ **6**

$\delta[18][15] =$ **INF**

12. [4 points] Consider the following algorithm.

Input: integer g
$f = 6$
$z = 0$
while $f \leq g$ **do**
  $z = z - 12$
  $v = 8$
  $f = f - v$
  $z = z + v$
  $f = f + 16$

Someone wrote the following invariant for the loop:

$$U = J \cdot f + K \cdot z$$

with the three constants

- $U = 42$
- $J = 6$
- $K = 14$

The invariant should hold each time the algorithm evaluates the loop condition. Unfortunately, the invariant has a mistake: one of the three constants, U, J, or K has a wrong value.

Fix the invariant by changing the value of one of the constants!

Which constant do you choose? _____**J**_____ *(Write the name of one of the three constants.)*

The new value for the chosen constant: _____**7**_____

Note: You are allowed to change only one constant.

13. 4 points (bonus)

Consider the following decision problem A:

You're catering an event and have n ingredients, each of identical weight w grams and a nutrition score. Can you select a subset whose total weight is at most W and whose total nutrition is at least N?

Consider the following decision problem B:

You're organizing a tour of n meeting rooms connected by corridors for a conference. You need a route starting and ending at the lobby that visits every room exactly once. Is such a route possible?

Mark all the correct sentences and give a short explanation.

- √ **A can be reduced to B in polynomial time.**
- ◯ B can be reduced to A in polynomial time.
- √ **A is in P.**
- ◯ B is in P.
- ◯ It is unknown whether A can be reduced to B in polynomial time.
- √ **It is unknown whether B can be reduced to A in polynomial time.**
- ◯ It is unknown whether A is in P.
- √ **It is unknown whether B is in P.**

Solution: Problem B is equivalent to the NP-complete problem Hamiltonian cycle (by representing rooms as vertices, corridors as edges, and seeking a Hamiltonian cycle), whereas problem A can be solved in polynomial time: since each ingredient weighs the same w, sort them by nutrition and pick the top $\lfloor W/w \rfloor$ items in polynomial time and compare their sum to N.

Consequently, as problem A can be solved in polynomial time, it can reduced in polynomial time to any other problem, thus also to B.

Since it is unknown whether NP-complete problems can be solved in polynomial time, we do not know whether B is in P, and, consequently, whether there exist a polynomial time reduction from B to any polynomial-time problem (e.g., to A).

Algorithms for Data Science
Exam 2025
Group **H**

June 2025

Your Name: _____ **Your Student ID:** _____

Question:	1	2	3	4	5	6	7	8	9	10	11	12	13	Total
Points:	5	4	4	6	5	9	5	4	4	5	5	4	0	60
Score:														

The last task gives 4 bonus points. Thus, you can obtain 60+4 points in total (regular+bonus points).

Check if your name and student ID are printed on top of every single page!

Write your name and your student ID on the first page!

Check whether you received all 14 pages (containing 13 tasks in total).

The regular time to solve the exam is 90 minutes.

1. [5 points] Consider the following incomplete Python function. A sequence of unit blocks is dropped one by one onto integer coordinates along a line; when a block lands on an occupied coordinate, it stacks on top, raising that column's height by 1.
Coordinates may be arbitrarily large and all start at height 0. The function *blocks*(*pos*,*h*) takes the list *pos* of drop positions and the target height *h*, and returns the x-coordinate where a column's height first reaches *h*.

   Hint: the task is related to the third programming task.

```
1  def blocks(pos, h):
2      n = len(pos)
3      # ??? [Line 3]
4      for i in range(n):
5          x = pos[i]
6          # ??? [Line 6] (Starts with if..)
7              # ??? [Line 7]
8          else:
9              # ??? [Line 9]
10         # ??? [Line 10]
11             return x
```

   Complete the code by choosing a correct code snippet for each empty line (that starts with `# ???`). Do it by filling the blanks with the correct labels from the code snippets below:

   - Line 3: _____**E**_____
   - Line 6: _____**F**_____
   - Line 7: _____**N**_____
   - Line 9: _____**D**_____
   - Line 10: _____**G**_____

   Code snippets to choose from:

   - (A) `heights = [0] * max(pos)`
   - (B) `heights[i] += x`
   - (C) `if heights[x] == 0:`
   - (D) `heights[x] += 1`
   - (E) `heights = {}`
   - (F) `if x not in heights:`
   - (G) `if heights[x] == h:`
   - (H) `heights[x] = h`
   - (I) `if x < len(heights):`
   - (J) `if heights[x] < h:`
   - (K) `if heights[h] == i:`
   - (L) `heights[x] += i`
   - (M) `heights[i] = x`
   - (N) `heights[x] = 1`
   - (O) `heights = [0] * n`

   Note: The indendation of your code is the same as of `# ???`*. In other words, your code for an empty line will start at the position of* `#`*.*

2. [4 points] Complete the following algorithm `AlgoA(`$a[1..n]$`)` such that its running time $T(n)$ satisfies the recurrence

$$T(n) = 14T(\lceil n/26 \rceil) + \Theta(n^{12}) \ .$$

You don't need to understand what the algorithm computes!

The input of `AlgoA(`$a[1..n]$`)` *is an array* a *of* n *integers.*

The input of the auxiliary function `Copy_Sub_Array(`$a[1..n]$`,`i`,`m`)` *is an array* $a[1..n]$*, a position* i *and a length* m*. The function returns a copy of the subarray* $a[i+1..i+m]$*.*

```
AlgoA(a[1..n])
    if n < 27 then
        return 13
    else
        d = 13
        m = ⌈n/(2 · d)⌉
        sum = 0
        for i = 1 to 6 do
            b = Copy_Sub_Array(a,i,m)
            sum = sum + AlgoA(b)

        for i = 1 to 8 do
            b = Copy_Sub_Array(a,i,m)
            sum = sum + AlgoA(b)
        w = 1

        for j = 1 to 12 do
            w = w · n // Time: O(1)
        for k = 1 to w do
            sum = sum + a[27]
        return sum
```

```
Copy_Sub_Array(a[1..n],i,m)
    b[1..m] = new empty array
    for j = 1 to m do
        b[j] = a[i + j]
    return b
```

Fill in the three blanks in the algorithm above, each time writing **a single appropriate integer number**!

3. [4 points] Solve the following recurrence using the master theorem. Determine (for yourself) the values for a, b, and $f(n)$ and write down the value for c (as defined in the master theorem). Then, determine which case holds and write the resulting running time. (For case C, you can assume that $f(n)$ satisfies the regularity condition.)

$$T(n) = 64 \cdot T(n/2) + \Theta(n^8)$$

Your answers:

$c =$ **6**

Case: **C**

Resulting running time $T(n) = \Theta(n^8)$

4. Simplify the following running times as far as possible:

(a) [2 points] $\Theta(\ 4 \cdot n^{13/16} + 4 \cdot n^{6/8} \cdot n^{2/8}\ ) = \Theta(\ n\ )$

(b) [2 points] $\Theta(\ (15^{\log_{15} n})^{15} + (5^{n/15})^5\ ) = \Theta(\ 5^{n/3}\ )$

(c) [2 points] $\Theta(\ 7 \log_2 n + 5 \cdot n^5 + n^4 \log_2 n\ ) = \Theta(\ n^5\ )$

5. [5 points] Consider the following array $a[1..21]$ containing twenty-one elements:

$$a = \langle 105, 100, 70, 50, 95, 65, 60, 40, 30, 90, 75, 55, 65, 40, 15, 45, 35, 20, 25, 80, 85 \rangle$$

1. Draw the *complete* array as a complete binary tree (as defined for heaps),
2. Mark the edge whose endpoints (parent and child) violate the heap invariant,
3. Choose an appropriate index position j and **increase** the value of $a[j]$ by the **maximum amount possible** such that a satisfies the heap invariant.

What index j do you choose?
(Write a single number between 1 *and* 21*; there is only one correct answer.)*

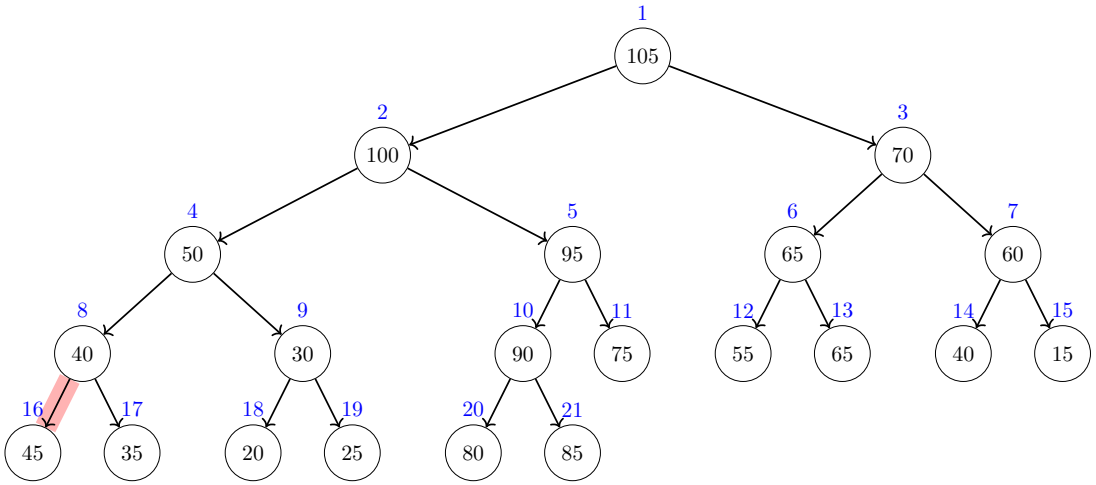
Your answer: $j =$ **8** ***(the violating edge's other endpoint is at* 16*.)***

What is the new value for $a[j]$?
(Write a single number, as large as possible and larger than the old value of $a[j]$*.)*

Your answer: $a[j] =$ **50** ***(the largest value from* $[45, 50]$*)***

Your tree hould contain ***all* the elements of the array**, from $a[1] = 105$ up to $a[21] = 85$:

Solution:



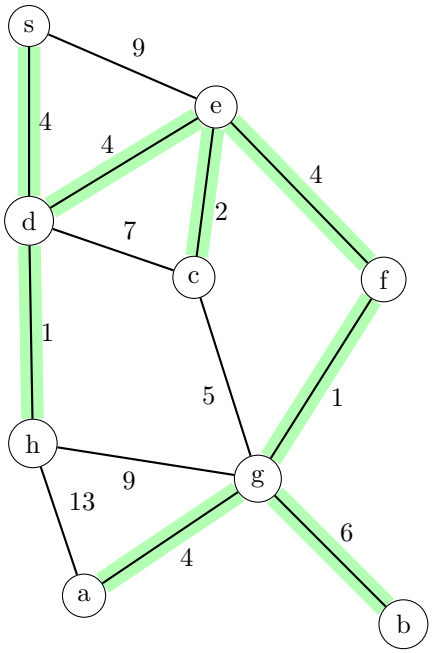
Explanation (not required): The heap invariant is violated at the red edge as the parent (index **8***) has a smaller value than its child (index* **16***). To fix the heap invariant, it suffices to increase* $a[8]$ *to any value between* 45 *and* 50*. The largest possible value (as asked in the task) is thus* 50*.*

6. [9 points] Dijkstra's algorithm (see below) is run on the following graph where the start node is s.

   What are the values of the *dist* and *parent* tables when Dijkstra's algorithm has finished? Fill in the missing entries in the table below!

   Hint: While solving, write the current distances next to the vertices and mark those vertices that have already been removed from the queue.

```
dist[0..n − 1] filled with INF
parent[0..n − 1] filled with NONE
dist[s] = 0
q=new priority queue
q.add(s,0)
while q not empty do
    v = q.extractMax()
    if v removed for the first time then
        for every edge (v, w) do
            if dist[w] > dist[v] + c_{v,w} then
                dist[w] = dist[v] + c_{v,w}
                parent[w] = v
                q.add(w, -dist[w])
```



Solution:

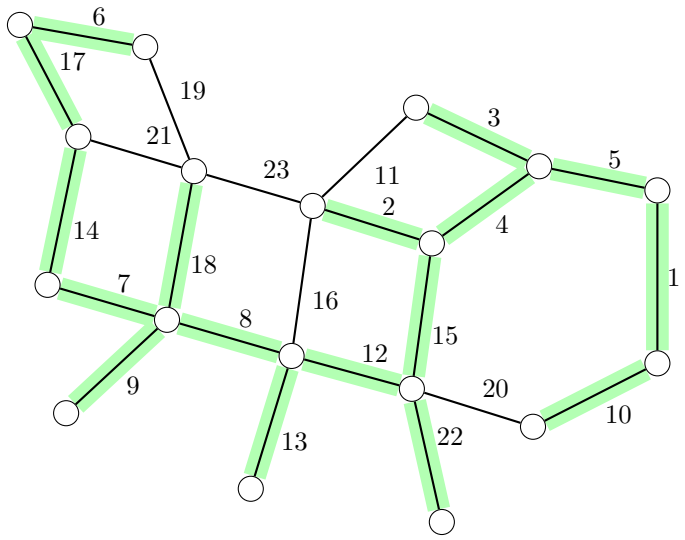
	a	b	c	d	e	f	g	h	s
distance	17	19	10	4	8	12	13	5	0
parent	g	g	e	s	d	e	f	d	NONE

Note:

- *The entries of the parent table consist of the names of the nodes or the special value* `NONE`*. For example, the parent of some node could be* s *whereas the parent of* s *is* `NONE`*.*
- *If a vertex has more neighbors, we consider them* ***in lexicographic order*** *(that is, node* a *would come before node* b *and so on)!*

7. [5 points] Mark all the edges that belong to the minimum spanning tree of the following graph. *(Marking an edge means to draw along that edge.)*

Solution:



8. [4 points] Add the following keys (in the order as given) into the hash table below using the hash function

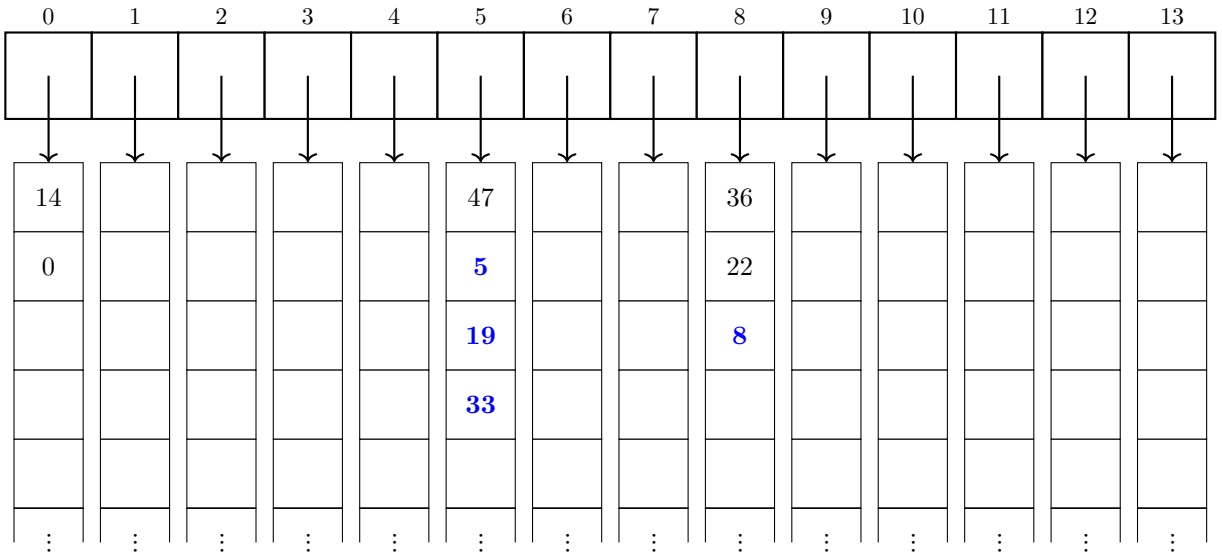
$$h(k) = k \mod 14$$

Keys: 5, 8, 19, 33

Conflicts are resolved via *separate chaining* as in the lecture, that is, each hash table cell stores a pointer to an unordered array where the elements are actually stored (in the order as added; the first element of an array is at the top).

Note that for simplicity, we consider only keys and not key-value pairs. Also note that the hash table has been already filled with some other keys.

Solution:
Hash table:



9. [4 points] What is the most appropriate data structure for the following scenario, chosen from the list below?

   Unless stated otherwise in the scenario, the goal is to minimize memory usage and the worst-case running time of each operation.

   In your answer, briefly explain how your chosen data structure applies to the scenario. Then give its memory and time complexity, and explain why it performs better than *every* other option, considering the scenario's specific constraints and goals.

   You are building the warehouse's inventory log. Each new transaction has a unique integer ID and must be added when it occurs. You never need to look up a specific ID, but you sometimes need to list all transaction IDs in the order they happened.

   1. Array indexed by keys
   2. AVL tree
   3. Find-union
   4. Hash table
   5. Heap/Priority queue
   6. Sorted Linked list
   7. Queue
   8. Stack
   9. Sorted array
   10. Unsorted array

   Most appropriate data structure: **Unsorted array, with modifications: stack, queue**

   Explanation:

   Solution: We use a simple array: each new ID is appended at the end, and reporting all IDs just reads the array from start to finish.
   The memory usage is optimal at $\Theta(n)$, where n is the number of elements. Given the guarantee that `Add` is called only on new keys, insertions take amortized constant time by appending at the end, which makes the total time of `Add` to be $\Theta(n)$. Furthermore, we can output all the elements in their insertion order (in total $\Theta(n)$ time) by going through the unordered array from the beginning to the end.

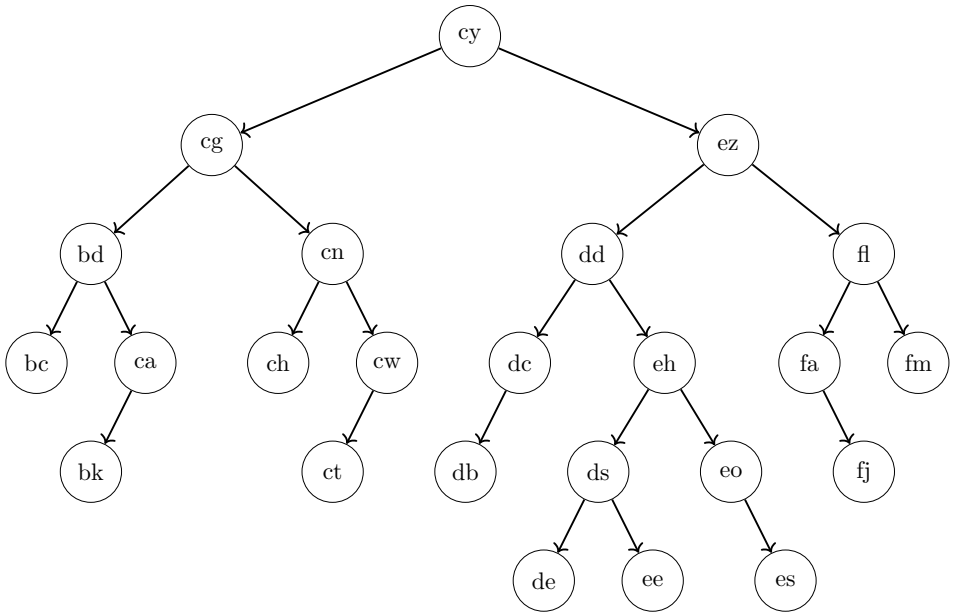
   All the other data structures are worse under this scenario as they cannot return all the elements in their insertion order. The only exception are stack and queue, but they don't provide such a functionality out of the box, but it could be achieved by using another auxiliary queue (or two auxiliary stacks) to scan through all the elements without loosing those elements.

10. [5 points] What is the height of the node *cg* in this binary search tree, what is the height of the root (as defined in the lecture and exercises)?

Recall that the height of a node v is the number of nodes *on a longest path starting at v within the subtree of v (not counting the dummy "NONE" nodes that have height* 0*).*

Height of *cg*: **4** (*Write only a single number.*)

Height of the root: **6** (*Write only a single number.*)



Is the AVL invariant violated? Your answer: **no** (*Write "yes" or "no".*)

Write the names of two nodes,

X=**bk** and Y=**bd**, such that the following holds:

- *If the tree violates the AVL invariant*, then the violation occurs at node X (caused by the heights of X's children), and adding a new child to node Y will restore the AVL invariant.
- *Otherwise, if the tree satisfies the AVL invariant*, then adding a new child to node X will cause a violation of the AVL invariant, and the violation will occur at node Y (caused by the heights of Y's children).
- *If there is more than one valid choice for X*, choose the one with the alphabetically **smallest** name. **After selecting** X, if there is more than one valid choice for Y, choose the one with the alphabetically **smallest** name. (*Example: "bc" is alphabetically smaller than "fm".)*

In other words, X is either a node that violates the AVL invariant, or a node where adding a new child would cause such a violation. Similarly, Y is either a node where adding a new child restores the AVL invariant, or it is a node that violates the invariant after a new child is added to X. If there are multiple valid choices for X, choose the one with the alphabetically smallest name. If there are multiple valid choices for Y (given your choice of X), choose the one with the alphabetically smallest name.

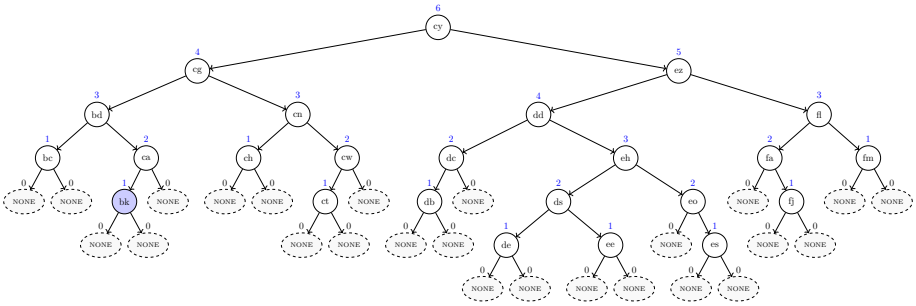
Note that there is exactly one correct solution that satisfies all the conditions above. It is also possible that $X = Y$.

See next page for an explanation of the solution!

Explanation:
The tree satisfies the AVL invariant. There are seven different candidates for X, where adding a node leads to a violation of the AVL invariant:

<u>bk</u>, **ct**, **db**, **de**, **ee**, **es** and **fj**.

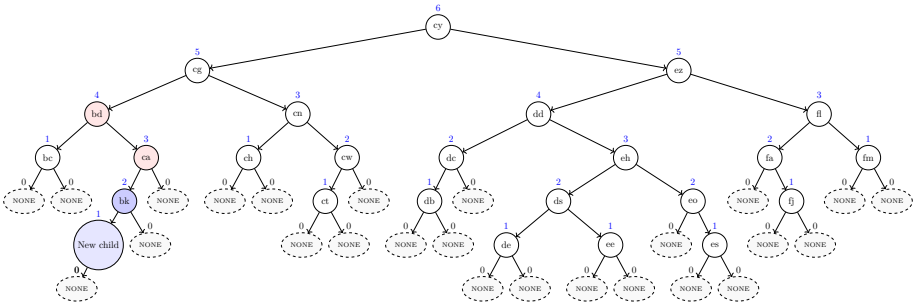
Recall that out of these different possibilties, the task asks you to choose the one where X is the alphabetically smallest node; hence, X = bk.
The first tree shows the heights and dummy nodes, as well as the node $\boldsymbol{X}$ **= bk** (in blue).



The second tree shows the new child (also in blue) that has been added to X as well as all the two violating parent nodes in red (whose children differ in height by more than 1):

ca and **<u>bd</u>**.

Among these violating nodes, the task asks you to choose the alphabetically smallest one; hence $\boldsymbol{Y}$ **= bd**.



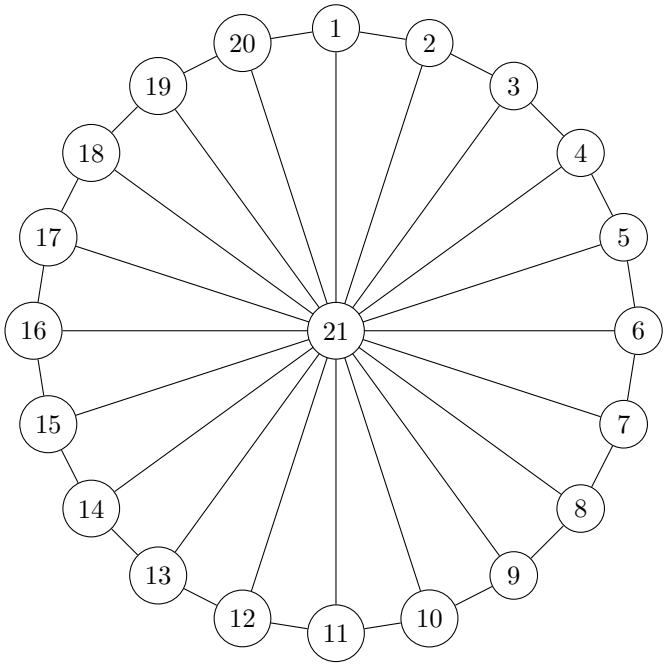
For completness, for each (wrong) candidate for X a list of violating parent nodes. The alphabetically optimal violating parent node is underlined.

- Candidate **ct** →**cw** and **<u>cn</u>**
- Candidate **db** →**<u>dc</u>**
- Candidate **de** →**dd**, **ez** and **<u>cy</u>**
- Candidate **ee** →**dd**, **ez** and **<u>cy</u>**
- Candidate **es** →**eo**, **dd**, **ez** and **<u>cy</u>**
- Candidate **fj** →**<u>fa</u>** and **fl**

11. [5 points] Floyd-Warshall's algorithm (see below) is executed on the following huge undirected graph where the vertices are numbered from 1 to n=21 and each edge has the same weight 1. Observe that the vertices 1 to 20 lie on a big cycle and each of them is connected to the center vertex 21 via an edge.

    Unfortunately, the algorithm as written below has an **error** because it stops the main for loop (line 7) already after 16 iterations.

```
1  δ[1..21][1..21] = new 2D array filled with +∞
2  for i = 1 to 21 do
3      δ[i][i] = 0
4  foreach edge (v, w, c) do
       // v and w are the endpoints and c is the weight of the edge
5      δ[v][w] = c
6      δ[w][v] = c
7  for k = 1 to 16 do
8      foreach pair (v, w) do
9          δ[v][w] = min(δ[v][w],   δ[v][k] + δ[k][w])
10 return δ
```



Write the values of the distance table for the following cells as computed by the **erroneous** algorithm above. That is, we still assume that the loop in line 7 iterates **only 16 times**!

Recall that the weight of every edge is equal to 1.

*If the distance is **infinite**, write "INF". Note: There are exactly two fields where the correct answer is "INF". If you write "INF" in more than two fields, you get 0 points for those fields.*

$\delta[20][17] =$ **17** $\quad\quad \delta[5][14] =$ **9**

$\delta[19][21] =$ **1** $\quad\quad \delta[19][16] =$ **INF**

$\delta[6][18] =$ **INF**

12. [4 points] Consider the following algorithm.

Input: integer l
$z = 2$
$e = 0$
while $l > l$ **do**
  $e = e + 6$
  $g = -4$
  $z = z + g$
  $e = e + g$
  $z = z + 18$

Someone wrote the following invariant for the loop:

$$Y \cdot z = U \cdot e + J$$

with the three constants

- $Y = 8$
- $U = 42$
- $J = 12$

The invariant should hold each time the algorithm evaluates the loop condition. Unfortunately, the invariant has a mistake: one of the three constants, Y, U, or J has a wrong value.

Fix the invariant by changing the value of one of the constants!

Which constant do you choose? _____**Y**_____ *(Write the name of one of the three constants.)*

The new value for the chosen constant: _____**6**_____

Note: You are allowed to change only one constant.

13. 4 points (bonus)

Consider the following decision problem A:

Your network administrator has n Boolean firewall rules, each can be enabled or disabled. The overall security policy is given as a complex logical formula over these rule variables using "and", "or", and "not". Is there an assignment of enable/disable values that makes the policy true?

Consider the following decision problem B:

You're organizing a tour of n meeting rooms connected by corridors for a conference. You need to start at the lobby, visit the keynote hall, and return to the lobby, with no room visited more than twice. Is such a tour possible?

Mark all the correct sentences and give a short explanation.

- ◯ A can be reduced to B in polynomial time.
- √ **B can be reduced to A in polynomial time.**
- ◯ A is in P.
- √ **B is in P.**
- √ **It is unknown whether A can be reduced to B in polynomial time.**
- ◯ It is unknown whether B can be reduced to A in polynomial time.
- √ **It is unknown whether A is in P.**
- ◯ It is unknown whether B is in P.

Solution: Problem A is equivalent to the NP-complete problem SAT (by modeling each rule as a Boolean variable, the security policy as a logical Boolean SAT formula, and asking whether the formula is satisfiable), whereas problem B can be solved in polynomial time: we interpret rooms as vertices and corridors as edges; the question reduces to whether there exists a simple path connecting the lobby and the keynote hall (walking this path forth and back gives us a tour visiting each room at most twice); reachability can be checked in polynomial time via breadth-fist search (BFS).

Consequently, as problem B can be solved in polynomial time, it can reduced in polynomial time to any other problem, thus also to A.

Since it is unknown whether NP-complete problems can be solved in polynomial time, we do not know whether A is in P, and, consequently, whether there exist a polynomial time reduction from A to any polynomial-time problem (e.g., to B).